



CS 610 GPU Thread Model

Jieyang Chen

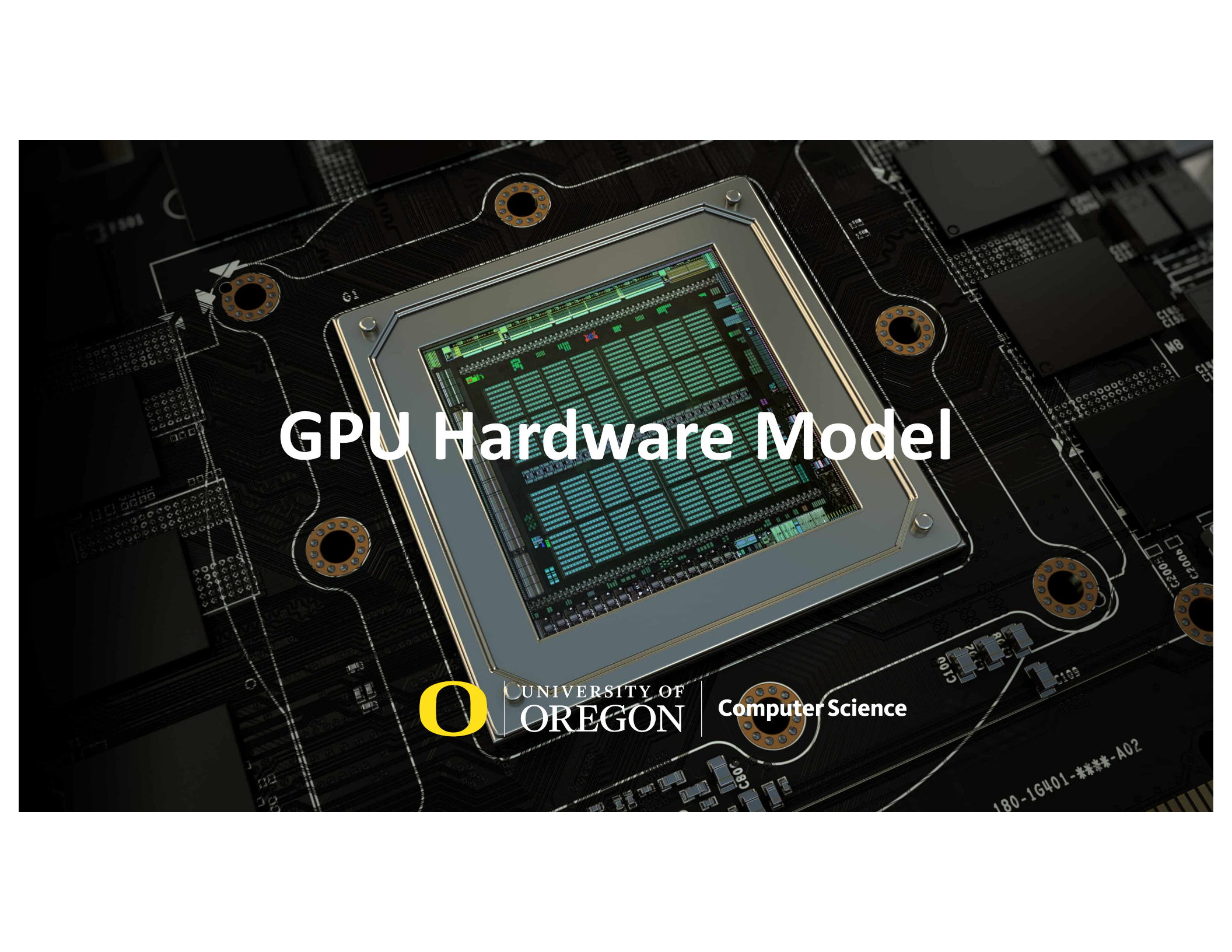


UNIVERSITY OF
OREGON

Computer Science

Reading Assignments

- Programming Massively Parallel Processors
 - Chapter 2 and 3



GPU Hardware Model



UNIVERSITY OF
OREGON

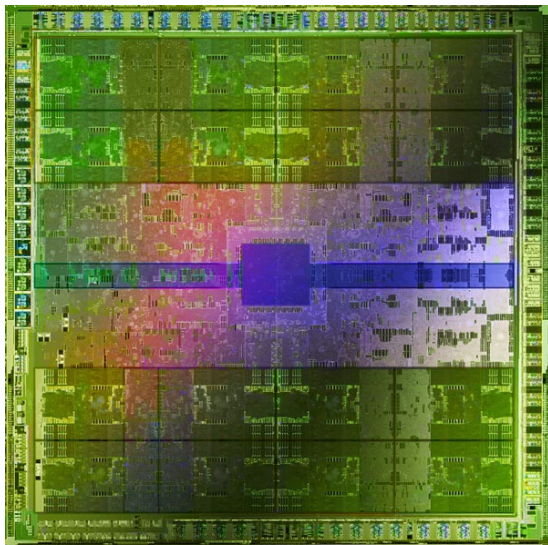
Computer Science

GPU Hardware Model

- Building an abstraction of the hardware architecture by keeping only the important parts to help us learn how GPU hardware works.
- Depending on which parts of the architecture we focus on, we can design different hardware model for different propose.

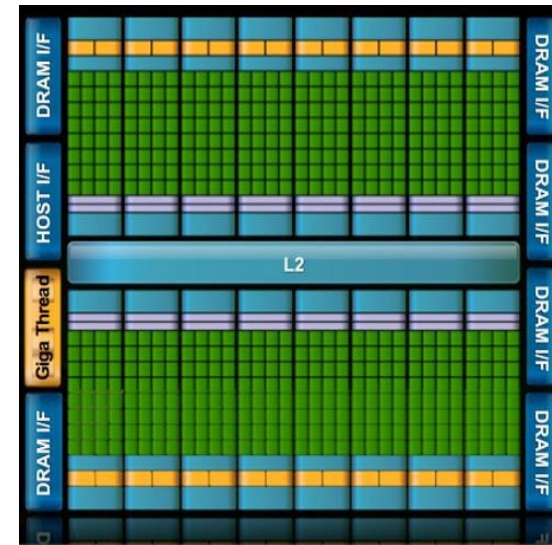
GPU Hardware Model

- Building an abstraction of the hardware architecture by keeping only the important parts to help us learn how GPU hardware works.



GPU Chip

Abstraction



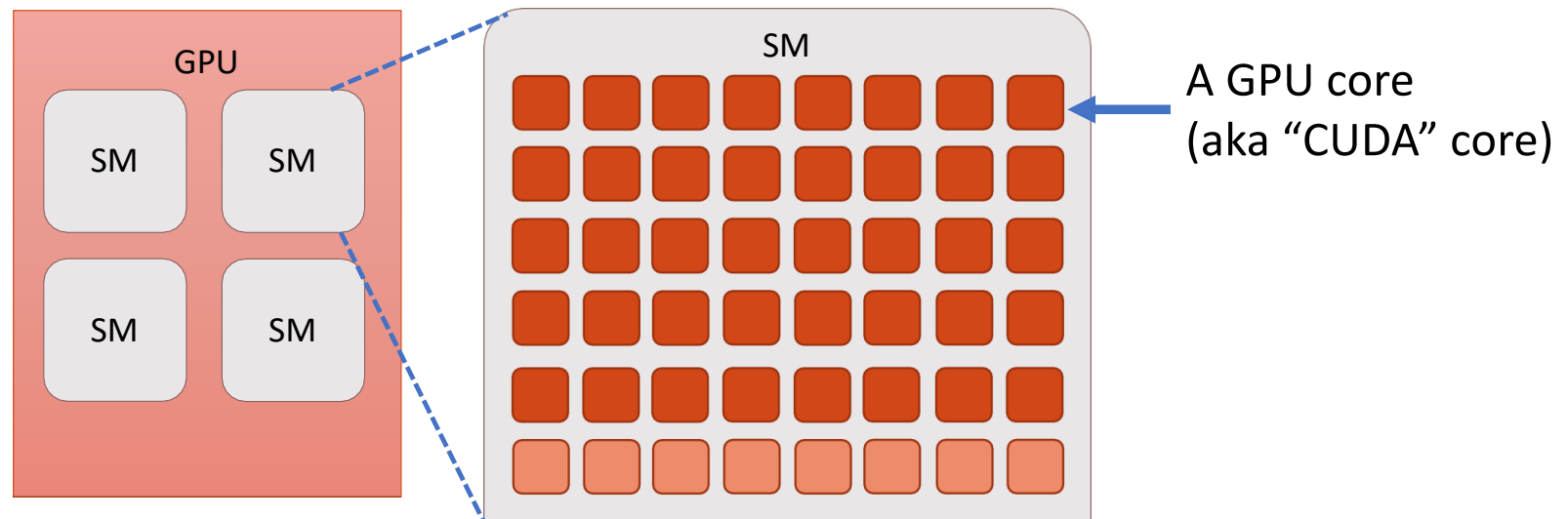
GPU Hardware Model

Hardware Model

GPUs have a 2-level hierarchy

- Each GPU chip has a number of Streaming Multiprocessor (SM)
- Each SM has a number of GPU cores (or “CUDA” cores)

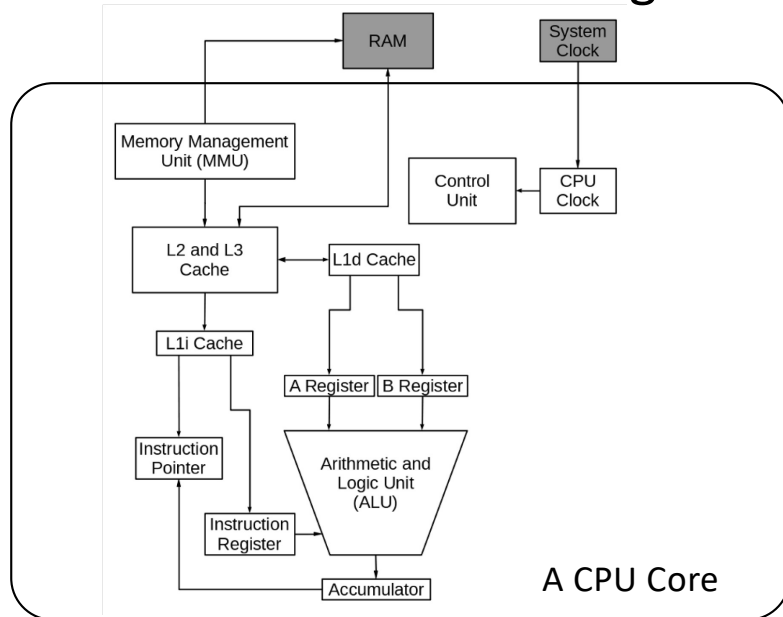
$$\text{Total Parallelism} = \text{Number of SMs} \times \text{Number of “CUDA” cores per SM}$$



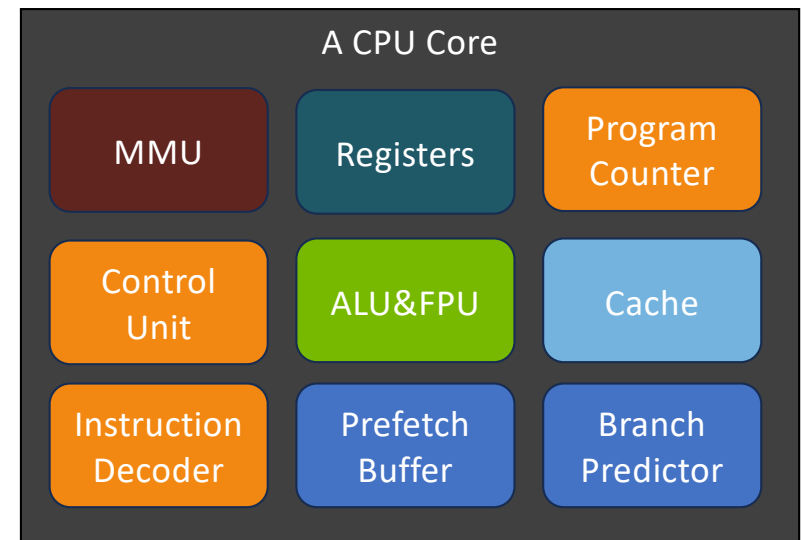
What exactly are GPU cores?

- Let's first check on a traditional CPU core
- Job of a core: executing instructions

Fetch → Decode → Execute

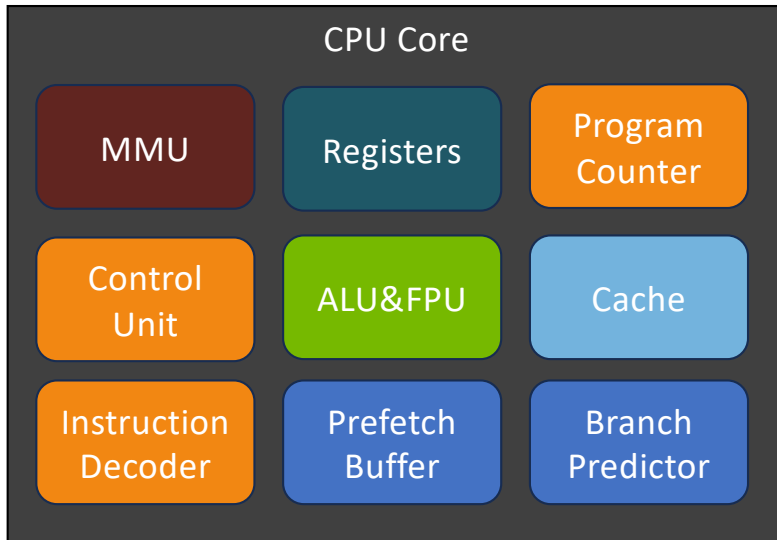


Abstraction



UNIVERSITY OF
OREGON

Computer Science



Three types of cores in a SM in A100

- CUDA cores are a mixture of
 - 32 INT units
 - 64 FP32 units
 - 32 FP64 units
- 4 Tensor core units
- 4 Special function units

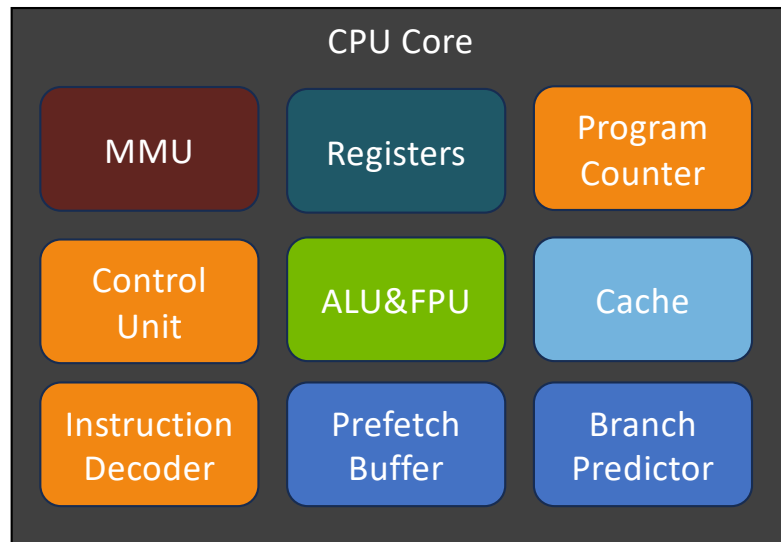


UNIVERSITY OF
OREGON

Computer Science

GPU SM





GPU SM



A GPU core is only for instruction execution $\approx$ CPU ALU/FPU

A GPU SM perform instruction fetch, decoding, caching, execution $\approx$ CPU core

Parallelism: CPU vs. GPU

CPU

- number of CPU cores (up to hundreds)

GPU

- Number of "CUDA" cores (tens to a hundred) * number of SMs (tens to hundreds)
= tens of thousand

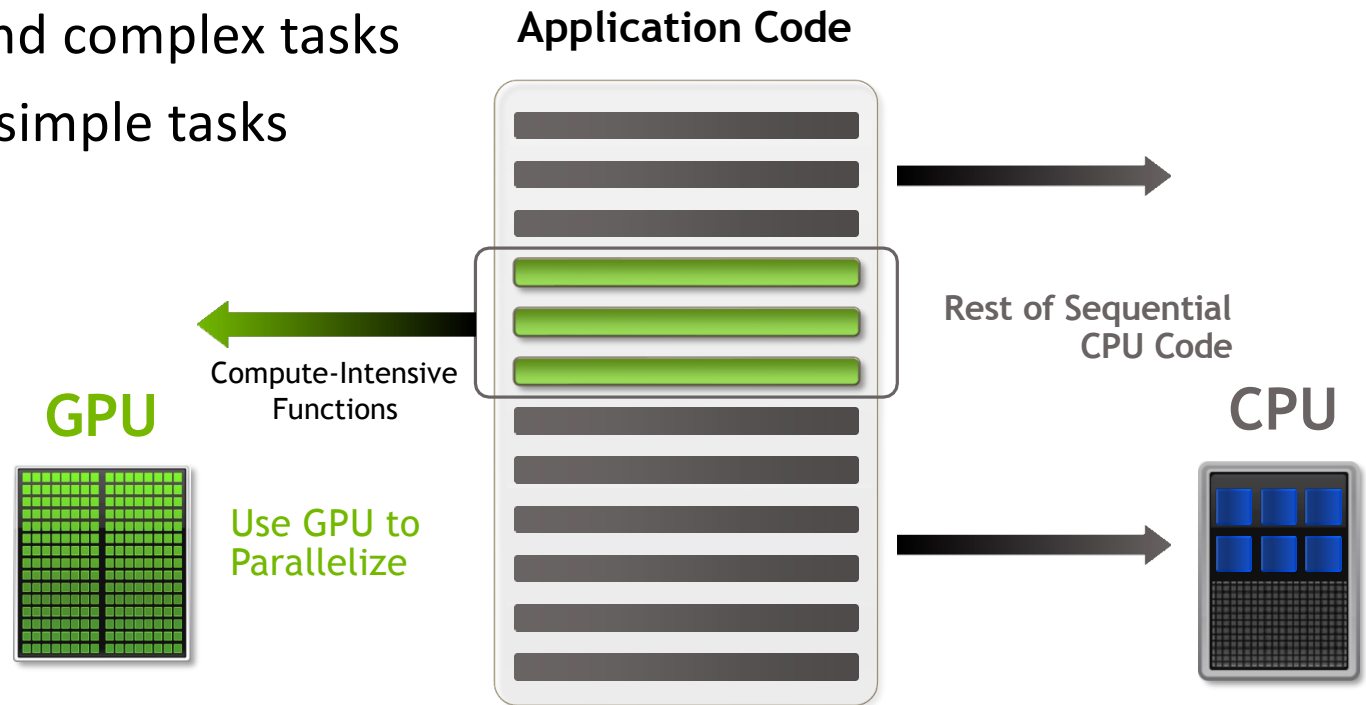
Resource per execution unit: CPU vs. GPU

- Execution unit
 - CPU: ALU/FPU (tens to hundreds)
 - GPU: “CUDA” core (tens of thousand)
- Cache:
 - CPU: A few to tens of MB cache per CPU
 - GPU: A few to tens of MB cache per GPU
- Memory bandwidth
 - CPU: a few hundred GB/s per CPU
 - GPU: a few TB/s GPU
- Control unit
 - CPU: one set per ALU/FPU
 - GPU: a few per SM (hundreds of cores)

On average, each execution unit of GPU has much less cache, memory bandwidth, and control unit resource

Best task: CPU vs. GPU

- CPU: sequential and complex tasks
- GPU: parallel and simple tasks

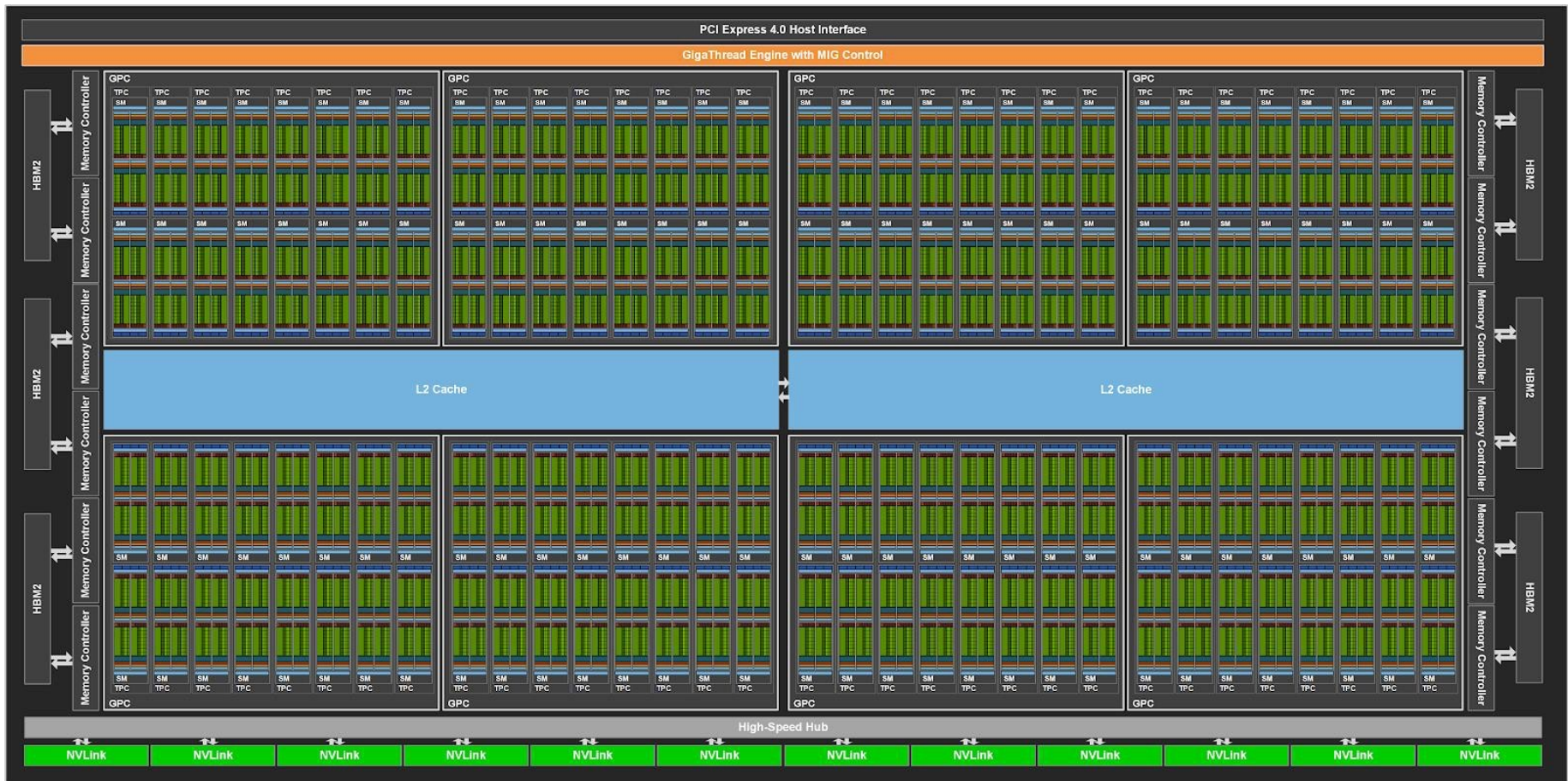


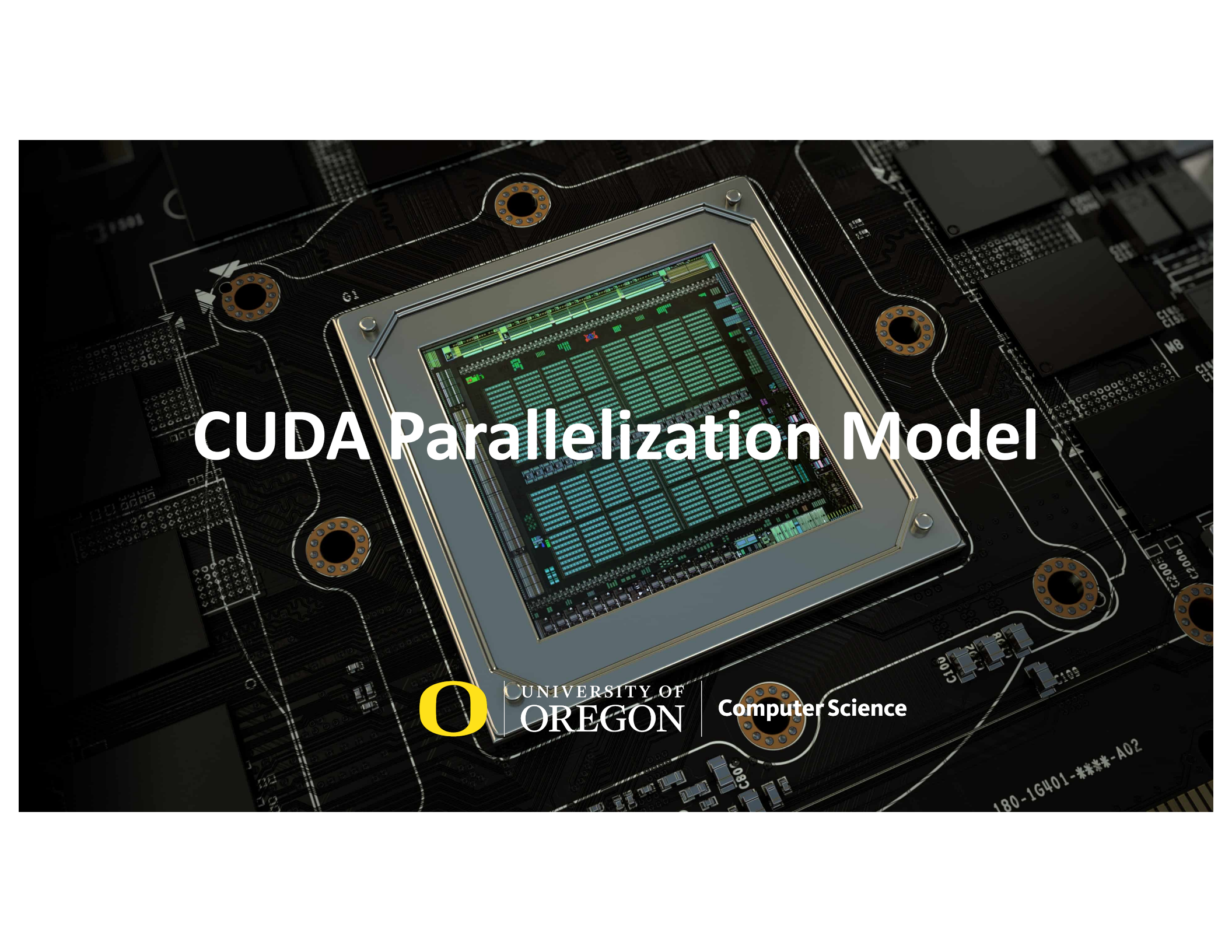
What are changing across generations?

GPU generations & Model	Kepler K40	Maxwell M40	Pascal P100	Volta V100	Ampere A100	Hopper H100
CUDA Cores per SM	192	128	64	64	64	128
Tensor Cores per SM	0	0	0	8	4	4
Total number of SM	15	24	56	80	108	132
Total CUDA Cores	2880	3072	3584	5120	6912	16,896
Total tensor Cores	0	0	0	640	432	528
Single Precision Performance	4.29 Tflops	7.0 Tflops	9.5 Tflops	15.4 Tflops	19.5 Tflops	60 Tflops
Double Precision Performance	1.43 Tflops	0.21 Tflops	4.7 Tflops	7.8Tflops	9.7 Tflops	60 Tflops
Memory Bandwidth	288 GB/s	288 GB/s	720 GB/s	900 GB/s	1555 GB/s	3900 GB/s
Memory Size	12 GB	12GB	16GB	32GB	40 GB	96 GB



Ampere A100 (108 SMs, 6912 FP32, 3456 FP64, 432 Tensor Cores)





CUDA Parallelization Model

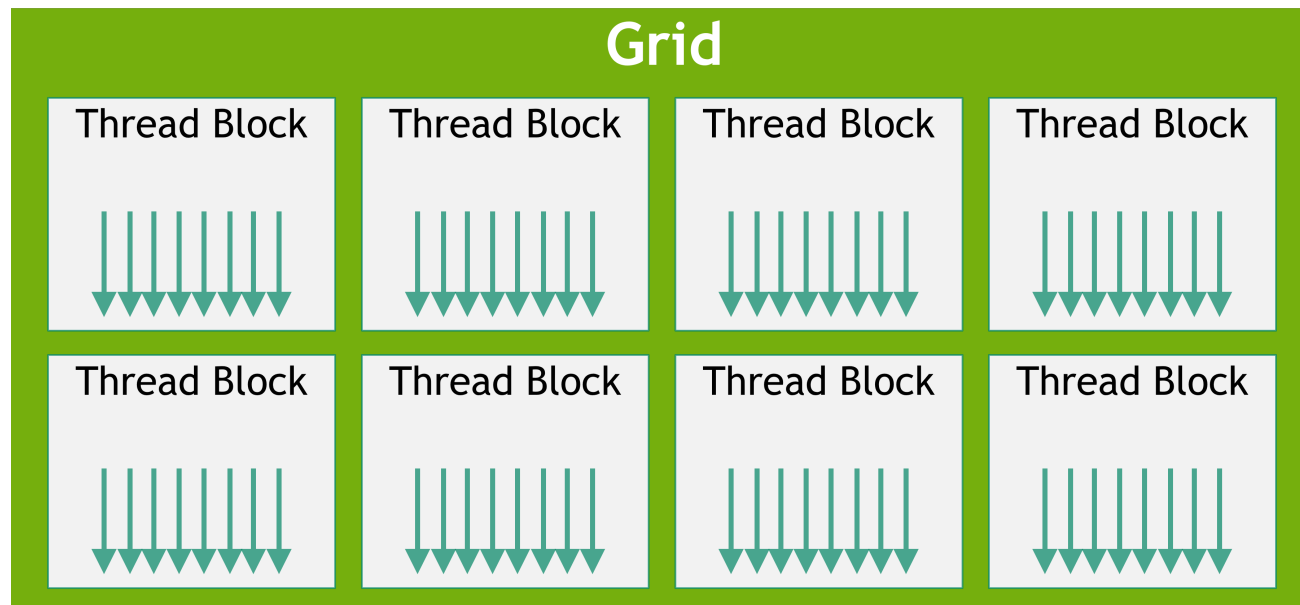


UNIVERSITY OF
OREGON

Computer Science

CUDA Parallelization Model

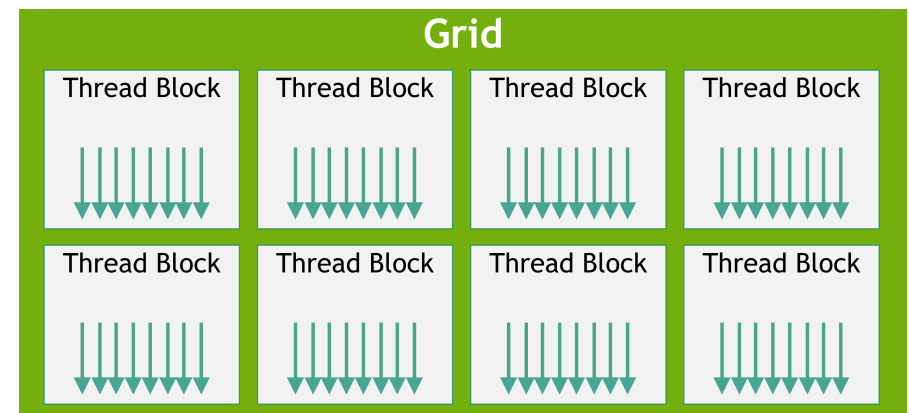
❑ Two-level parallelization model



CUDA Parallelization Model

- ❑ Two configuration parameters:
 - ❑ Number of thread per thread block (TB)
 - ❑ Number of thread block
- ❑ All thread block must have **same** number of threads

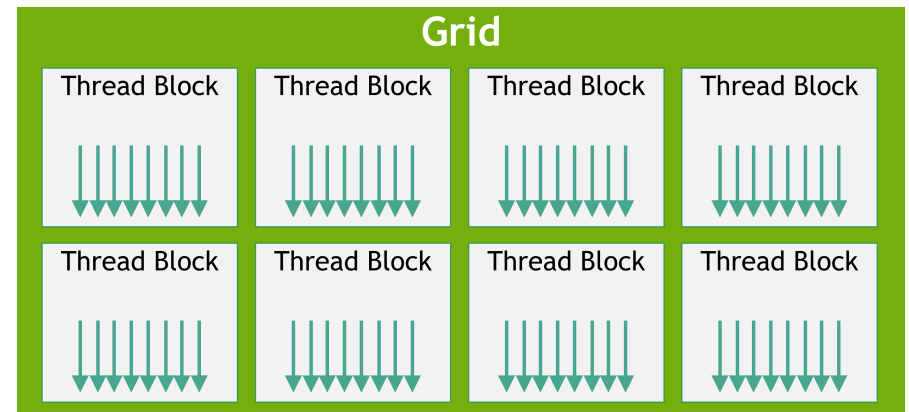
Example: 8 threads per thread block * 8 thread blocks



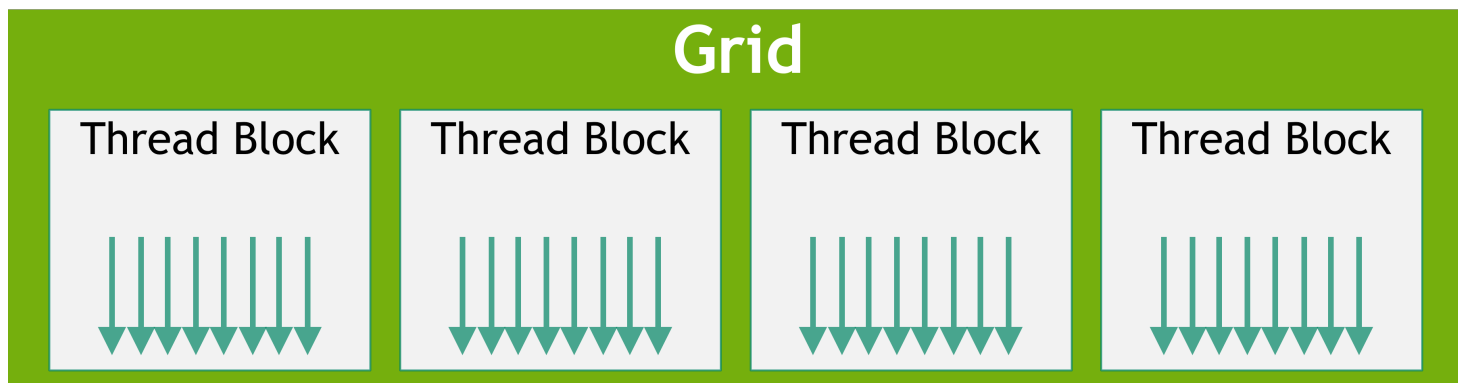
Total number of threads = Number of thread per thread block * Number of thread block

CUDA Parallelization Model

- ❑ Both grid and thread block can be organized in 1D, 2D, or 3D.
- ❑ A grid and thread blocks can have different numbers of dimensions
- ❑ For example, 1D thread blocks group into 2*4 grids



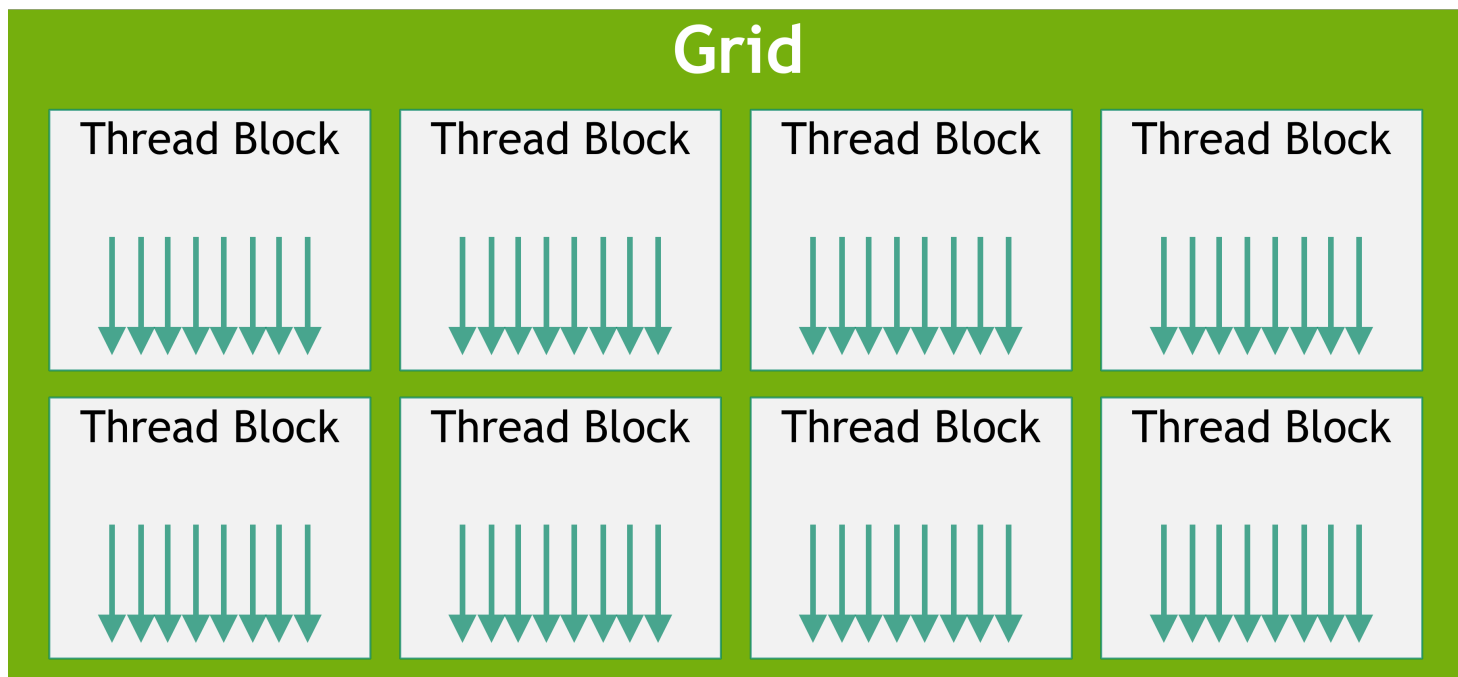
CUDA Parallelization Model



1D grid ($1 \times 1 \times 4$)

1D thread block ($1 \times 1 \times 8$)

CUDA Parallelization Model



2D grid (1*2*4)

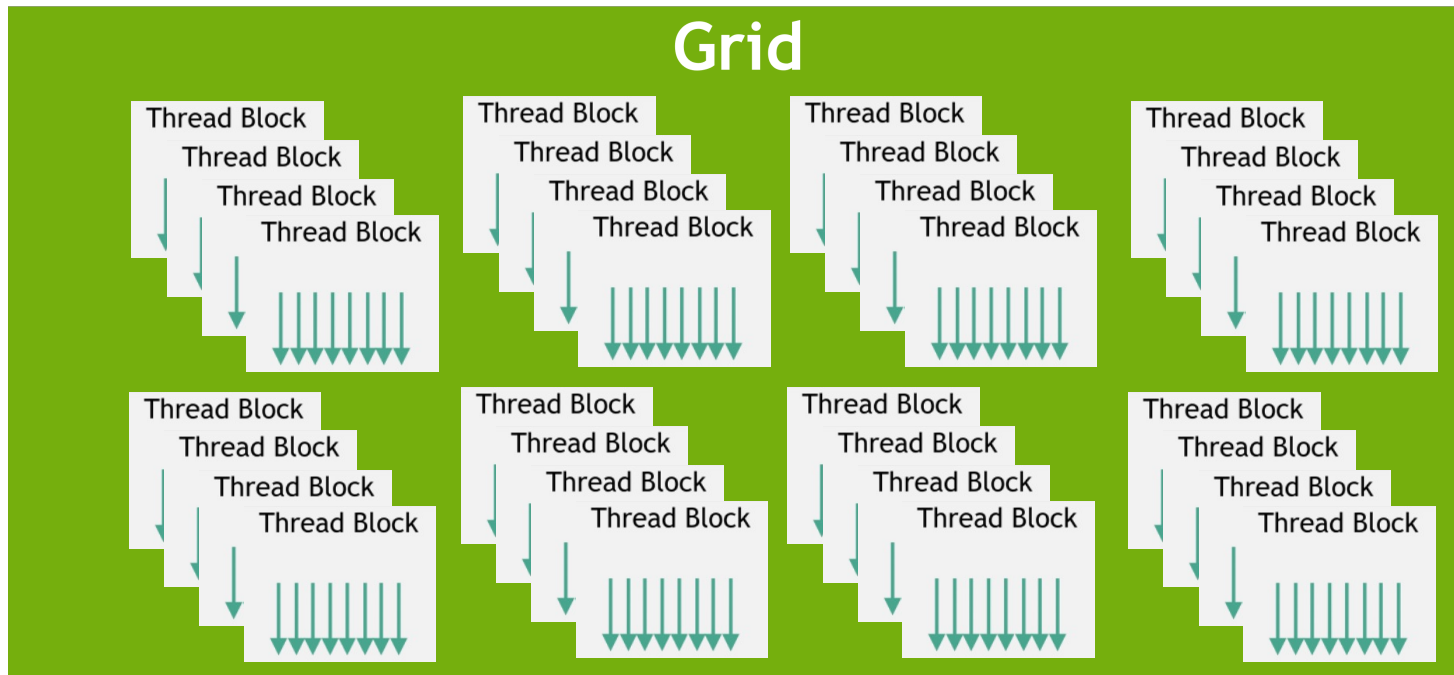
1D thread block (1*1*8)



UNIVERSITY OF
OREGON

Computer Science

CUDA Parallelization Model



3D grid ($2 \times 4 \times 4$)

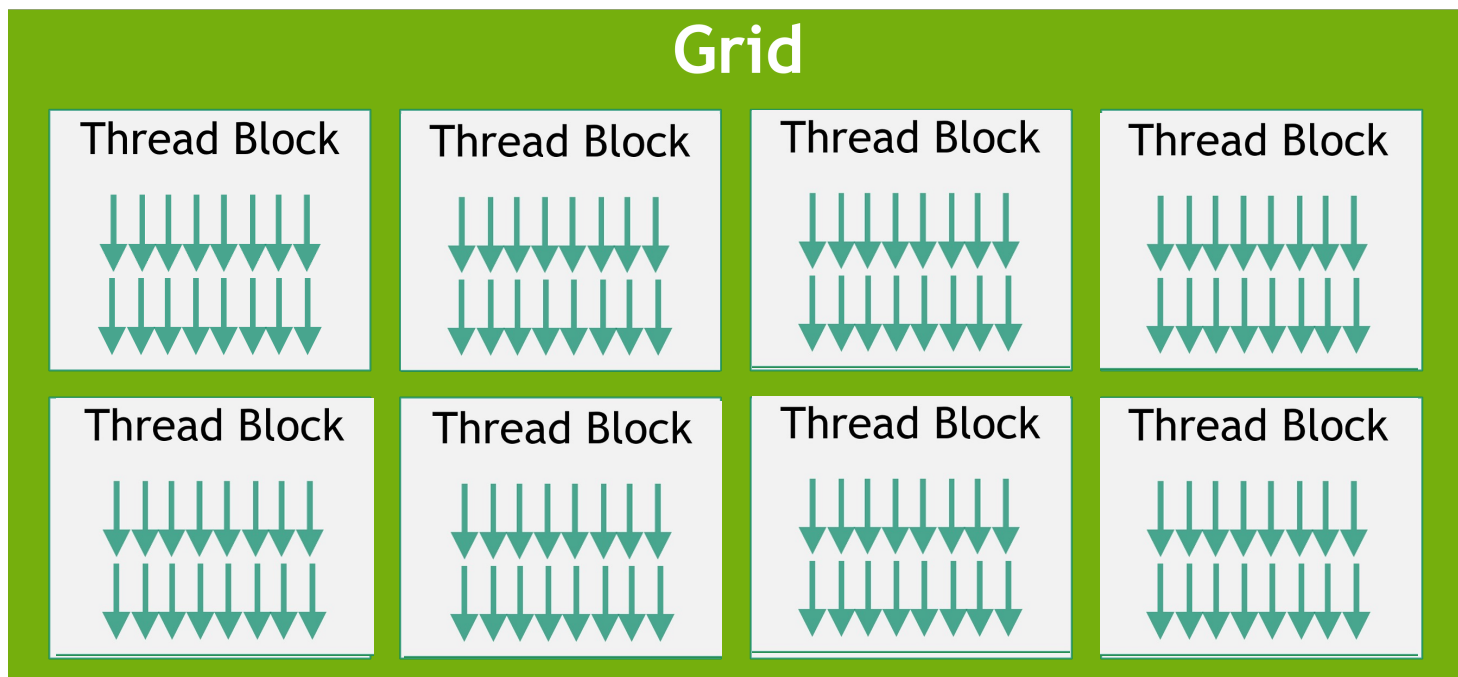
1D thread block ($1 \times 1 \times 8$)



UNIVERSITY OF
OREGON

Computer Science

CUDA Parallelization Model



2D grid (1*2*4)

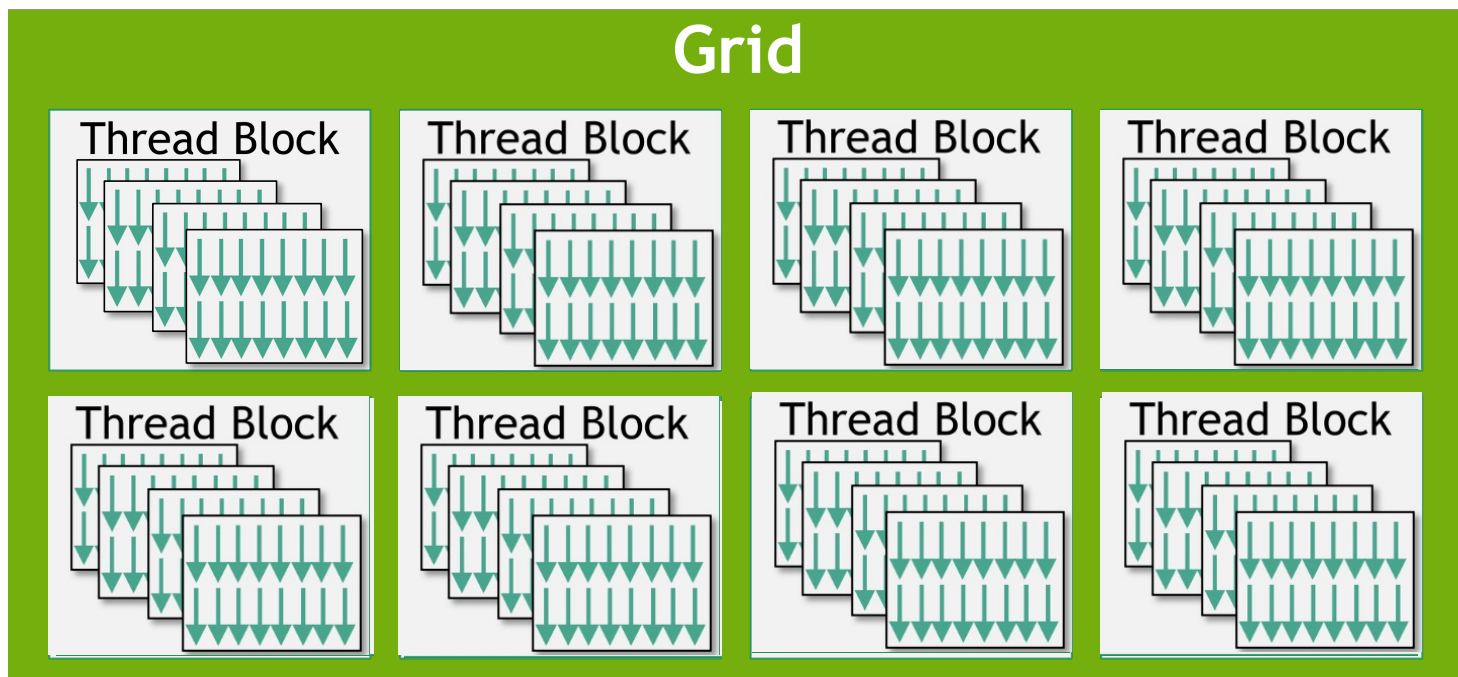
2D thread block (1*2*8)



UNIVERSITY OF
OREGON

Computer Science

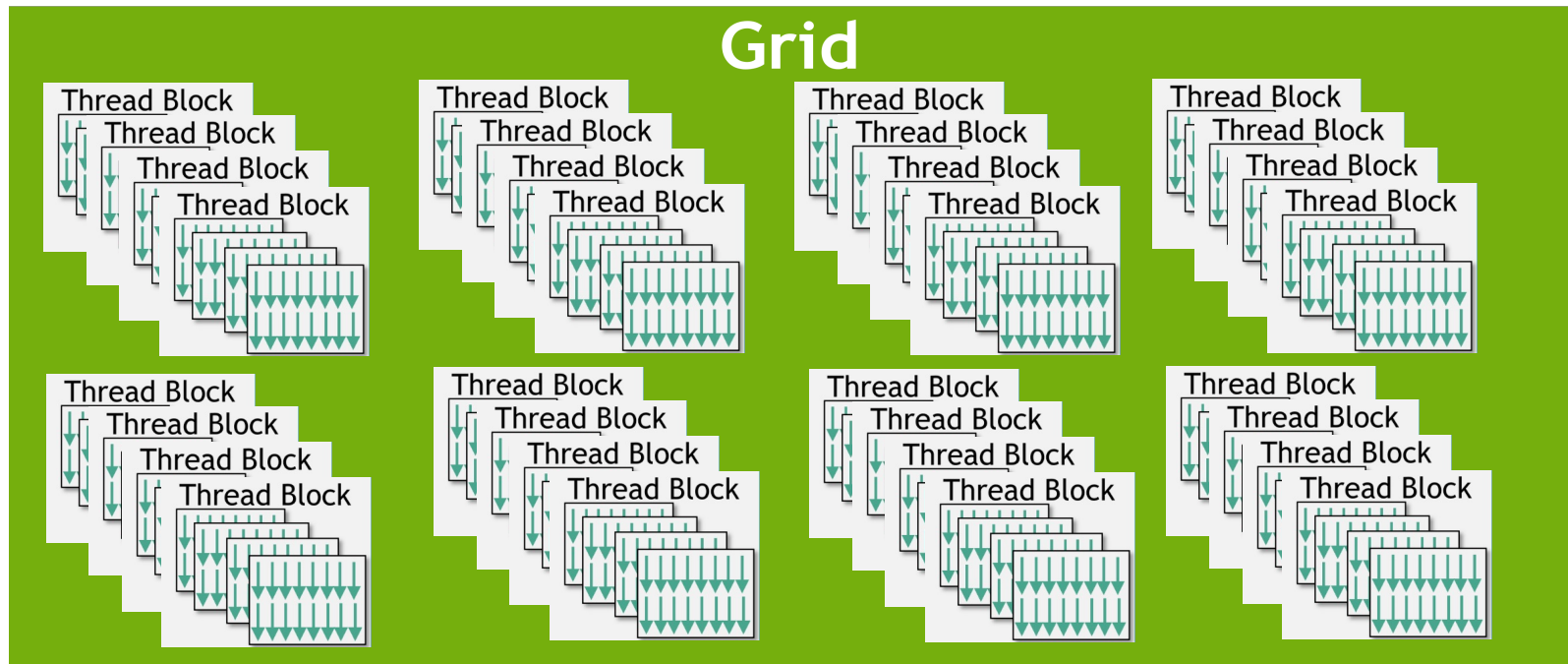
CUDA Parallelization Model



2D grid ($1 \times 2 \times 4$)

3D thread block ($4 \times 2 \times 8$)

CUDA Parallelization Model



3D grid ($2 \times 2 \times 4$)

3D thread block ($4 \times 2 \times 8$)

CUDA Parallelization Model

Maximum dimensionality of grid of thread blocks	3
Maximum x -dimension of a grid of thread blocks	$2^{31}-1$
Maximum y- or z-dimension of a grid of thread blocks	65535
Maximum dimensionality of thread block	3
Maximum x- or y-dimensionality of a block	1024
Maximum z-dimension of a block	64
Maximum number of threads per block	1024

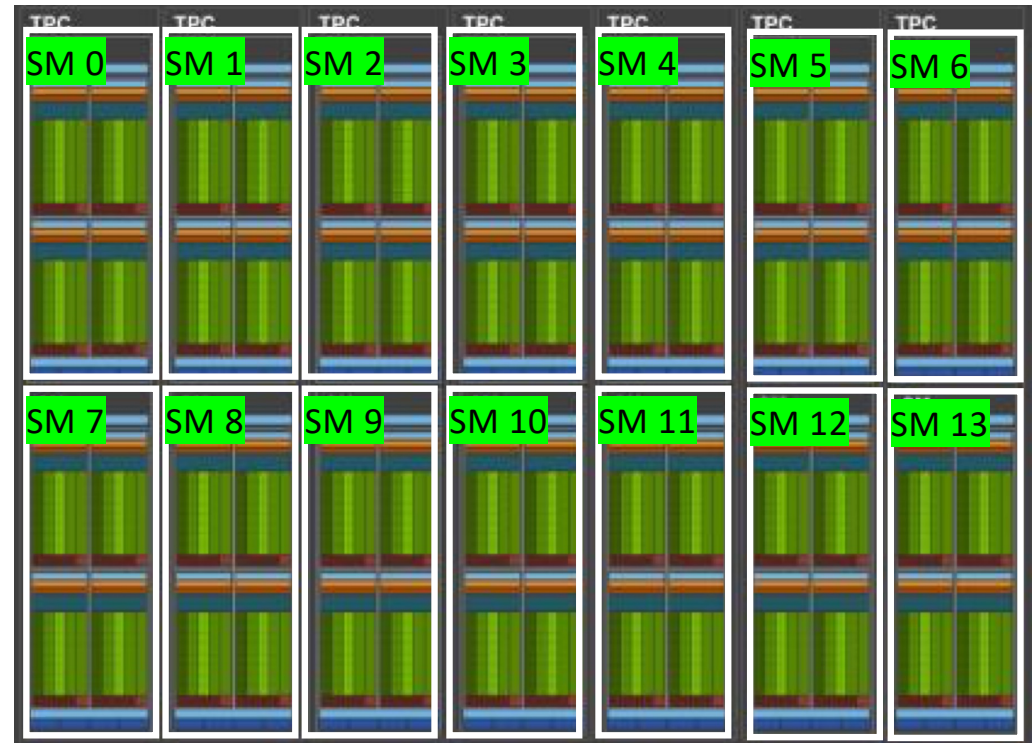
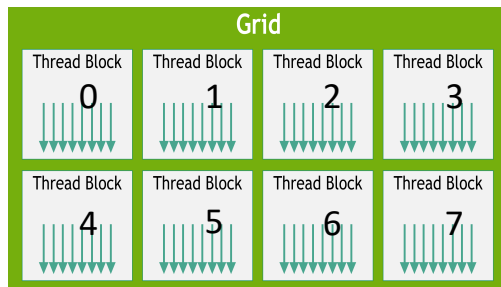


Mapping parallelization model to hardware

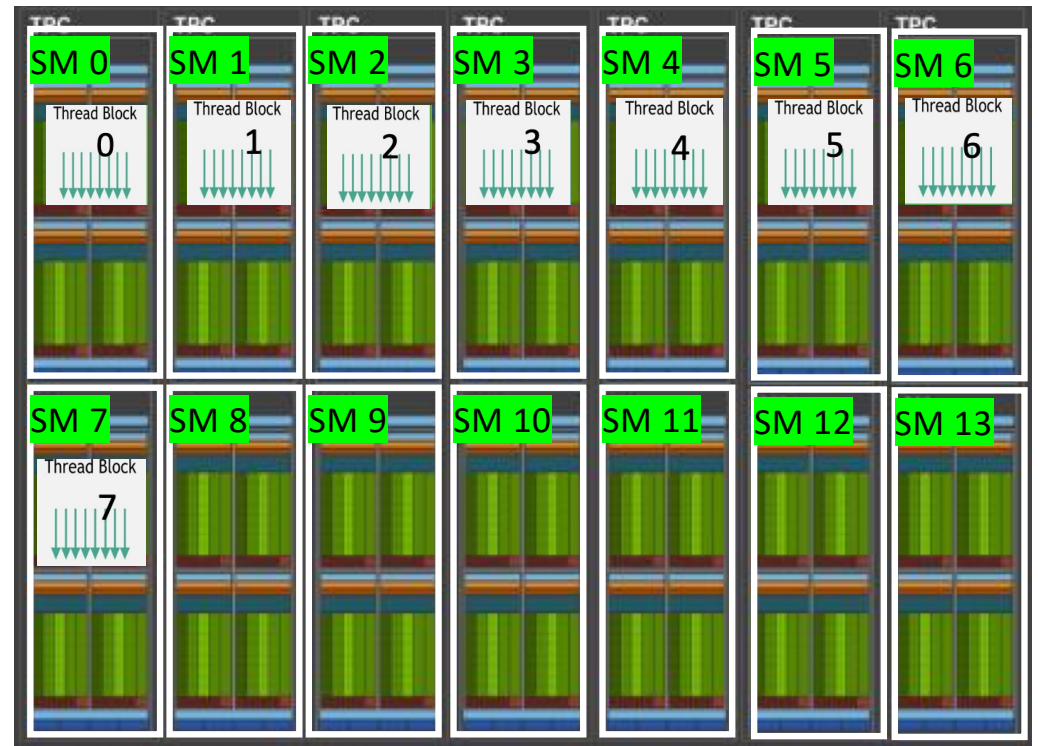
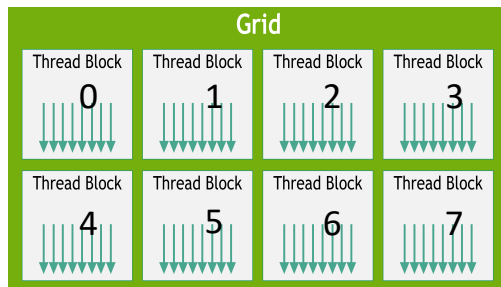
Mapping rules (done by scheduler)

- A thread block is mapped to a stream multiprocessor (SM)
- Thread blocks will reside on their SMs for the **entire lifecycle** (no switching in between SMs)
- The entire lifecycle of a thread block begins from the moment it is assigned to a SM until all its threads have finished execution
- One thread block **CANNOT** be split and assigned to multiple SMs
- Multiple thread blocks may **share** a same SM

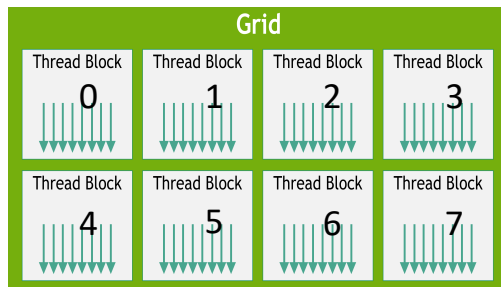
Mapping parallelization model to hardware



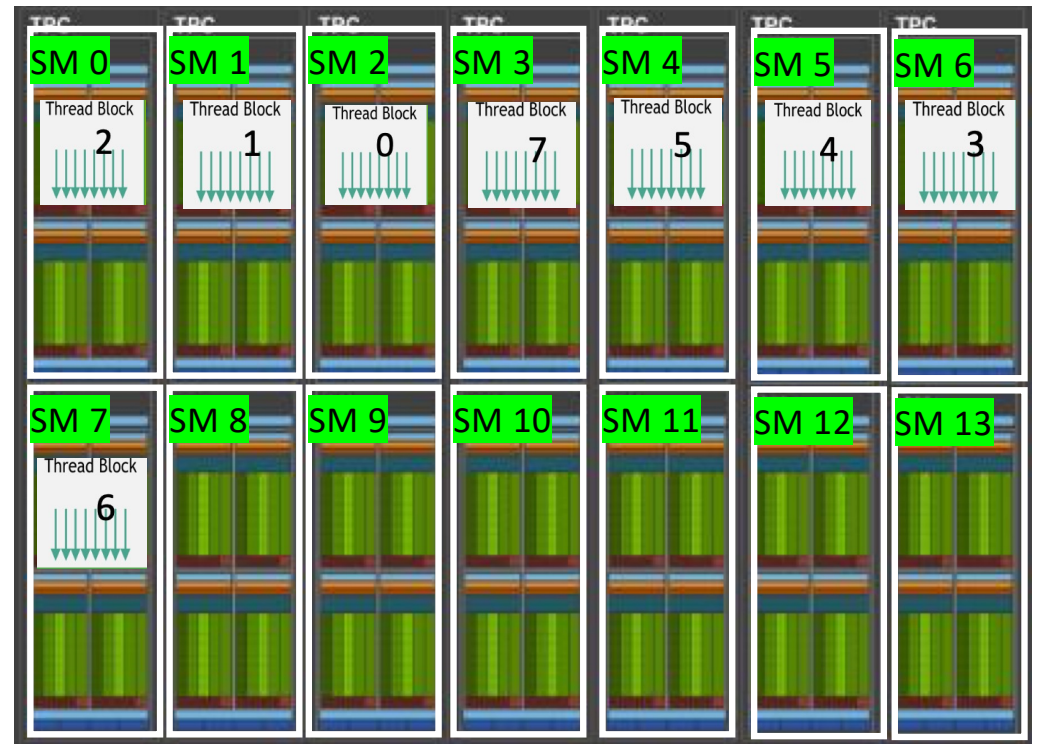
Mapping parallelization model to hardware



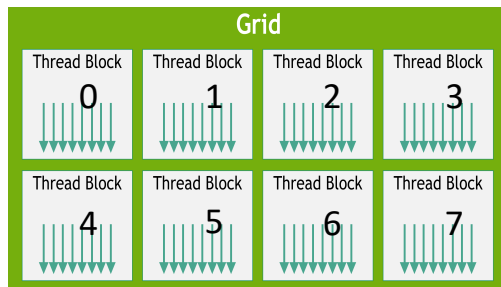
Mapping parallelization model to hardware



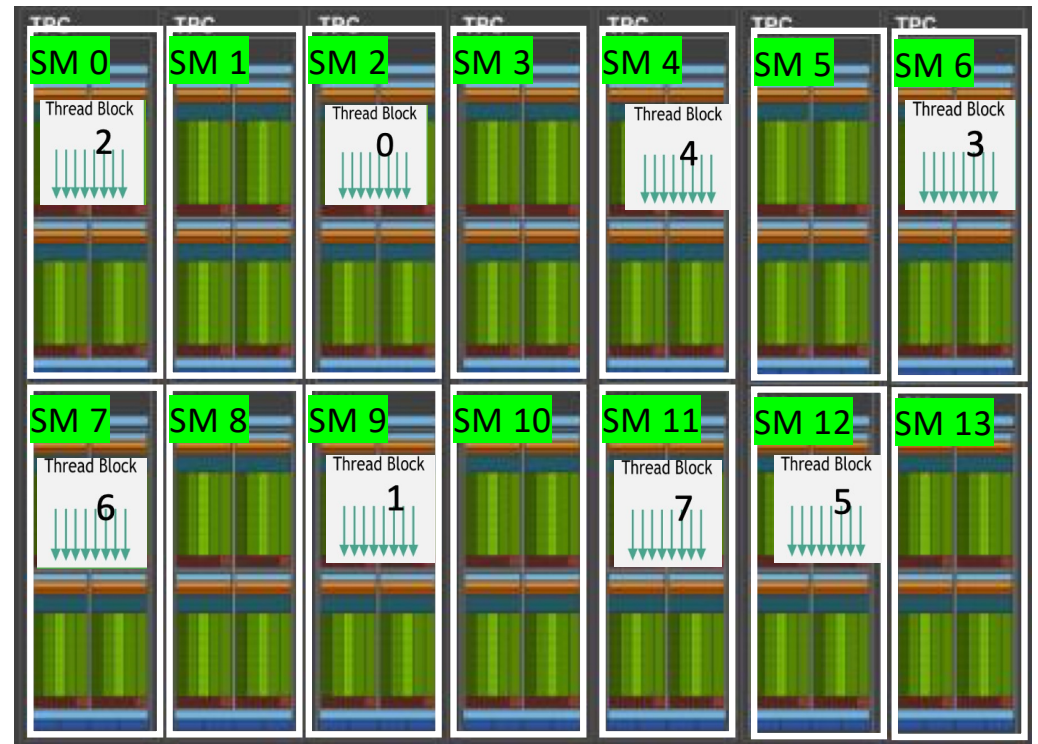
- Thread blocks may be assigned SMs in any random order
- Do **not** assume that thread blocks will be mapped in a certain order



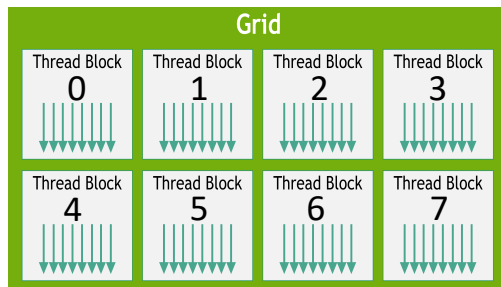
Mapping parallelization model to hardware



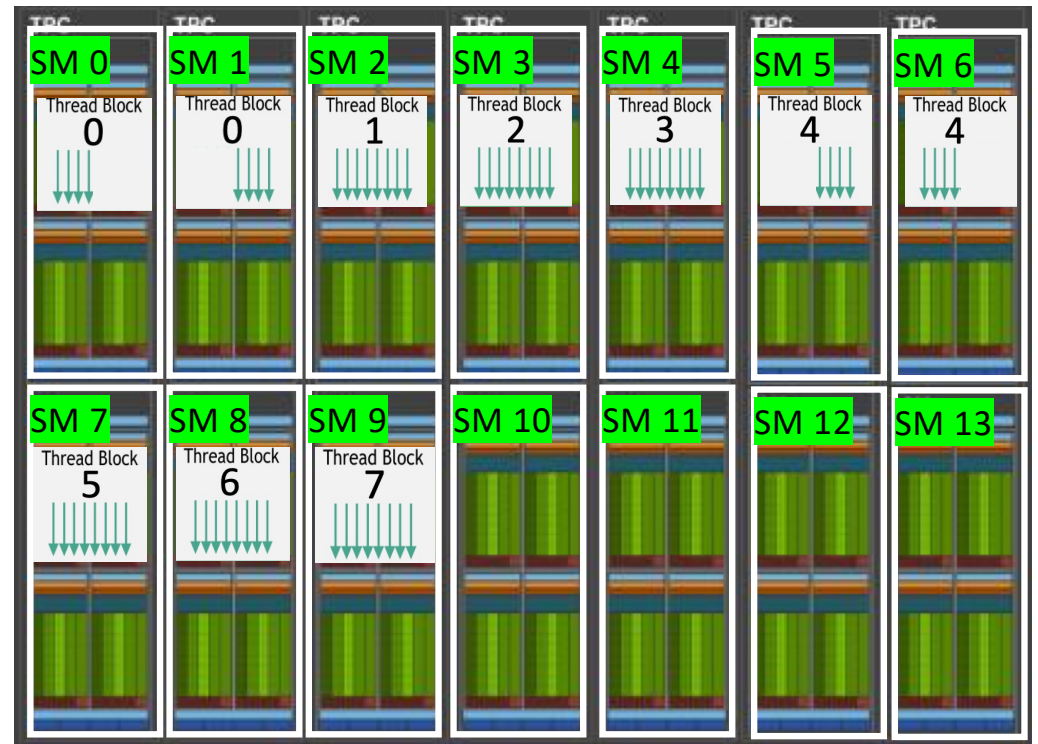
- Thread blocks may be assigned SMs in any random order
- Do **not** assume that thread blocks will be mapped in a certain order



Mapping parallelization model to hardware

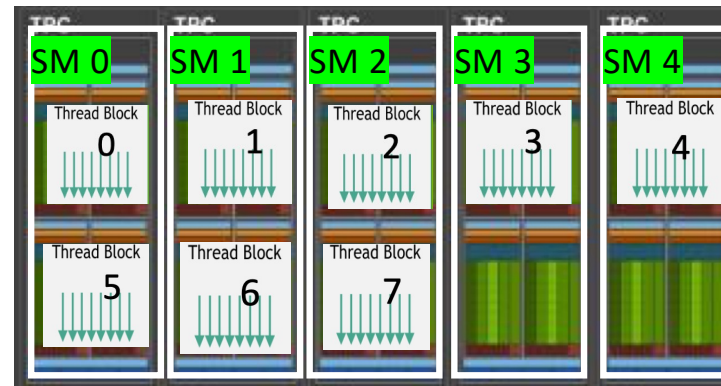
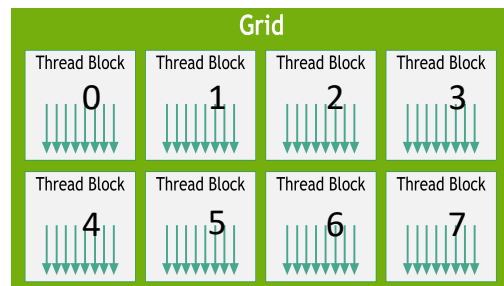


Not a valid mapping!



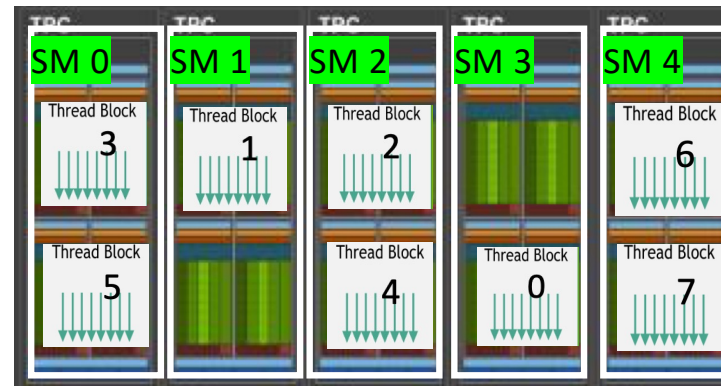
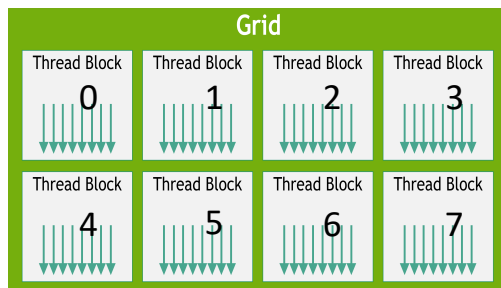
A thread block **CANNOT** be split and assigned to multiple SMs
UNIVERSITY OF OREGON | Computer Science

Mapping parallelization model to hardware



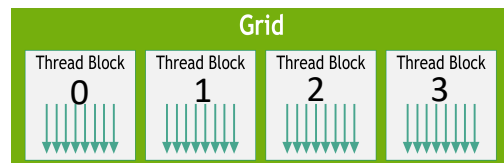
Multiple thread blocks may **share** a same SM

Mapping parallelization model to hardware

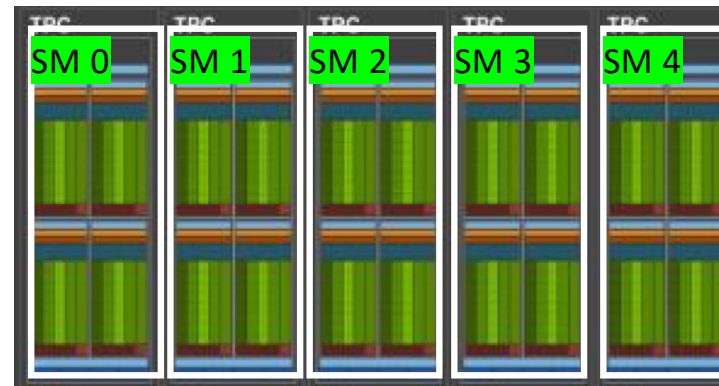


Again, do **not** assume that thread blocks will be mapped in a certain order
Also, do **not** assume how multiple thread blocks may share a same SM

Mapping parallelization model to hardware



Time: t_0

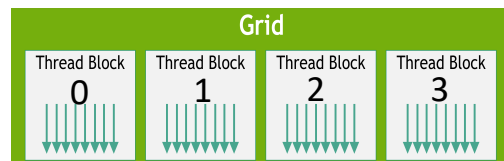


Thread blocks are executed independently

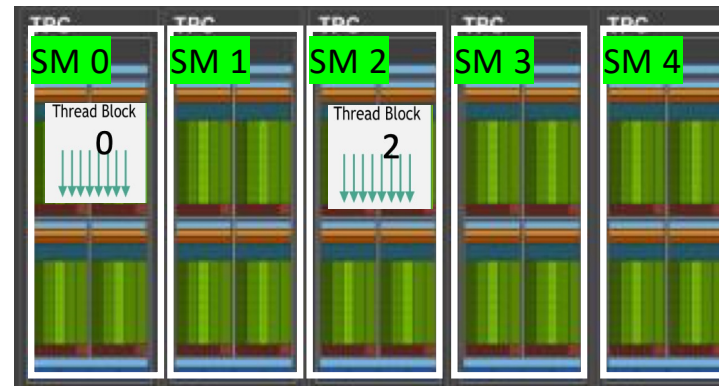
Depending on the scheduler, some thread blocks may start early or late

Depending on the workload, some thread blocks may finish early or late

Mapping parallelization model to hardware



Time: t_1

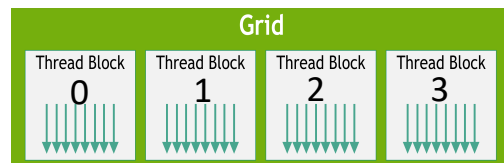


Thread blocks are executed independently

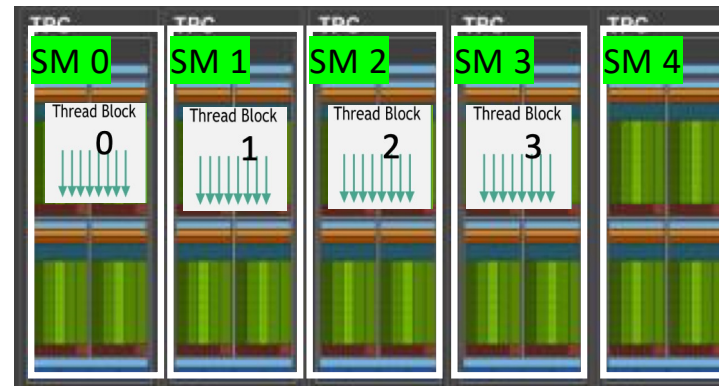
Depending on the scheduler, some thread blocks may start early or late

Depending on the workload, some thread blocks may finish early or late

Mapping parallelization model to hardware



Time: t_2

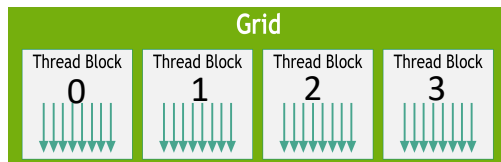


Thread blocks are executed independently

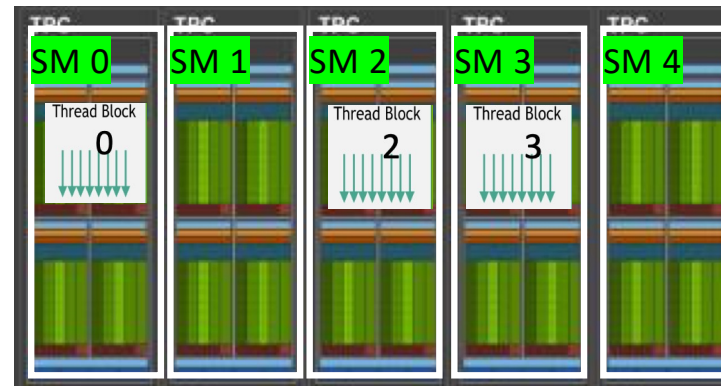
Depending on the scheduler, some thread blocks may start early or late

Depending on the workload, some thread blocks may finish early or late

Mapping parallelization model to hardware



Time: t3

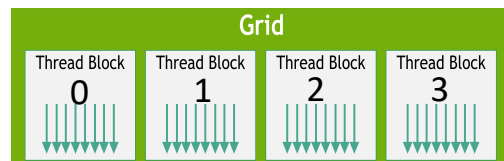


Thread blocks are executed independently

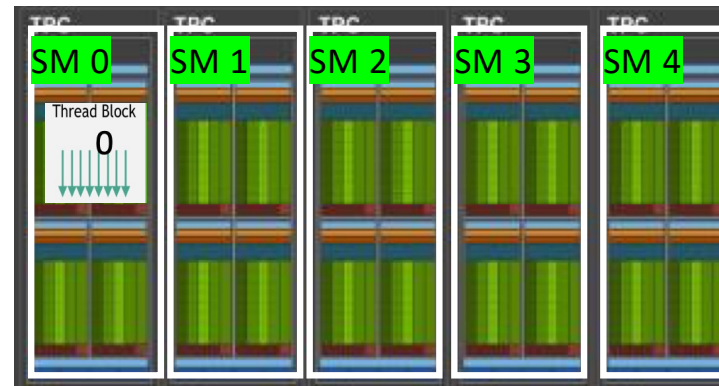
Depending on the scheduler, some thread blocks may start early or late

Depending on the workload, some thread blocks may finish early or late

Mapping parallelization model to hardware

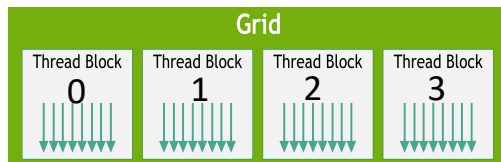


Time: t_4

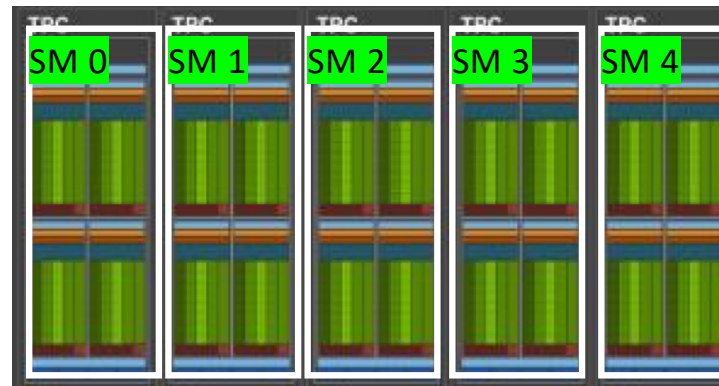


Thread blocks are executed independently
Depending on the scheduler, some thread blocks may start early or late
Depending on the workload, some thread blocks may finish early or late

Mapping parallelization model to hardware



Time: t_5

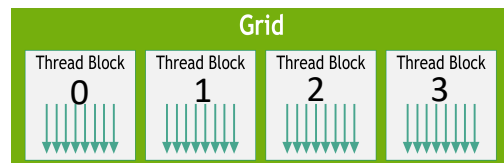


Thread blocks are executed independently

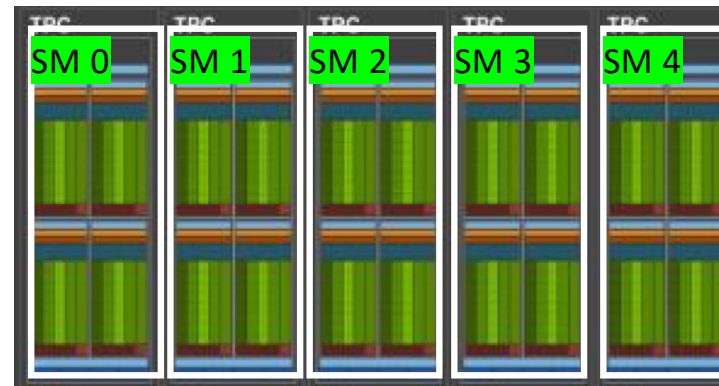
Depending on the scheduler, some thread blocks may start early or late

Depending on the workload, some thread blocks may finish early or late

Mapping parallelization model to hardware

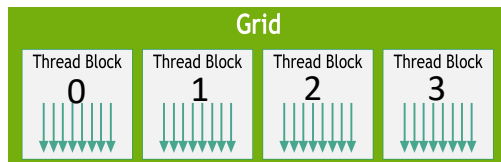


Time: t_0

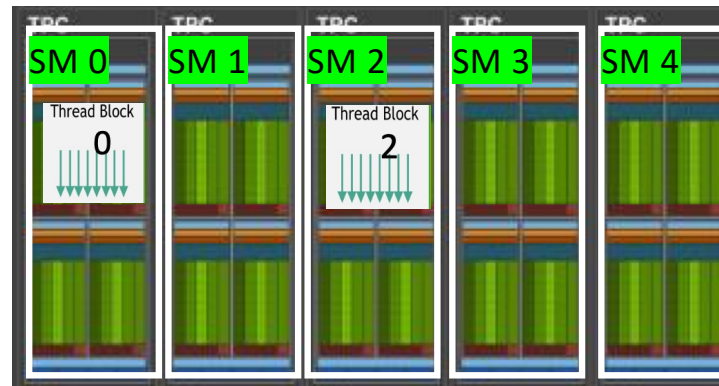


Thread blocks will reside on their SMs for the **entire lifecycle** (no switching in between SMs)

Mapping parallelization model to hardware

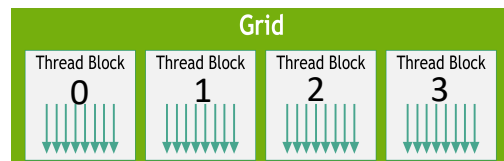


Time: t_1

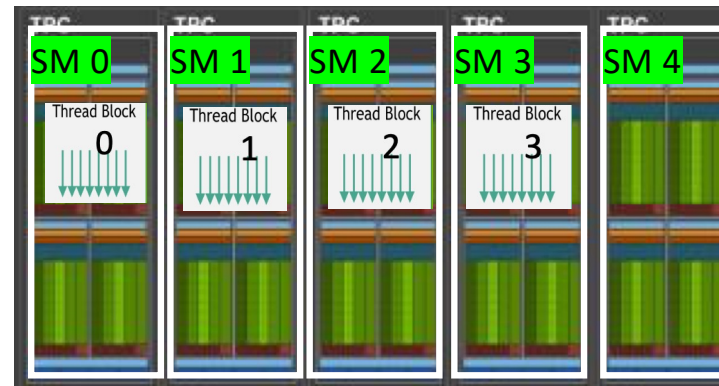


Thread blocks will reside on their SMs for the **entire lifecycle** (no switching in between SMs)

Mapping parallelization model to hardware

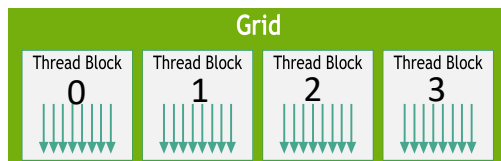


Time: t_2

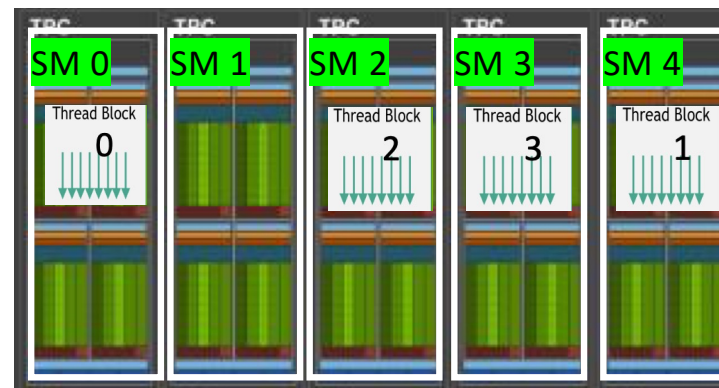


Thread blocks will reside on their SMs for the **entire lifecycle** (no switching in between SMs)

Mapping parallelization model to hardware



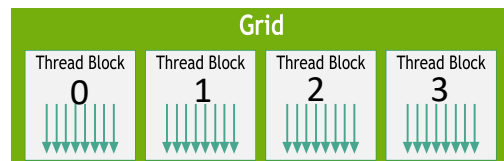
Time: t3



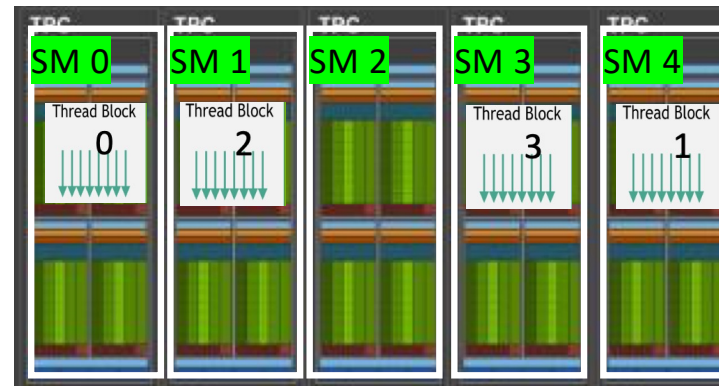
This is not possible

Thread blocks will reside on their SMs for the **entire lifecycle** (no switching in between SMs)

Mapping parallelization model to hardware



Time: t_4



This is not possible

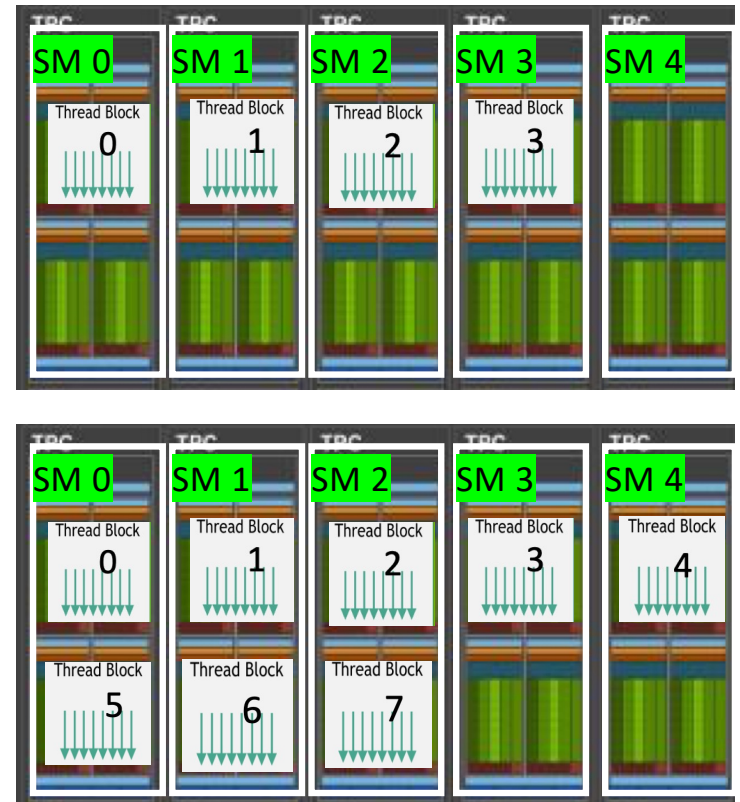
Thread blocks will reside on their SMs for the **entire lifecycle** (no switching in between SMs)

Mapping parallelization model to hardware

When would thread blocks share a same SM?

- if # of SMs $\geq$ # of TBs
 - No need to share
- if # of SMs $<$ # of TBs
 - if a SM's **resource capacity** $\geq K$ TBs' total **resource requirement**
 - K TBs will share a same SM
 - otherwise
 - No sharing

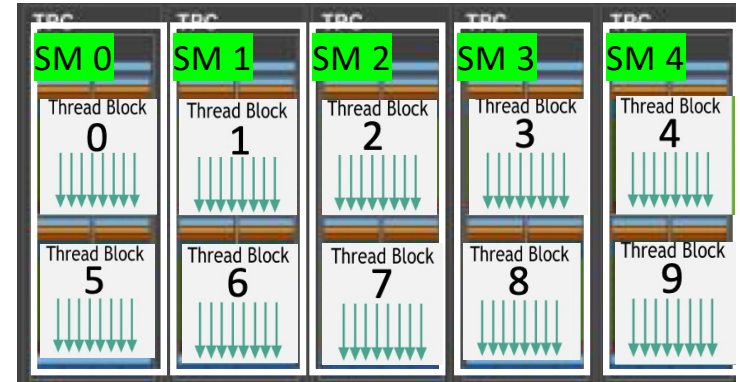
Will talk about SM and TB resource in later lectures



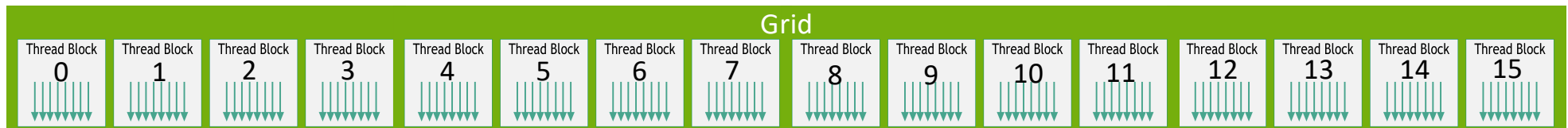
Mapping parallelization model to hardware

- Assume each SM can take K TBs and we have N SMs
- So, the GPU can execute $K*N$ TBs at the same time

For example, $K = 2$ and $N = 5$. So, total 10 TBs



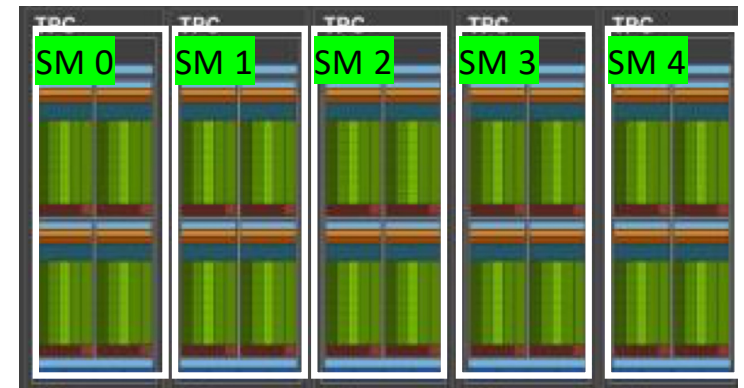
Mapping parallelization model to hardware



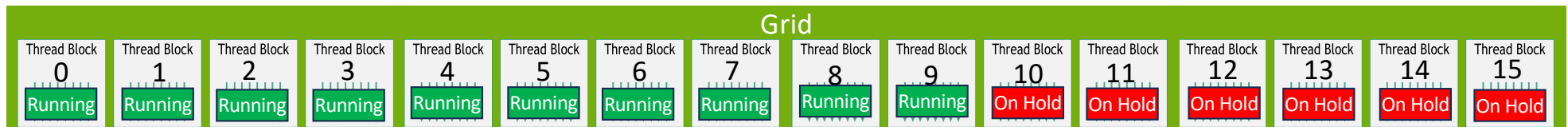
- Assume each SM can take K TBs and we have N SMs
- So, the GPU can execute $K*N$ TBs at the same time

What if a grid contains more than $K*N$ TBs?

At most $K*N$ TBs will be running and rest TBs will be put on hold until some active TBs finished execution and release resources on SM. Then, TBs on hold will be assigned to SMs for execution



Mapping parallelization model to hardware

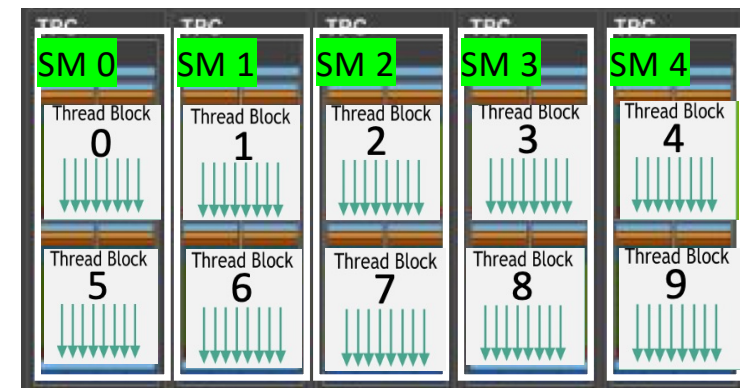


- Assume each SM can take K TBs and we have N SMs
- So, the GPU can execute $K*N$ TBs at the same time

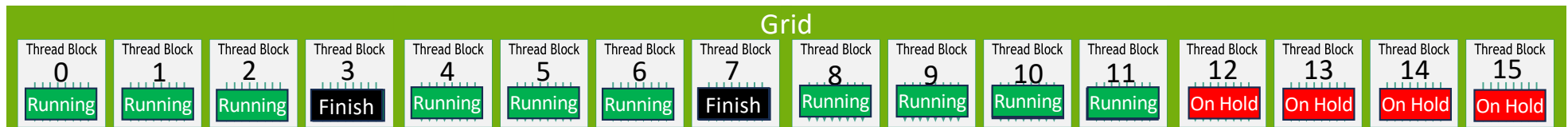
What if a grid contains more than $K*N$ TBs?

At most $K*N$ TBs will be running and rest TBs will be put on hold until some active TBs finished execution and release resources on SM. Then, TBs on hold will be assigned to SMs for execution

Time = t_0



Mapping parallelization model to hardware

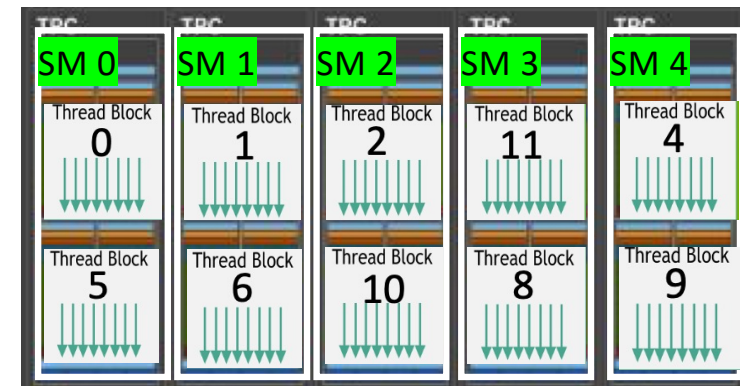


- Assume each SM can take K TBs and we have N SMs
- So, the GPU can execute $K*N$ TBs at the same time

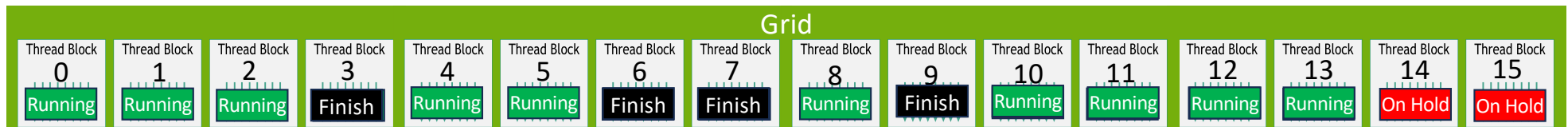
What if a grid contains more than $K*N$ TBs?

At most $K*N$ TBs will be running and rest TBs will be put on hold until some active TBs finished execution and release resources on SM. Then, TBs on hold will be assigned to SMs for execution

Time = t_1



Mapping parallelization model to hardware

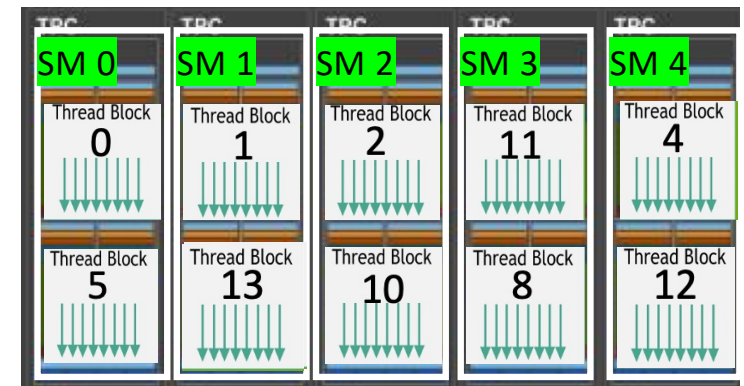


- Assume each SM can take K TBs and we have N SMs
- So, the GPU can execute $K*N$ TBs at the same time

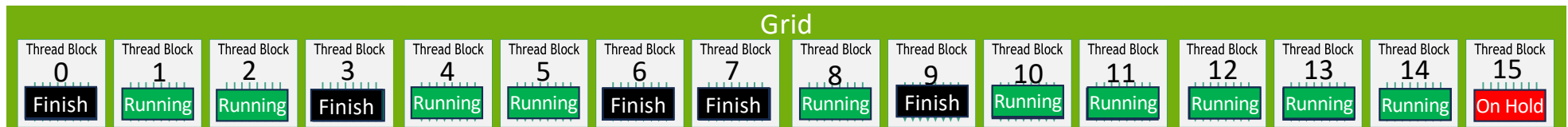
What if a grid contains more than $K*N$ TBs?

At most $K*N$ TBs will be running and rest TBs will be put on hold until some active TBs finished execution and release resources on SM. Then, TBs on hold will be assigned to SMs for execution

Time = t_2



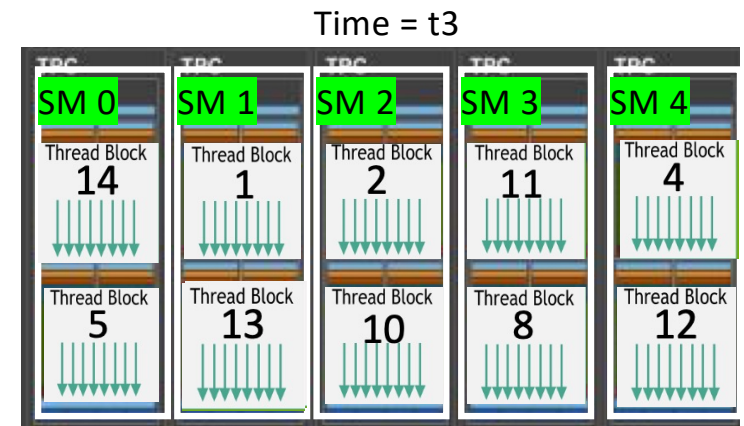
Mapping parallelization model to hardware



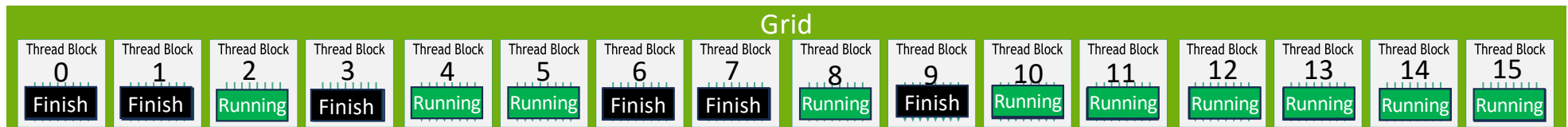
- Assume each SM can take K TBs and we have N SMs
- So, the GPU can execute $K*N$ TBs at the same time

What if a grid contains more than $K*N$ TBs?

At most $K*N$ TBs will be running and rest TBs will be put on hold until some active TBs finished execution and release resources on SM. Then, TBs on hold will be assigned to SMs for execution



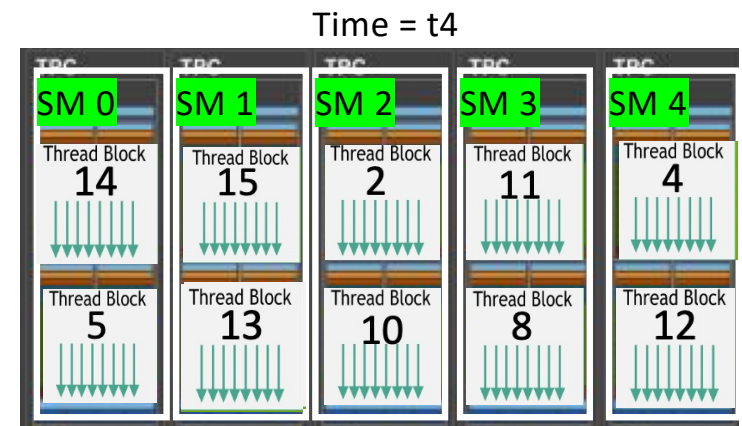
Mapping parallelization model to hardware



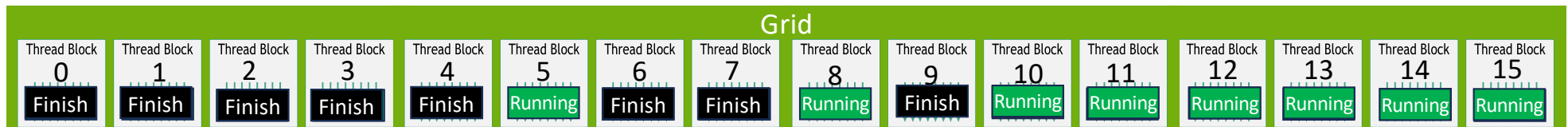
- Assume each SM can take K TBs and we have N SMs
- So, the GPU can execute $K*N$ TBs at the same time

What if a grid contains more than $K*N$ TBs?

At most $K*N$ TBs will be running and rest TBs will be put on hold until some active TBs finished execution and release resources on SM. Then, TBs on hold will be assigned to SMs for execution



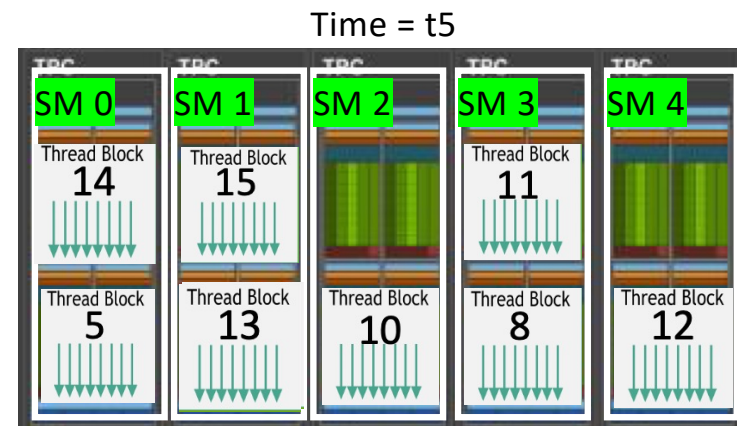
Mapping parallelization model to hardware



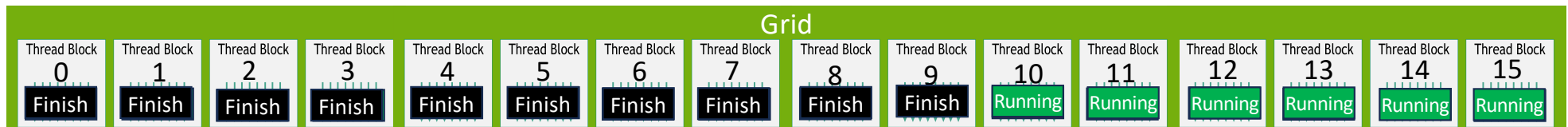
- Assume each SM can take K TBs and we have N SMs
- So, the GPU can execute $K*N$ TBs at the same time

What if a grid contains more than $K*N$ TBs?

At most $K*N$ TBs will be running and rest TBs will be put on hold until some active TBs finished execution and release resources on SM. Then, TBs on hold will be assigned to SMs for execution



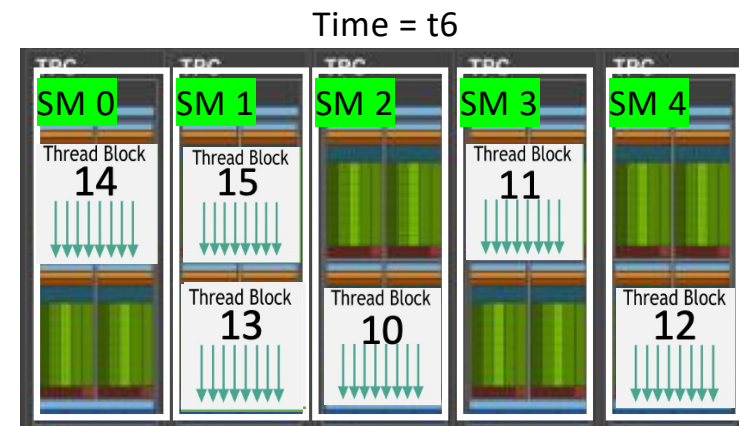
Mapping parallelization model to hardware



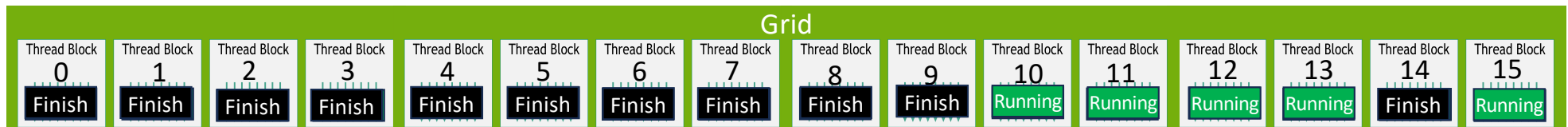
- Assume each SM can take K TBs and we have N SMs
- So, the GPU can execute $K*N$ TBs at the same time

What if a grid contains more than $K*N$ TBs?

At most $K*N$ TBs will be running and rest TBs will be put on hold until some active TBs finished execution and release resources on SM. Then, TBs on hold will be assigned to SMs for execution



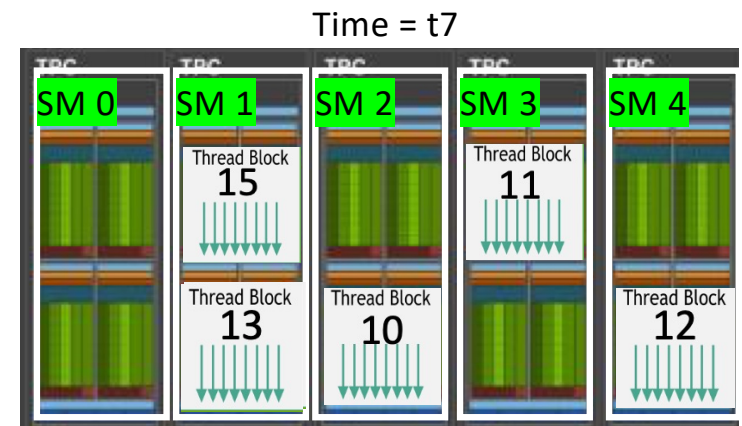
Mapping parallelization model to hardware



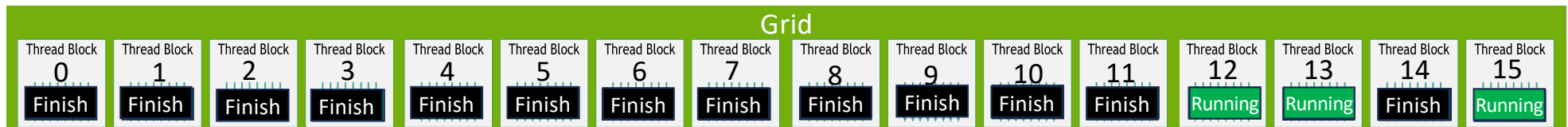
- Assume each SM can take K TBs and we have N SMs
- So, the GPU can execute $K*N$ TBs at the same time

What if a grid contains more than $K*N$ TBs?

At most $K*N$ TBs will be running and rest TBs will be put on hold until some active TBs finished execution and release resources on SM. Then, TBs on hold will be assigned to SMs for execution



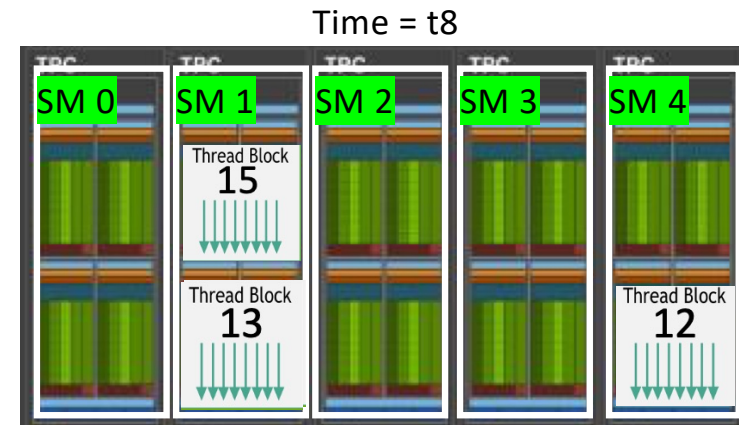
Mapping parallelization model to hardware



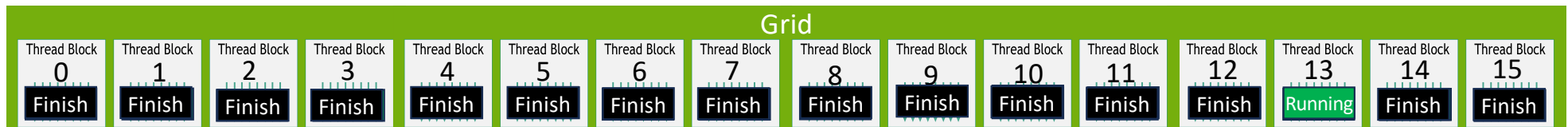
- Assume each SM can take K TBs and we have N SMs
- So, the GPU can execute $K*N$ TBs at the same time

What if a grid contains more than $K*N$ TBs?

At most $K*N$ TBs will be running and rest TBs will be put on hold until some active TBs finished execution and release resources on SM. Then, TBs on hold will be assigned to SMs for execution



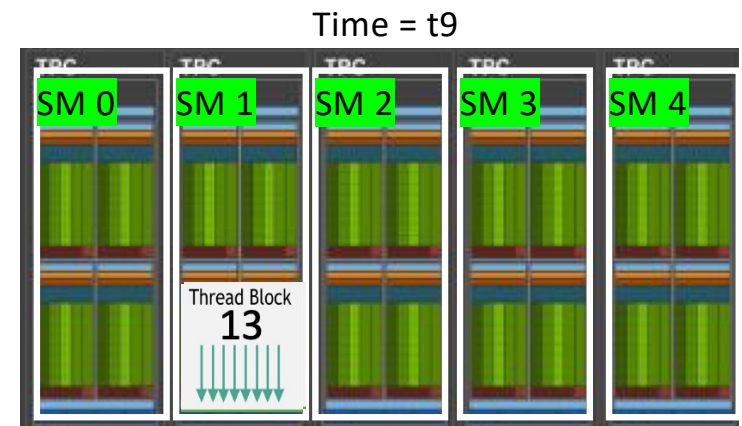
Mapping parallelization model to hardware



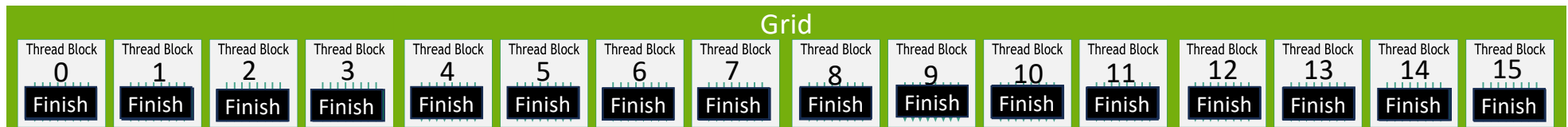
- Assume each SM can take K TBs and we have N SMs
- So, the GPU can execute $K*N$ TBs at the same time

What if a grid contains more than $K*N$ TBs?

At most $K*N$ TBs will be running and rest TBs will be put on hold until some active TBs finished execution and release resources on SM. Then, TBs on hold will be assigned to SMs for execution



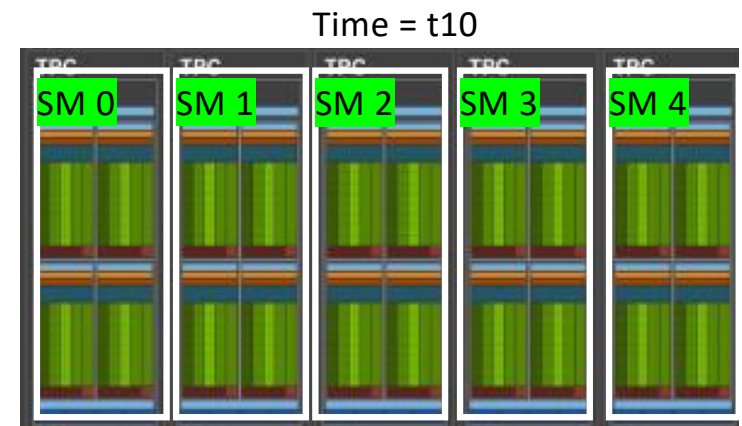
Mapping parallelization model to hardware



- Assume each SM can take K TBs and we have N SMs
- So, the GPU can execute $K*N$ TBs at the same time

What if a grid contains more than $K*N$ TBs?

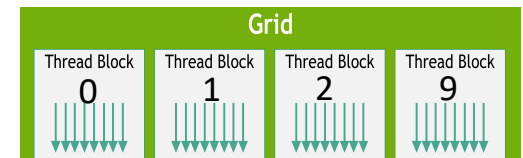
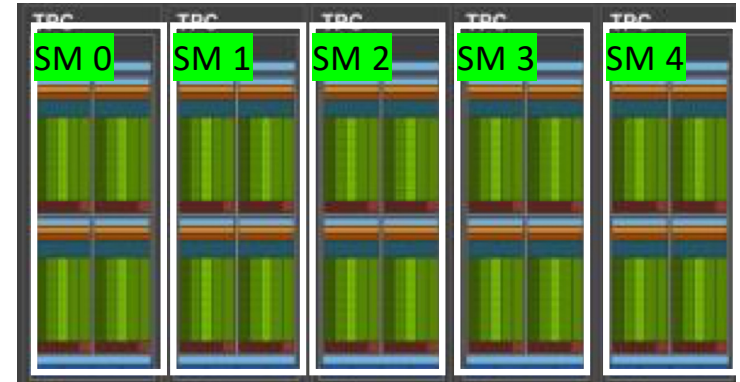
At most $K*N$ TBs will be running and rest TBs will be put on hold until some active TBs finished execution and release resources on SM. Then, TBs on hold will be assigned to SMs for execution



Mapping parallelization model to hardware

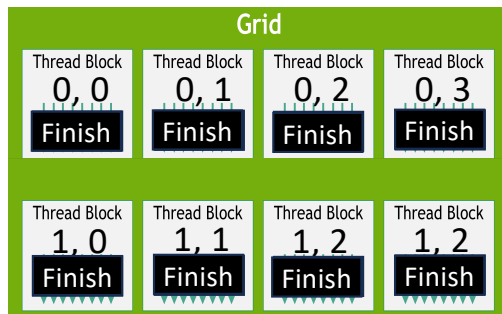
if a SM's **resource capacity** < *one* TB's total **resource requirement**,
the kernel will **fail** to launch.

Will talk about SM and TB resource in later lectures

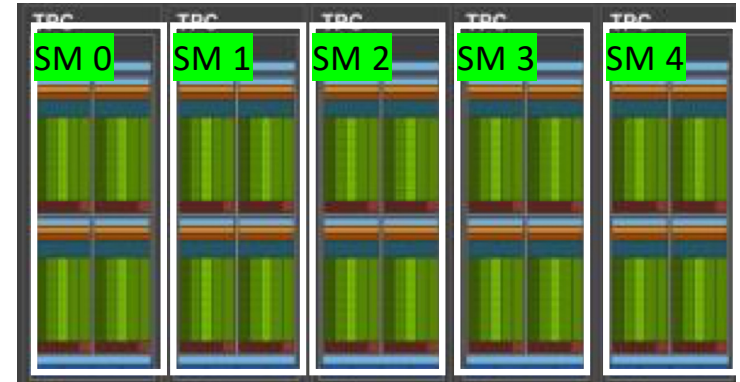
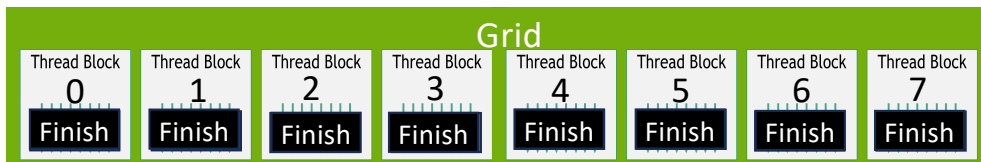


Mapping parallelization model to hardware

How is a multidimensional grid executed on SMs?



Linearized by CUDA internally



Executed as 1D grid

Mapping parallelization model to hardware

Automatic Scalability

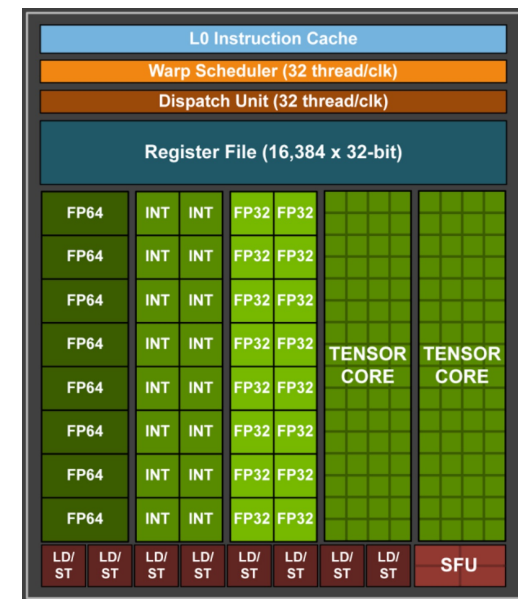
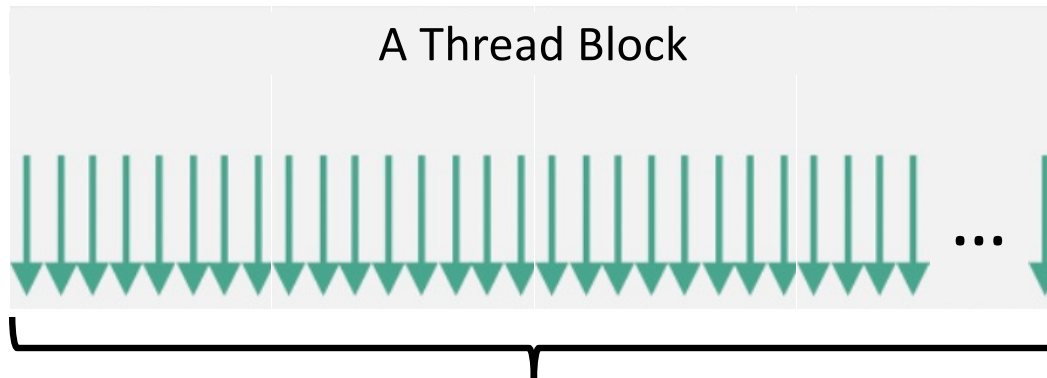
- Allow a CUDA program to run on any GPU without modification
- Example, debugging on a consumer-class GPU on your laptop and running at server-class GPUs without changing code
- Your kernel will automatically take advantage of more powerful GPUs



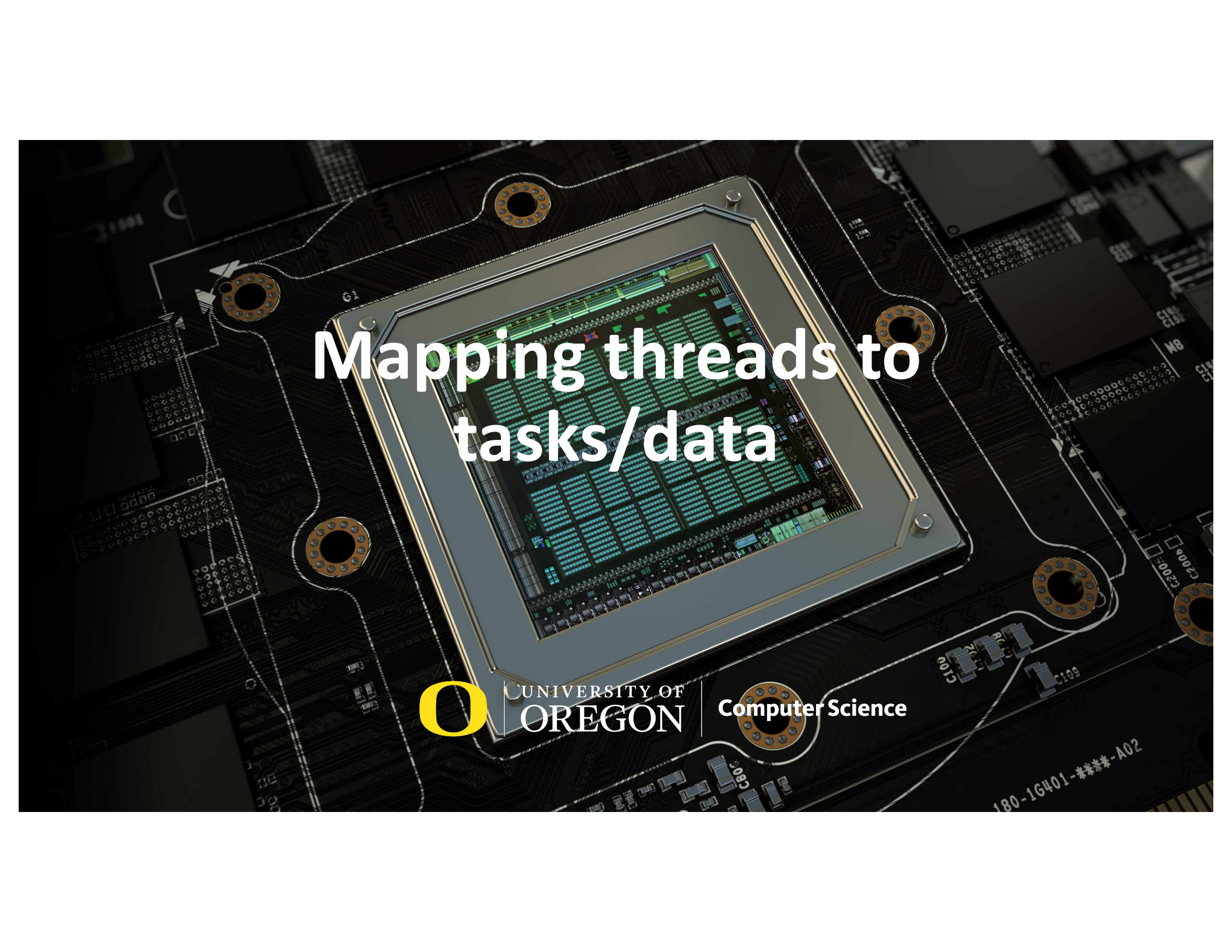
Mapping parallelization model to hardware

How is a thread block executed on a SM?

- A thread block can have up to 1024 threads
- Each SM contain tens of CUDA cores per data type
- All threads **cannot** always run simultaneously
- SM schedulers are designed to manage and execute all threads
(will discuss in details in later lectures)



An example SM (8 FP64 cores, 16 FP32 cores, 32 INT cores, 2 tensor cores)



Mapping threads to tasks/data



UNIVERSITY OF
OREGON

Computer Science

Mapping threads to tasks/data

- GPU is SIMD
- MD = Multiple Data means different threads need to access different data
- How each thread distinguish itself from other thread?
- CUDA runtime has its built-in coordinate system to organize threads

CUDA Vector Types

❑ CUDA Introduces a new `dim` types. E.g. `dim2`, `dim3`, `dim4`

❑ `dim3` contains a collection of three unsigned integers (X, Y, Z)

```
dim3 my_xyz (x_value, y_value, z_value);
```

❑ Values are accessed as members

```
int x = my_xyz.x;  
int y = my_xyz.y;  
int z = my_xyz.z;
```



Built-in variables

- **threadIdx**: thread coordinate within a thread block
- **blockIdx**: thread block coordinate within a grid
- **blockDim**: dimensions of the thread block
- **gridDim**: dimensions of the grid

Built-in variables - **threadIdx**

- thread coordinate within a thread block
- allow threads to distinguish themselves from each other in a thread block
- dim3 type
- Accessible members: **threadIdx.x**, **threadIdx.y**, **threadIdx.z**

	1D thread block (N, 1, 1)	2D thread block (N, M, 1)	3D thread block (N, M, K)
threadIdx.x	0 ... N-1	0 ... N-1	0 ... N-1
threadIdx.y	Always 0	0 ... M-1	0 ... M-1
threadIdx.z	Always 0	Always 0	0 ... K-1

Built-in variables - **blockIdx**

- thread block coordinate within a grid
- allow thread block to distinguish themselves from each other in a grid
- dim3 type
- Accessible members: **blockIdx.x**, **blockIdx.y**, **blockIdx.z**

	1D grid (N, 1, 1)	2D grid (N, M, 1)	3D grid (N, M, K)
blockIdx.x	0 ... N-1	0 ... N-1	0 ... N-1
blockIdx.y	Always 0	0 ... M-1	0 ... M-1
blockIdx.z	Always 0	Always 0	0 ... K-1

Built-in variables - **blockDim**

- get the size of the current thread block
- dim3 type
- Accessible members: **blockDim.x**, **blockDim.y**, **blockDim.z**
- **Values are same for all threads**

	1D thread block (N, 1, 1)	2D thread block (N, M, 1)	3D thread block (N, M, K)
blockDim.x	N	N	N
blockDim.y	1	M	M
blockDim.z	1	1	K

Built-in variables - **gridDim**

- get the size of the current grid
- dim3 type
- Accessible members: **gridDim.x**, **gridDim.y**, **gridDim.z**
- **Values are same for all threads**

	1D grid (N, 1, 1)	2D grid (N, M, 1)	3D grid (N, M, K)
gridDim.x	N	N	N
gridDim.y	1	M	M
gridDim.z	1	1	K

Built-in variables

- **threadIdx**: thread coordinate within a thread block
- **blockIdx**: thread block coordinate within a grid
- **blockDim**: dimensions of the thread block
- **gridDim**: dimensions of the grid

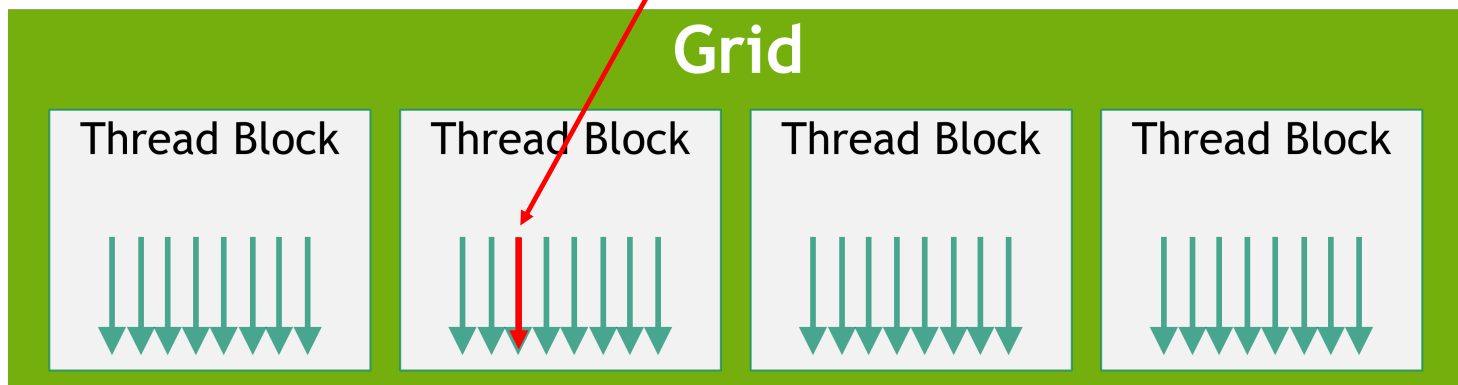
Pre-initialized by the runtime system

Read-only

Cannot be redefined

Example

```
threadIdx = (2, 0, 0) // x, y, z order  
blockIdx = (1, 0, 0)  
blockDim = (8, 1, 1)  
gridDim = (4, 1, 1)
```



1D grid (1*1*4)

1D thread block (1*1*8)

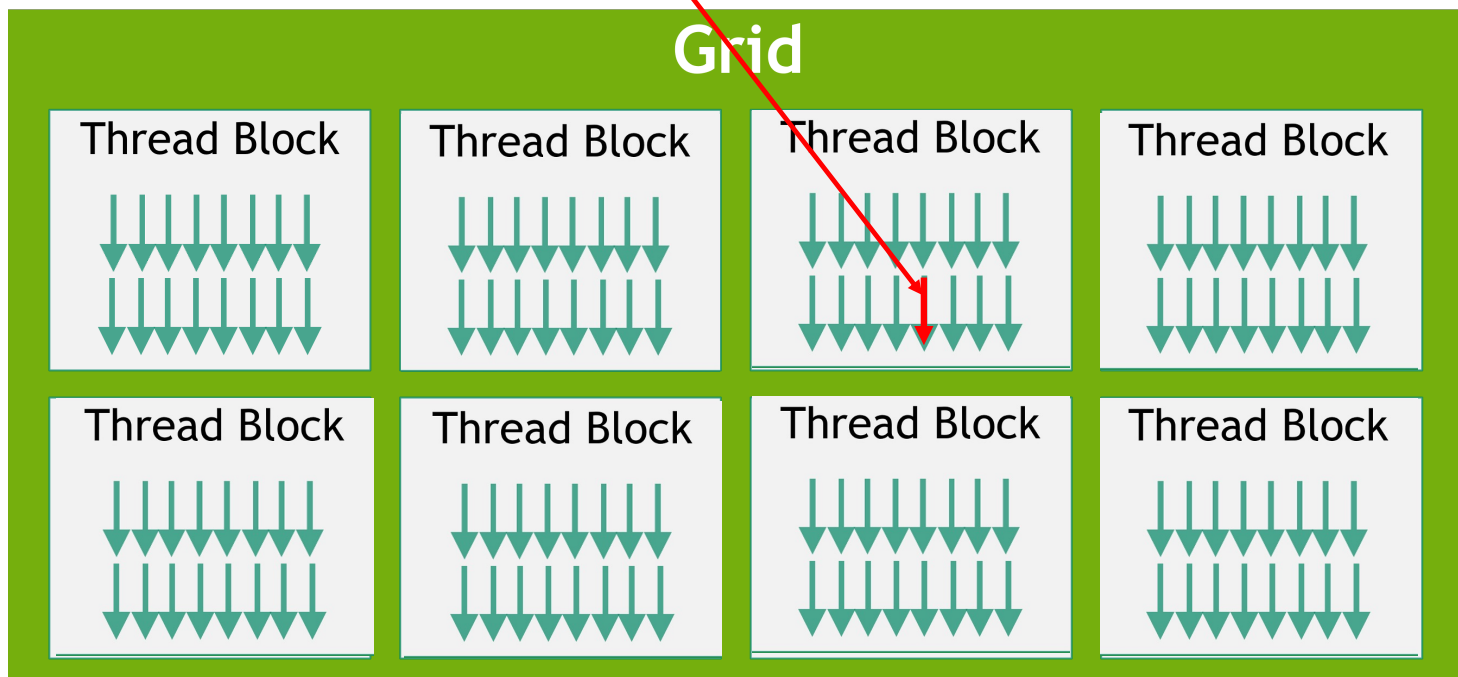


UNIVERSITY OF
OREGON

Computer Science

Example

`threadIdx = (4, 1, 0) // x, y, z order`
`blockIdx = (2, 0, 0)`
`blockDim = (8, 2, 1)`
`gridDim = (4, 2, 1)`



2D grid (1*2*4)

2D thread block (1*2*8)



Alternative terms

CPU = Host

GPU = Device

For example:

Host code

Device code

Host memory

Device memory

GPU Kernel

- CUDA-extended C++ function
- If called, kernel function are executed N times in parallel by N different CUDA threads
- Must start with `__global__` declaration specifier (**two** underscores **on each side**)
- Must have **void** return type (no return)
- Parameters can only be C++ **primitive types** or **class type with default copy constructor**

```
// Kernel definition
__global__ void <kernel name>(<parameters>)
{
    ...
    statements
    ...
}
```

} This part is executed by
N different threads in
parallel

Invoking GPU Kernels

```
// Kernel invocation with N threads  
Kernel Name<<<number of blocks, number of thread per block>>>(...parameters...);
```

Examples

```
Kernel Name<<<16, 256>>>(...parameters...);
```

```
dim3 numBlocks(32, 64)  
dim3 threadsPerBlock(16, 16)  
Kernel Name<<< numBlocks, threadsPerBlock >>>(...parameters...);
```

A First CUDA Example

❑ Serial code

```
for (i=0; i<N; i++) {  
    result[i] = 2*i;  
}
```

❑ We can parallelize this by assigning each iteration to a CUDA thread!

CUDA C Example: Device

```
__global__ void myKernel(int *result)
{
    int i = threadIdx.x;
    result[i] = 2*i;
}
```

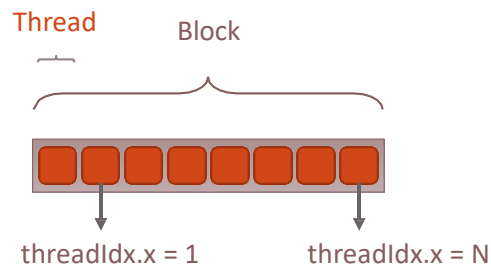
- ❑ Replace loop with a “kernel”
 - ❑ Use **__global__** specifier to indicate it is a CUDA kernel
- ❑ Use **threadIdx** dim variable to get a unique index
 - ❑ Assuming for simplicity we have only **one block** which is **1-dimensional**

CUDA C Example: Host

- ❑ Call the kernel by using the CUDA kernel launch syntax
 - ❑ `kernel<<<GRID OF BLOCKS, BLOCK OF THREADS>>>(arguments);`

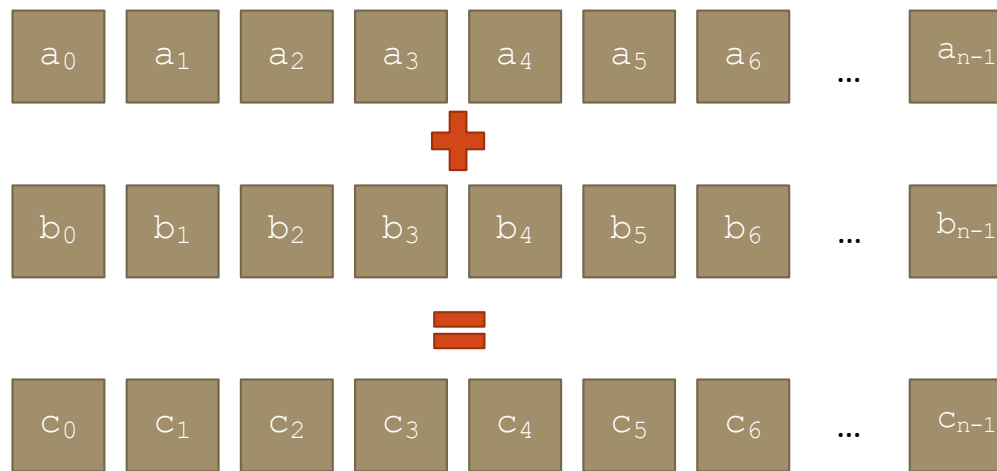
```
dim3 blocksPerGrid(1,1,1);           //use only one block
dim3 threadsPerBlock(N,1,1);         //use N threads in the block
```

```
myKernel<<<blocksPerGrid, threadsPerBlock>>>(result);
```



Vector Addition Example

- ❑ Consider a more interesting example
 - ❑ Vector addition: e.g. $a + b = c$



GPU Kernel

```
// Kernel definition
```

```
__global__ void VecAdd(float* A, float* B, float* C)
```

```
{
```

```
    int i = threadIdx.x;
```

```
    C[i] = A[i] + B[i];
```

```
}
```



This part is executed by
N different threads in
parallel

Invoking GPU Kernels

```
// Kernel definition
__global__ void VecAdd(float* A, float* B, float* C)
{
    int i = threadIdx.x;
    C[i] = A[i] + B[i];
}

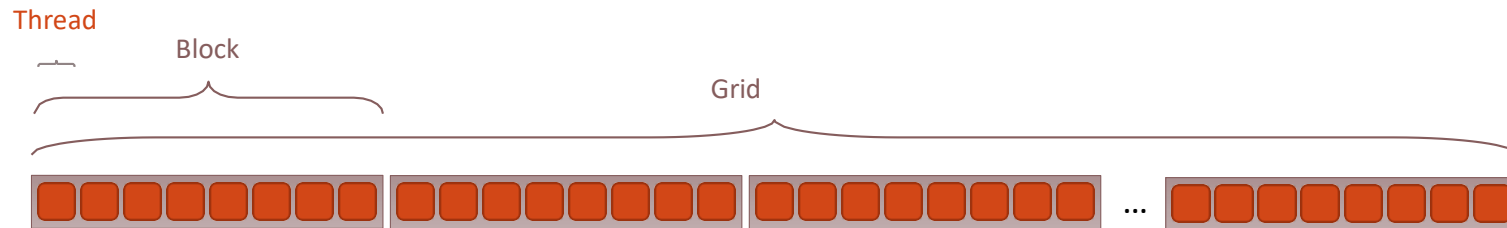
int main()
{
    ...
    // Kernel invocation with N threads
    VecAdd<<<1, N>>>(A, B, C);
    ...
}
```

CUDA C Example: Host

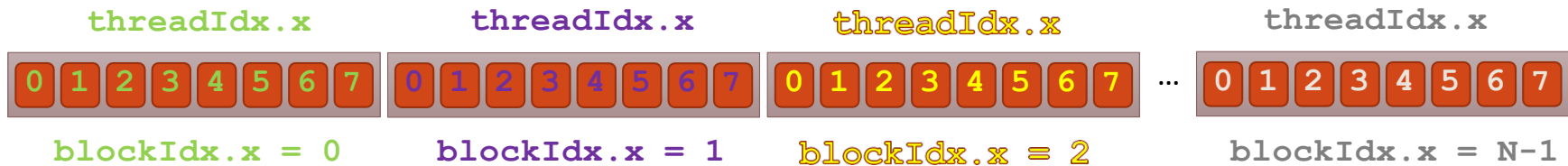
- ❑ Only one block will give poor performance
 - ❑ A block gets allocated to a single SM!
 - ❑ Solution: Use multiple blocks

```
dim3 blocksPerGrid(N/8,1,1);    // assumes 8 divides N exactly
dim3 threadsPerBlock(8,1,1);    // 8 threads in the block
```

```
myKernel<<<blocksPerGrid, threadsPerBlock>>>(result);
```



Vector Addition Example



```
//Kernel Code
__global__ void vectorAdd(float *a, float *b, float *c)
{
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    c[i] = a[i] + b[i];
}
```

- ❑ The integer `i` gives a unique thread Index used to access a unique value from the vectors `a`, `b` and `c`



Device functions

- ❑ Kernels are always prefixed with `__global__`
- ❑ To call another function from a kernel the function must be a device function (i.e. it must be compiled for the GPU device)
- ❑ A device function must be prefixed with `__device__`
- ❑ A device function is not available from the host
 - ❑ Unless it is also prefixed with `__host__`

```
int increment(int a){ return a + 1; }
```

Host only

```
__device__ int increment(int a){ return a + 1; }
```

Device only

```
__device__ __host__ int increment(int a){ return a + 1; }
```

Host and device

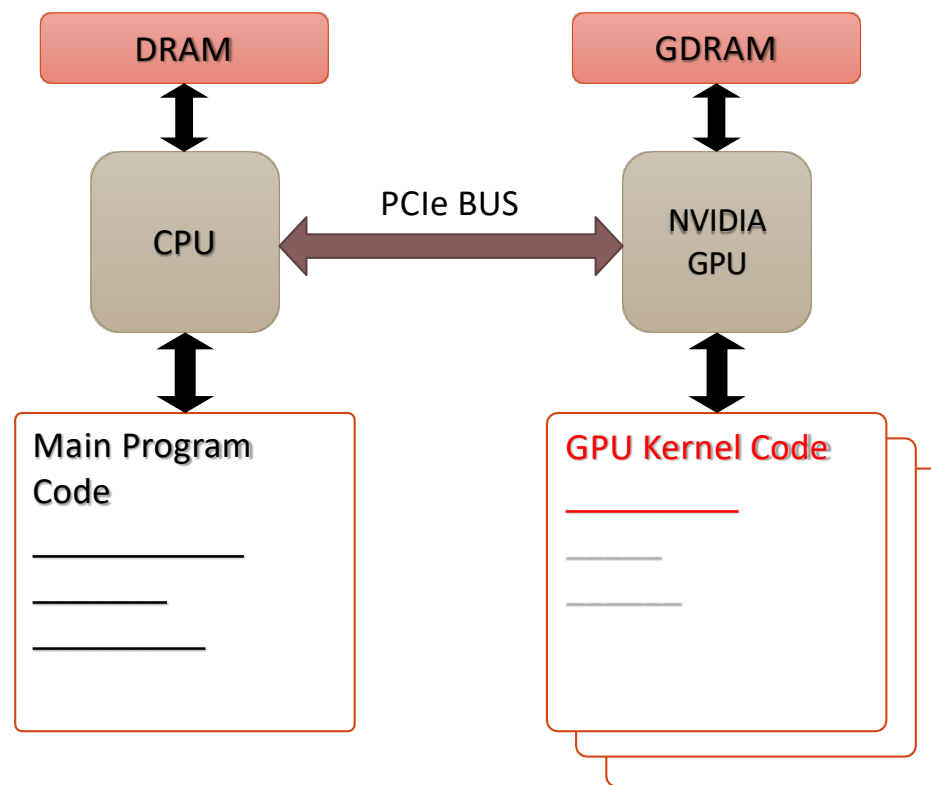
Device functions

Qualifier Keyword	Callable From	Executed On	Executed By
<code>__host__</code> (default)	Host	Host	Caller host thread
<code>__global__</code>	Host (or Device)	Device	New grid of device threads
<code>__device__</code>	Device	Device	Caller device thread

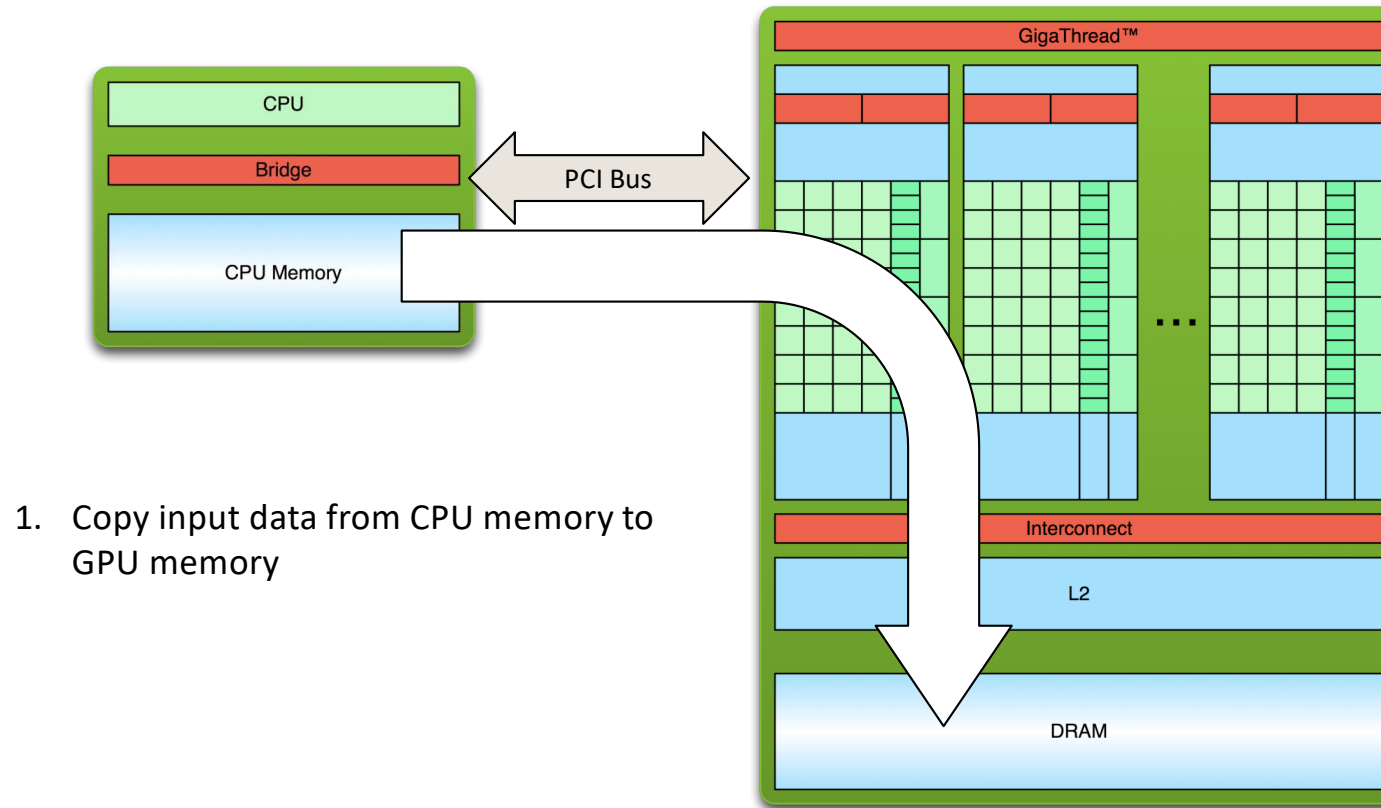
If no qualifier is used, it is equivalent to `__host__` only



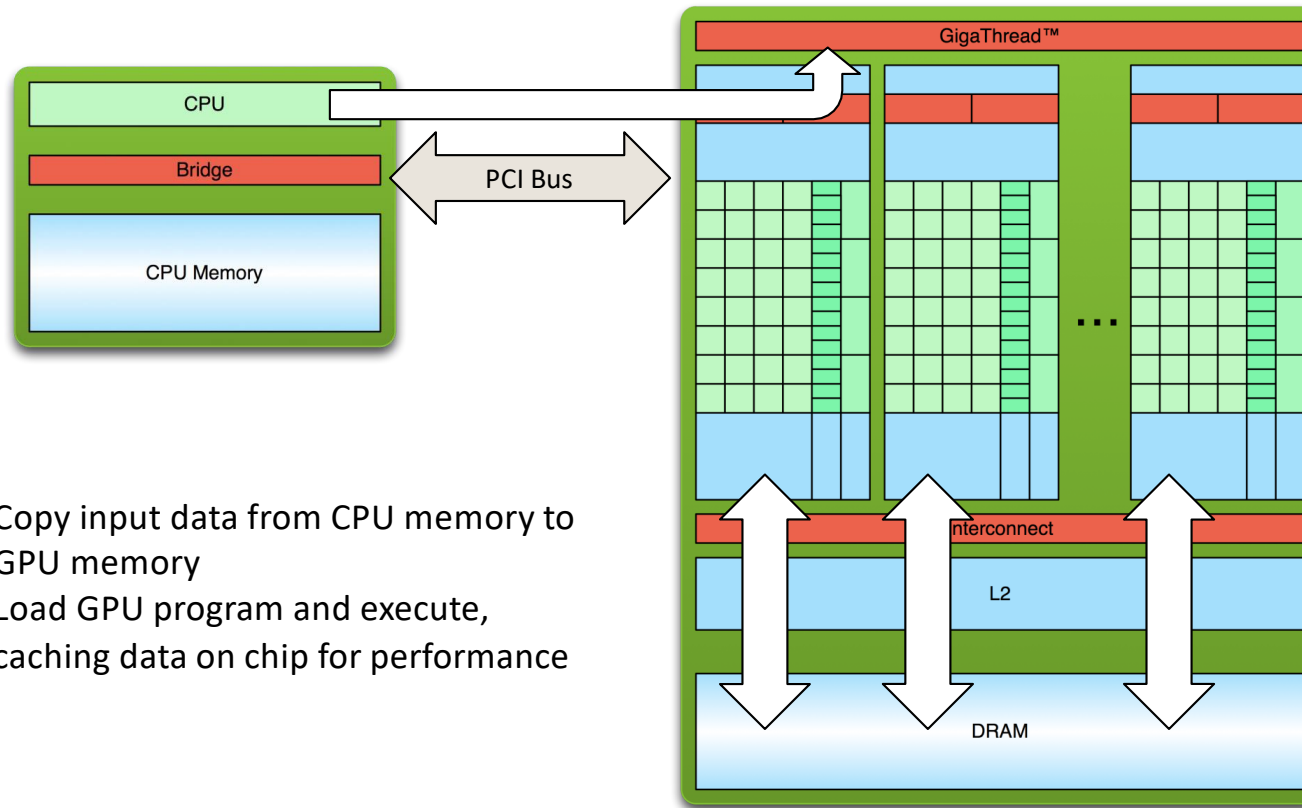
Structure of a GPU program



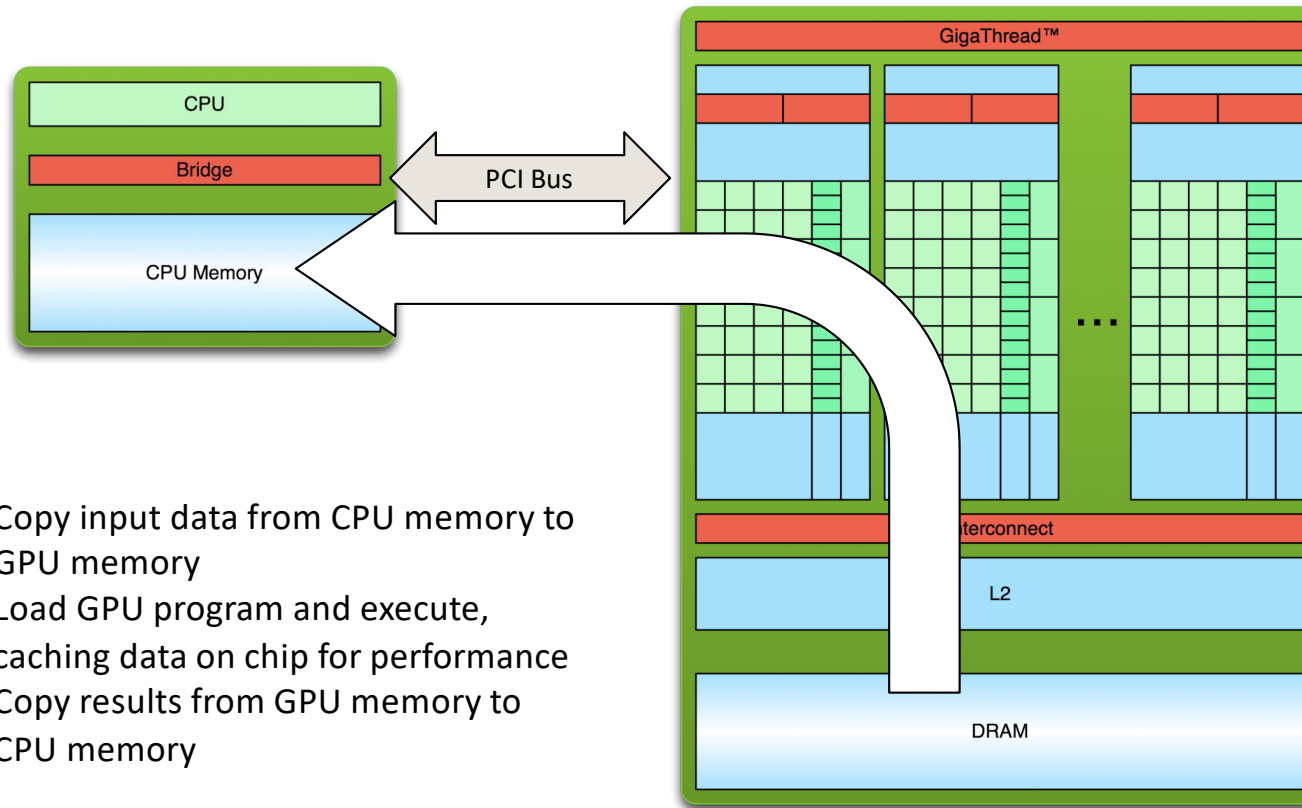
Simple processing flow



Simple processing flow



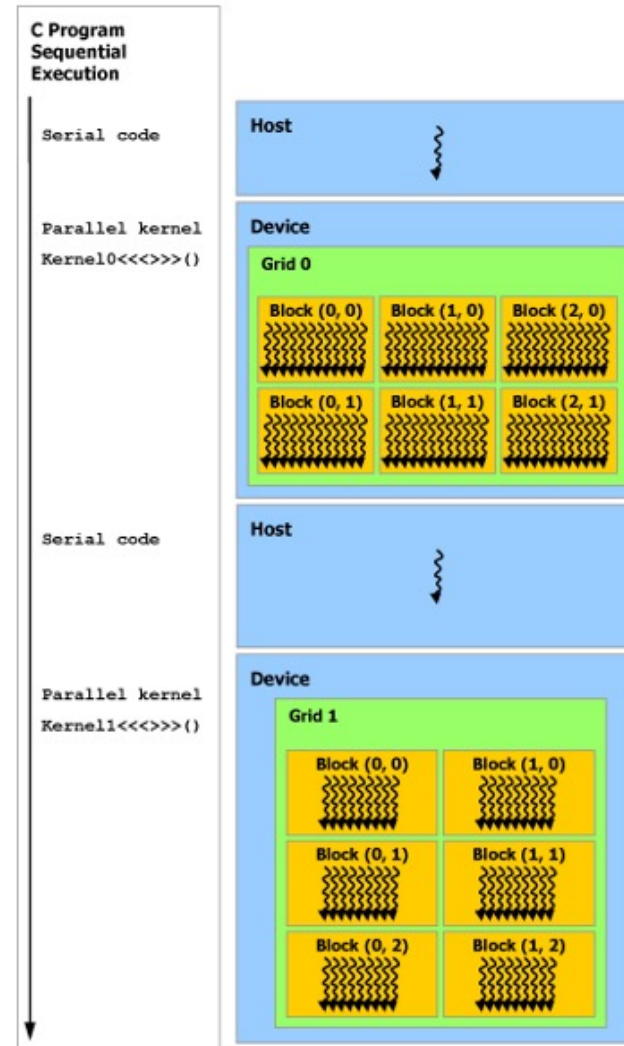
Simple processing flow



1. Copy input data from CPU memory to GPU memory
2. Load GPU program and execute, caching data on chip for performance
3. Copy results from GPU memory to CPU memory



Structure of a GPU program source code



Memory Spaces

- CPU and GPU have separate memory spaces
 - Data is moved across PCIe bus
 - Use functions to allocate/set/copy memory on GPU
 - Very similar to corresponding C functions
 - GPU memory not accessible from CPU
 - CPU memory not accessible from GPU
- Pointers are just addresses
 - Pointers are sharable in between CPU and GPU
 - Must exercise care when dereferencing:
 - Dereferencing CPU pointer on GPU will likely crash
 - Same for vice versa

GPU Memory Allocation / Release

- Host (CPU) manages device (GPU) memory:
 - `cudaMalloc (void ** pointer, size_t nbytes)`
 - `cudaMemset (void * pointer, int value, size_t count)`
 - `cudaFree (void* pointer)`

```
int n = 1024;  
int nbytes = 1024*sizeof(int);  
int * d_a = 0;  
cudaMalloc( (void**)&d_a, nbytes );  
cudaMemset( d_a, 0, nbytes);  
cudaFree(d_a);
```



Data Copies

- `cudaMemcpy( void *dst, void *src, size_t nbytes, enum cudaMemcpyKind direction);`
 - returns after the copy is complete
 - blocks CPU thread until all bytes have been copied
 - doesn't start copying until previous CUDA calls complete
- `enum cudaMemcpyKind`
 - `cudaMemcpyHostToDevice`
 - `cudaMemcpyDeviceToHost`
 - `cudaMemcpyDeviceToDevice`
- Non-blocking copies are also available

Example: Vector Addition Kernel

Device Code

```
// Compute vector sum C = A+B
// Each thread performs one pair-wise addition
__global__ void vecAdd(float* A, float* B, float* C)
{
    int i = threadIdx.x + blockDim.x * blockIdx.x;
    C[i] = A[i] + B[i];
}
```

```
int main()
{
    // Run grid of N/256 blocks of 256 threads each
    vecAdd<<< N/256, 256>>>>(d_A, d_B, d_C);
}
```



Example: Vector Addition Kernel

```
// Compute vector sum C = A+B
// Each thread performs one pair-wise addition
__global__ void vecAdd(float* A, float* B, float* C)
{
    int i = threadIdx.x + blockDim.x * blockIdx.x;
    C[i] = A[i] + B[i];
}
```

Host Code

```
int main()
{
    // Run grid of N/256 blocks of 256 threads each
    vecAdd<<< N/256, 256>>>>(d_A, d_B, d_C);
}
```

How about input/output data?
(see next few slides)



UNIVERSITY OF
OREGON

Computer Science

Example: Host code for `vecAdd` (1/2)

```
// allocate and initialize host (CPU) memory
float *h_A = ..., *h_B = ...; *h_C = ... (empty)

// allocate device (GPU) memory
float *d_A, *d_B, *d_C;
cudaMalloc( (void**) &d_A, N * sizeof(float));
cudaMalloc( (void**) &d_B, N * sizeof(float));
cudaMalloc( (void**) &d_C, N * sizeof(float));

// copy host memory to device
cudaMemcpy( d_A, h_A, N * sizeof(float), cudaMemcpyHostToDevice );
cudaMemcpy( d_B, h_B, N * sizeof(float), cudaMemcpyHostToDevice );

// execute grid of N/256 blocks of 256 threads each
vecAdd<<<N/256, 256>>>(d_A, d_B, d_C);
```



Example: Host code for `vecAdd` (2/2)

```
// execute grid of N/256 blocks of 256 threads each
vecAdd<<<N/256, 256>>>(d_A, d_B, d_C);

// copy result back to host memory
cudaMemcpy( h_C, d_C, N * sizeof(float), cudaMemcpyDeviceToHost );

// do something with the result...

// free device (GPU) memory
cudaFree(d_A);
cudaFree(d_B);
cudaFree(d_C);
```

Exercise: what would be in array a?

```
__global__ void kernel( int *a )  
{  
    int idx = blockIdx.x*blockDim.x + threadIdx.x;  
    a[idx] = 7;  
}
```

Output: 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7

```
__global__ void kernel( int *a )  
{  
    int idx = blockIdx.x*blockDim.x + threadIdx.x;  
    a[idx] = blockIdx.x;  
}
```

Output: 0 0 0 0 1 1 1 1 2 2 2 2 3 3 3 3

```
__global__ void kernel( int *a )  
{  
    int idx = blockIdx.x*blockDim.x + threadIdx.x;  
    a[idx] = threadIdx.x;  
}
```

Output: 0 1 2 3 0 1 2 3 0 1 2 3 0 1 2 3

Code executed on GPU

- C/C++ with some restrictions:
 - Can only access GPU memory
 - No variable number of arguments
 - No static variables
 - No recursion
 - No dynamic polymorphism
- Must be declared with a qualifier:
 - `__global__` : launched by CPU, cannot be called from GPU must return void
 - `__device__` : called from other GPU functions, cannot be called by the CPU
 - `__host__` : can be called by CPU
 - `__host__` and `__device__` qualifiers can be combined
 - sample use: overloading operators



Example: use memory operations

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int dimx = 16;
```

```
    int num_bytes = dimx*sizeof(int);
```

```
    int *d_a=0, *h_a=0; // device and host pointers
```

```
    h_a = (int*)malloc(num_bytes);
```

```
    cudaMalloc( (void**)&d_a, num_bytes );
```

```
    if( 0==h_a || 0==d_a )
```

```
    {
```

```
        printf("couldn't allocate memory\n");
```

```
        return 1;
```

```
    }
```

```
    cudaMemset( d_a, 0, num_bytes );
```

```
    cudaMemcpy( h_a, d_a, num_bytes,
```

```
                cudaMemcpyDeviceToHost );
```

```
    for(int i=0; i<dimx; i++)
```

```
        printf("%d ", h_a[i] );
```

```
    printf("\n");
```

```
    free( h_a );
```

```
    cudaFree( d_a );
```

```
    return 0;
```

```
}
```

Example: Shuffling data based on index array

```
// Reorder values based on keys
// Each thread moves one element
__global__ void shuffle(int* prev_array, int* new_array, int*
indices)
{
    int i = threadIdx.x + blockDim.x * blockIdx.x;
    new_array[i] = prev_array[indices[i]];
}
```

Host Code

```
int main()
{
    // Run grid of N/256 blocks of 256 threads each
    shuffle<<< N/256, 256>>>(d_old, d_new, d_ind);
}
```



Example: Kernel with 2D Indexing

```
__global__ void kernel( int *a, int dimx, int dimy )  
{  
    int ix  = blockIdx.x*blockDim.x + threadIdx.x;  
    int iy  = blockIdx.y*blockDim.y + threadIdx.y;  
  
    if (ix >= dimx || iy >= dimy) return;  
  
    int idx = iy*dimx + ix;  
  
    a[idx] = a[idx]+1;  
}
```

What happens when the data dimension is not a multiple of the thread block size?

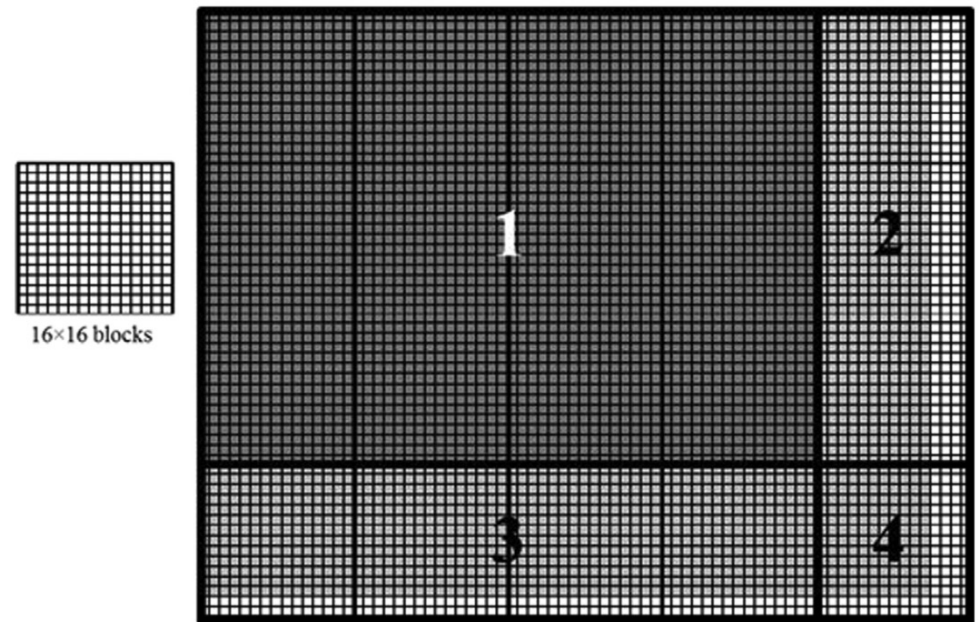


FIGURE 3.5

Covering a 76 × 62 picture with 16 × 16 blocks.

Example: Kernel with 2D Indexing

```
__global__ void kernel( int *a, int dimx, int dimy )
{
    int ix  = blockIdx.x*blockDim.x + threadIdx.x;
    int iy  = blockIdx.y*blockDim.y + threadIdx.y;
    int idx = iy*dimx + ix;

    if (ix >= dimx || iy >= dimy) return;

    a[idx] = a[idx]+1;
}
```

```
int main()
{
    int dimx = 76;
    int dimy = 62;
    int num_bytes = dimx*dimy*sizeof(int);

    int *d_a=0, *h_a=0; // device and host pointers

    h_a = (int*)malloc(num_bytes);
    cudaMalloc( (void**)&d_a, num_bytes );

    if( 0==h_a || 0==d_a )
    {
        printf("couldn't allocate memory\n");
        return 1;
    }

    cudaMemset( d_a, 0, num_bytes );

    dim3 grid, block;
    block.x = 16;
    block.y = 16;
    grid.x = dimx / block.x;
    grid.y = dimy / block.y;

    kernel<<<grid, block>>>( d_a, dimx, dimy );

    cudaMemcpy( h_a, d_a, num_bytes, cudaMemcpyDeviceToHost );

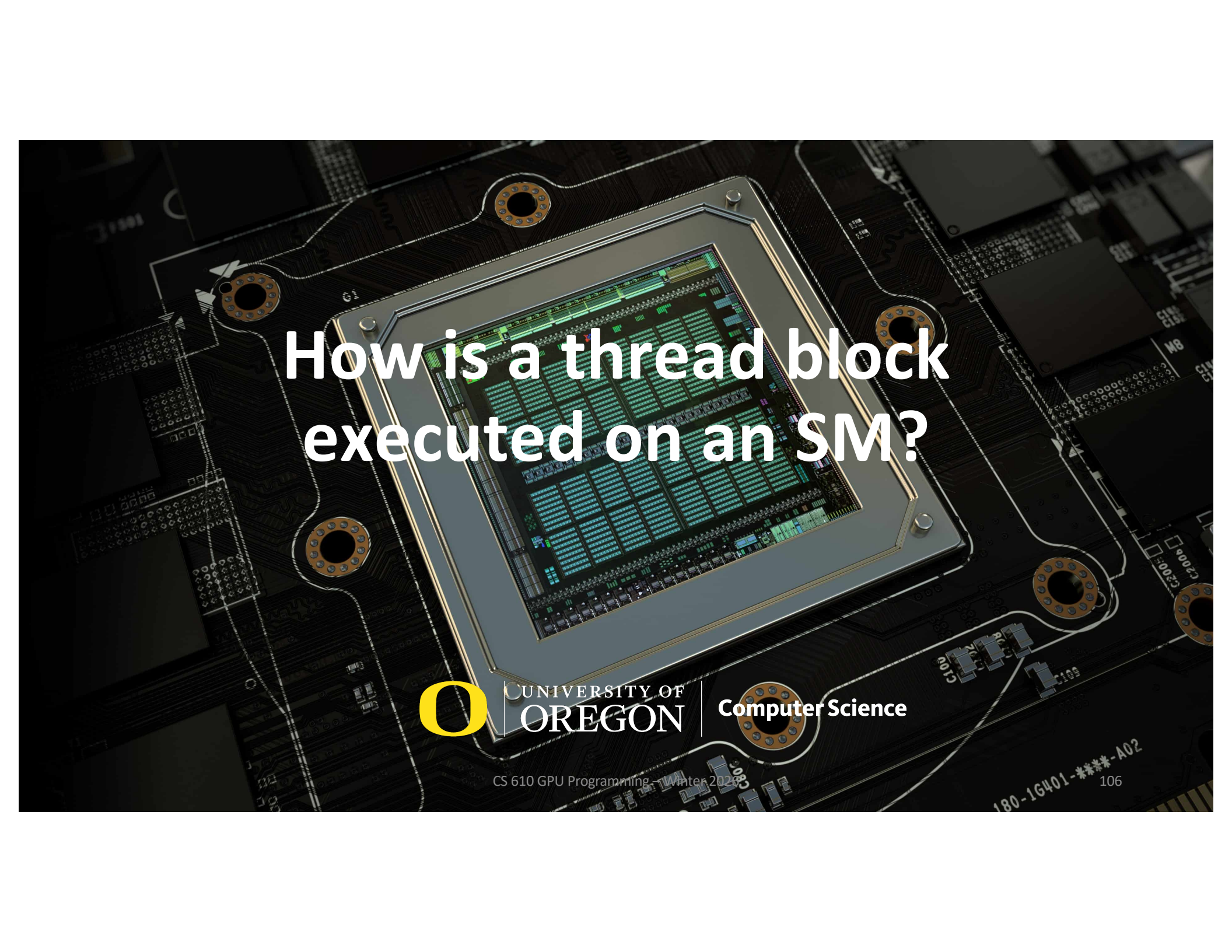
    for(int row=0; row<dimy; row++)
    {
        for(int col=0; col<dimx; col++)
            printf("%d ", h_a[row*dimx+col] );
        printf("\n");
    }

    free( h_a );
    cudaFree( d_a );

    return 0;
}
```

Blocks must be independent

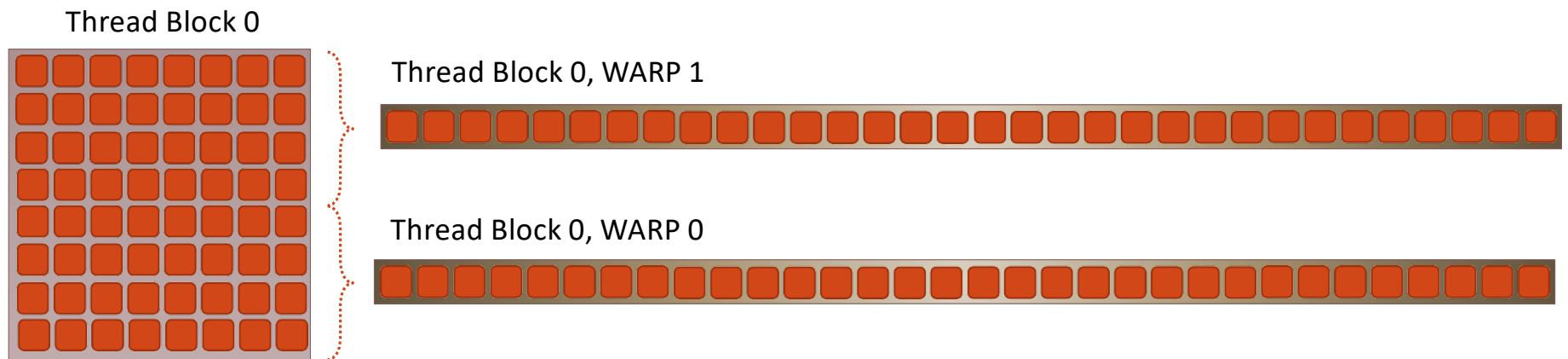
- Any possible interleaving of blocks should be valid
 - presumed to run to completion without pre-emption
 - can run in any order
 - can run concurrently OR sequentially
- Blocks may coordinate but not synchronize
 - shared queue pointer: OK
 - shared lock: BAD ... can easily deadlock
- Independence requirement gives scalability



How is a thread block executed on an SM?

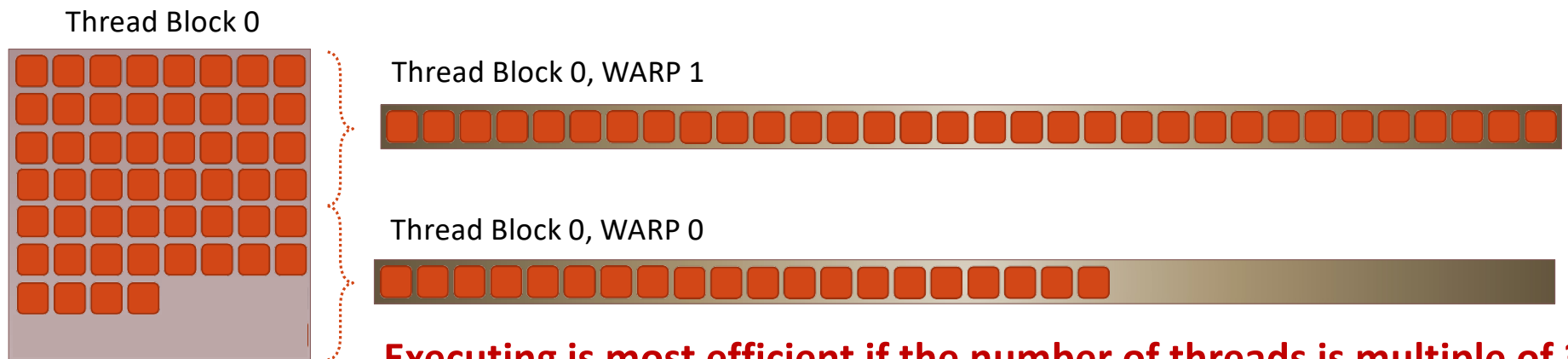
Warp

- ❑ Each thread block is mapped to one or more warps
- ❑ Warp is the **basic scheduling unit** in GPU
- ❑ Warps contain only threads from a single thread block



Warp

- ❑ Each thread block is mapped to one or more warps
- ❑ Warp is the **basic scheduling unit** in GPU
- ❑ Warps contain only threads from a single thread block



Executing is most efficient if the number of threads is multiple of 32



How are warps formed?

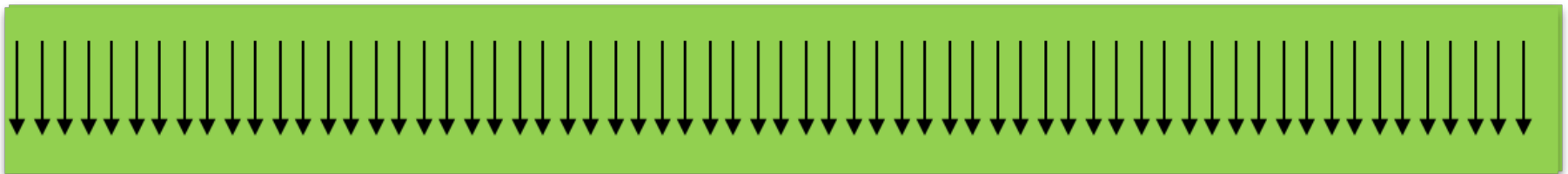
- Warp partition is based on an internal **thread ID** (not accessible by developers)
- 2D and 3D thread blocks are converted to 1D index
- You can calculate thread ID using:
 - 1D thread block: **thread ID** = threadIdx.x
 - 2D thread block: **thread ID** = threadIdx.x + blockDim.x * threadIdx.y
 - 3D thread block: **thread ID** = threadIdx.x + blockDim.x * threadIdx.y + blockDim.x * blockDim.y * threadIdx.z
- **Every consecutive 32 threads forms a warp**
 - thread ID: 0 ... 31 → warp 0
 - thread ID: 32 ... 63 → warp 1
 - thread ID: 64 ... 95 → warp 2
 - thread ID: 96 ... 127 → warp 3
 - ...
- The way a block is partitioned into warps **will not change** for the entire lifecycle



Warp and Lane

- Each thread in a warp is also called a **lane**
- Lane index: 0 ... 31
- Calculating warp id and lane id
 - Warp ID = Thread ID / 32
 - Lane ID = Thread ID % 32
- Faster way
 - Warp ID = Thread ID >> 5
 - Lane ID = Thread ID & 0x1f

A warp



Exercise: calculate warp and lane id

Given 1D thread block (size 256)

- `threadIdx.x == 50`
- warp id = ?
- lane id = ?

Answer:

thread id = 50

warp id = $50 / 32 = 1$

lane id = $50 \% 32 = 18$

- `threadIdx.x == 200`
- warp id = ?
- lane id = ?

Answer:

thread id = 200

warp id = $200 / 32 = 6$

lane id = $200 \% 32 = 8$

Exercise: calculate warp and lane id

Given 2D thread block of size $32(x) * 16(y)$

- `threadIdx.x == 12, threadIdx.y == 5`
- warp id = ?
- lane id = ?

Answer:

$$\begin{aligned}\text{thread id} &= \text{threadIdx.x} + \text{blockDim.x} * \text{threadIdx.y} \\ &= 12 + 32 * 5 = 172\end{aligned}$$

$$\text{warp id} = 172 / 32 = 5$$

$$\text{lane id} = 172 \% 32 = 12$$

- `threadIdx.x == 20, threadIdx.y == 4`
- warp id = ?
- lane id = ?

Answer:

$$\begin{aligned}\text{thread id} &= \text{threadIdx.x} + \text{blockDim.x} * \text{threadIdx.y} \\ &= 20 + 32 * 4 = 148\end{aligned}$$

$$\text{warp id} = 148 / 32 = 4$$

$$\text{lane id} = 148 \% 32 = 20$$

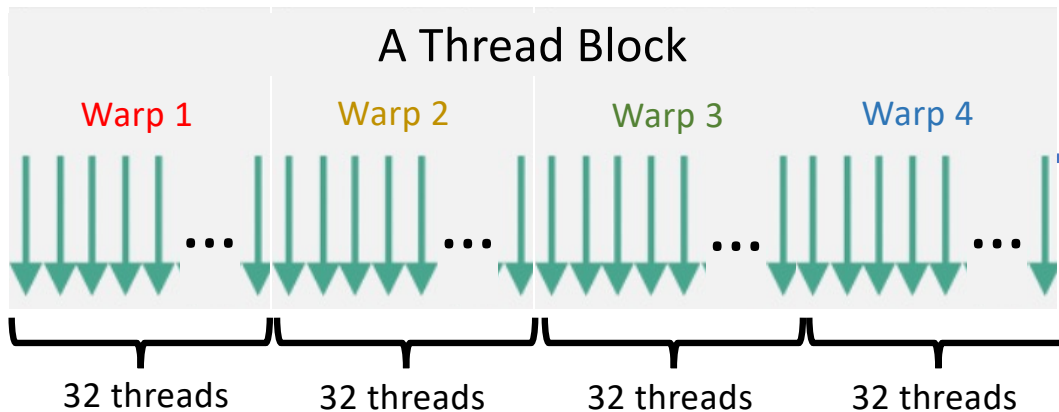
Exercise: calculate warp and lane id

Given 3D thread block of size $16(x) * 8(y) * 8(z)$

- `threadIdx.x == 4, threadIdx.y == 5, threadIdx.z == 6`
 - warp id = ?
 - lane id = ?
- `threadIdx.x == 1, threadIdx.y == 2, threadIdx.z == 5`
 - warp id = ?
 - lane id = ?

How is a thread block executed on an SM?

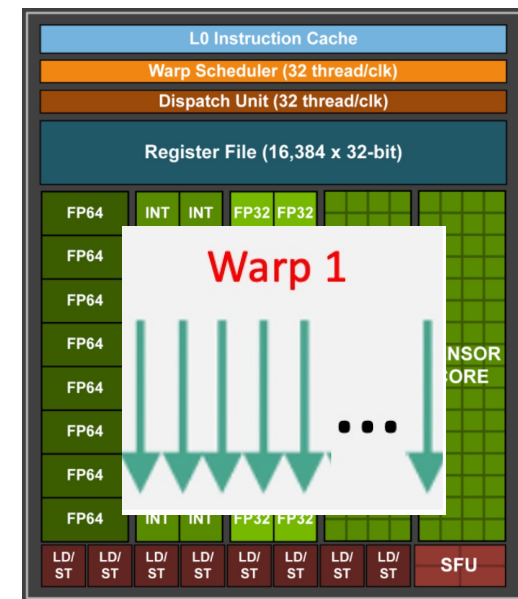
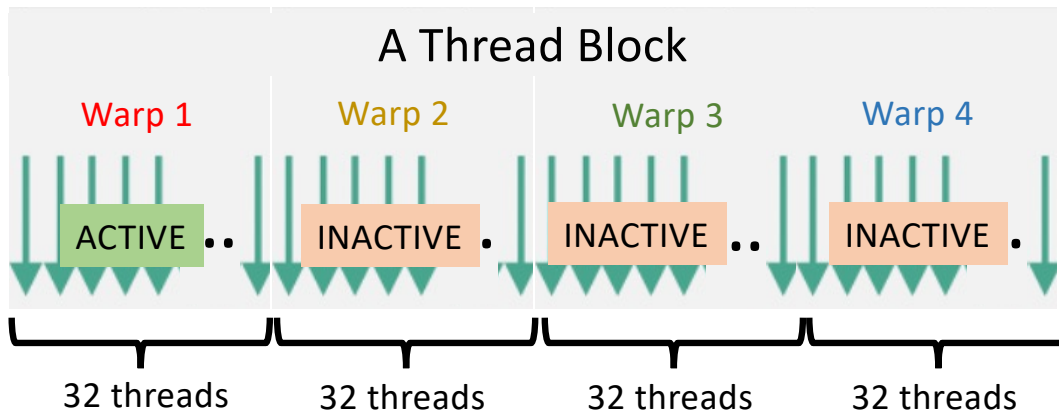
- The SM creates, manages, schedules, and executes threads in groups of 32 parallel threads called *warps*.
- Each warp gets scheduled by a *warp scheduler* for execution



An example SM (8 FP64 cores, 16 FP32 cores, 32 INT cores, 2 tensor cores)

How is a thread block executed on an SM?

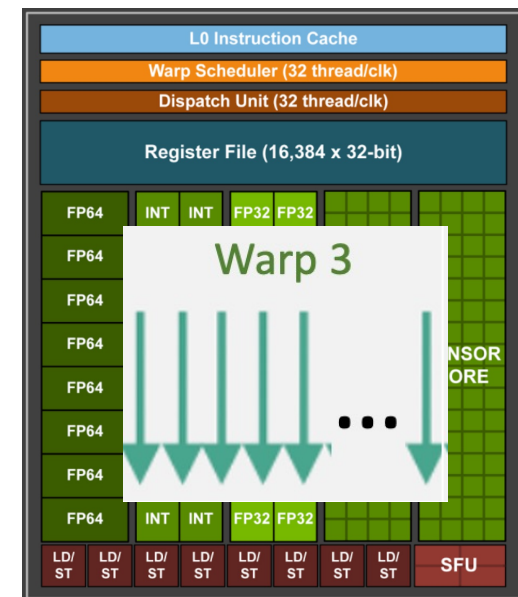
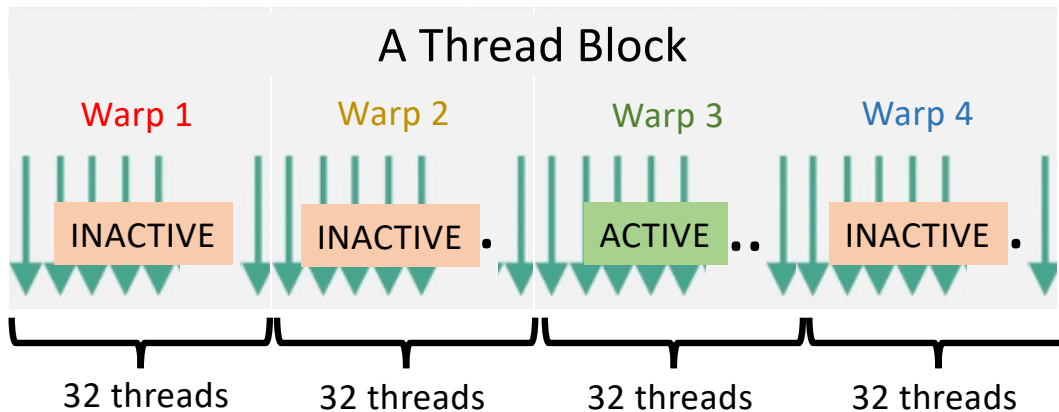
- The SM creates, manages, schedules, and executes threads in groups of 32 parallel threads called *warps*.
- Each warp gets scheduled by a **warp scheduler** for execution
- A warp executes **one common instruction** at a time



Execute warp 1's next instruction
(all 32 threads execute the same instruction)

How is a thread block executed on a SM?

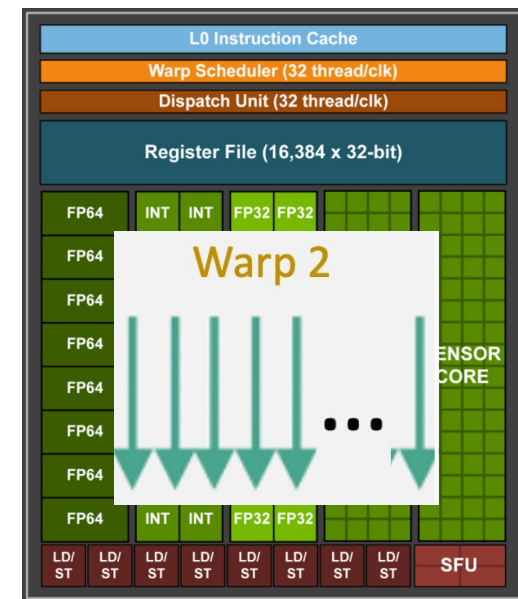
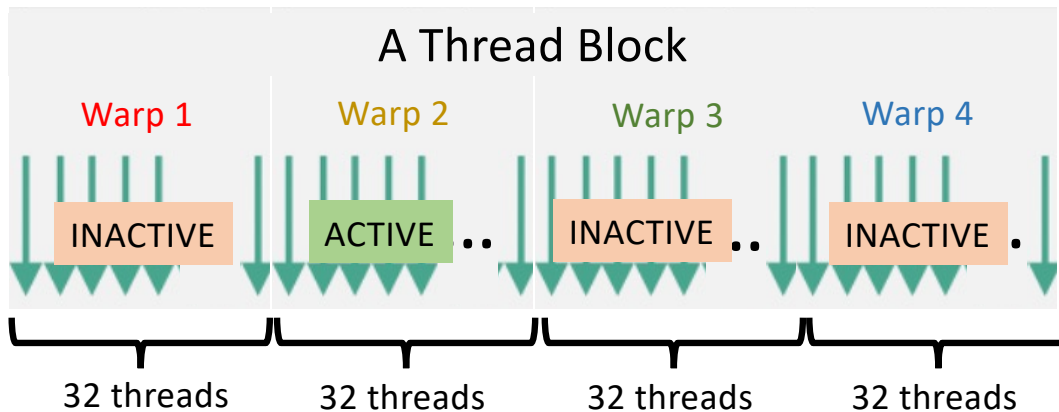
- The SM creates, manages, schedules, and executes threads in groups of 32 parallel threads called *warps*.
- Each warp gets scheduled by a **warp scheduler** for execution
- A warp executes **one common instruction** at a time



Swap out warp 1 and execute warp 3's next instruction

Mapping parallelization model to hardware

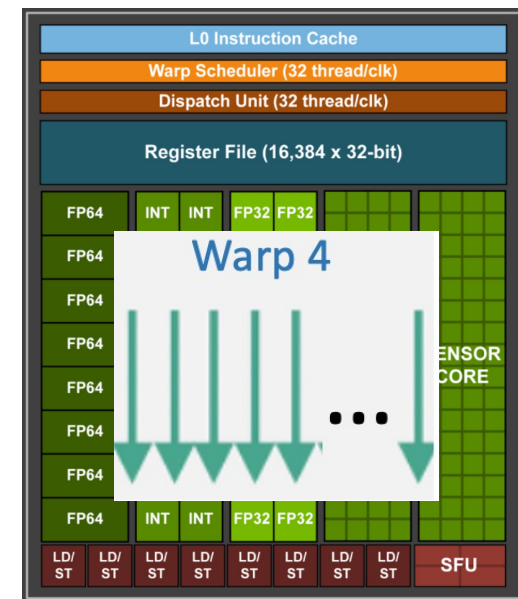
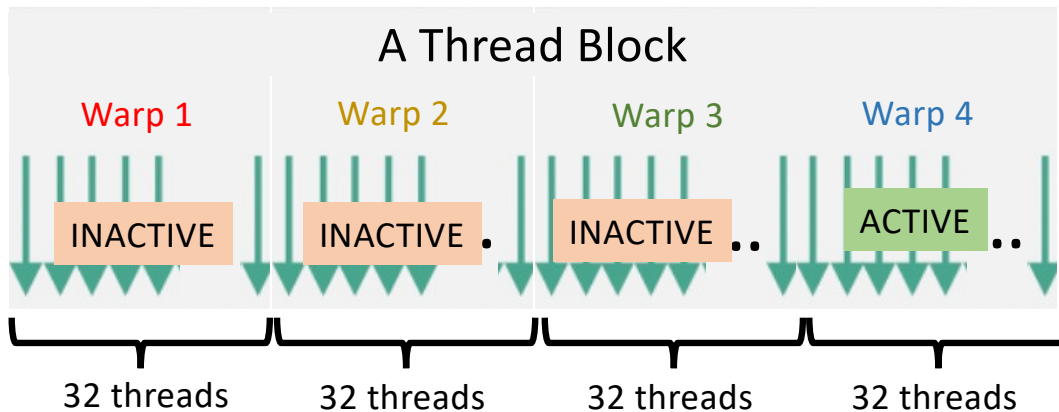
- The SM creates, manages, schedules, and executes threads in groups of 32 parallel threads called *warps*.
- Each warp gets scheduled by a **warp scheduler** for execution
- A warp executes **one common instruction** at a time



Swap out warp 3 and execute warp 2's next instruction

How is a thread block executed on an SM?

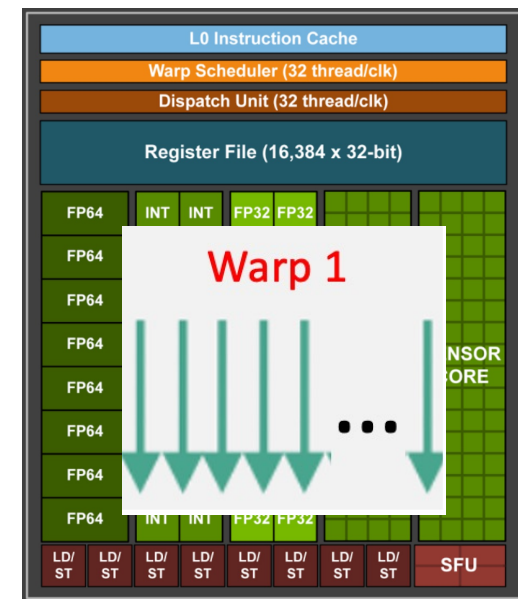
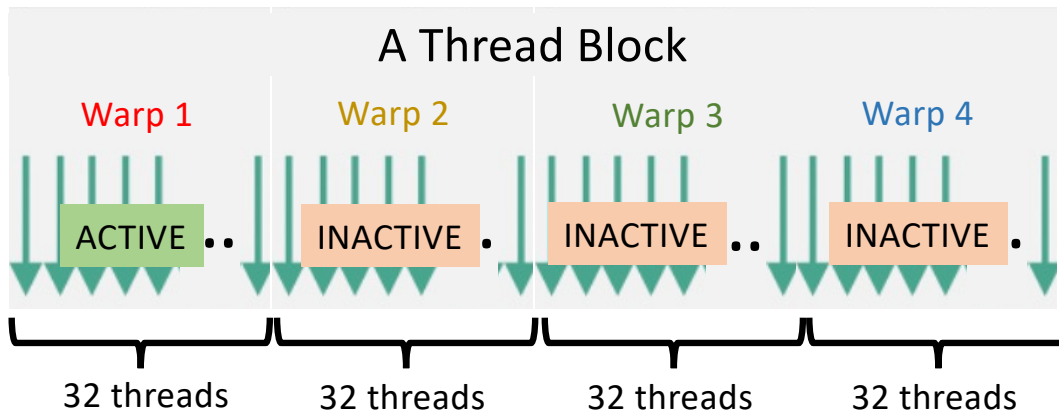
- The SM creates, manages, schedules, and executes threads in groups of 32 parallel threads called *warps*.
- Each warp gets scheduled by a **warp scheduler** for execution
- A warp executes **one common instruction** at a time



Swap out warp 2 and execute warp 4's instruction

How is a thread block executed on an SM?

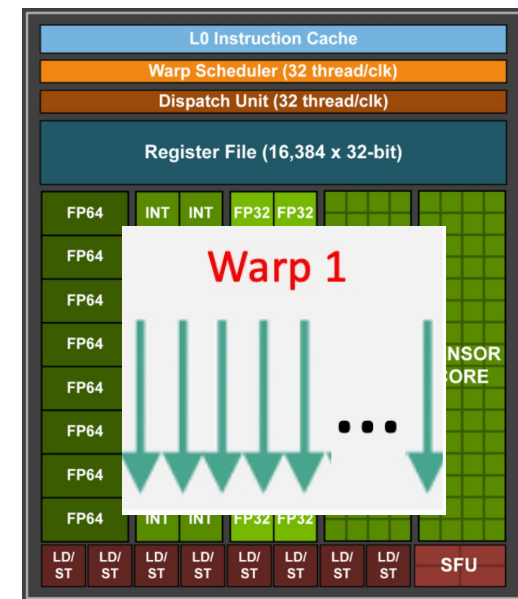
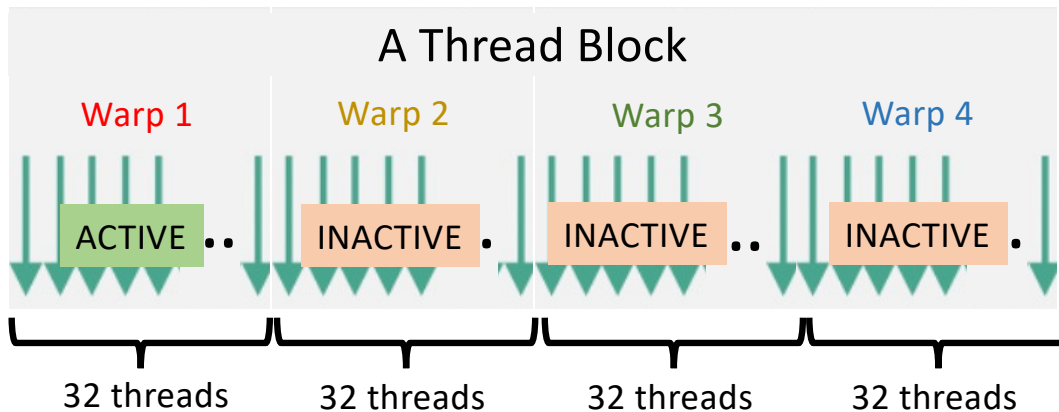
- The SM creates, manages, schedules, and executes threads in groups of 32 parallel threads called *warps*.
- Each warp gets scheduled by a **warp scheduler** for execution
- A warp executes **one common instruction** at a time



Swap out warp 4 and Execute warp 1's next instruction

How is a thread block executed on an SM?

- The SM creates, manages, schedules, and executes threads in groups of 32 parallel threads called *warps*.
- Each warp gets scheduled by a **warp scheduler** for execution
- A warp executes **one common instruction** at a time



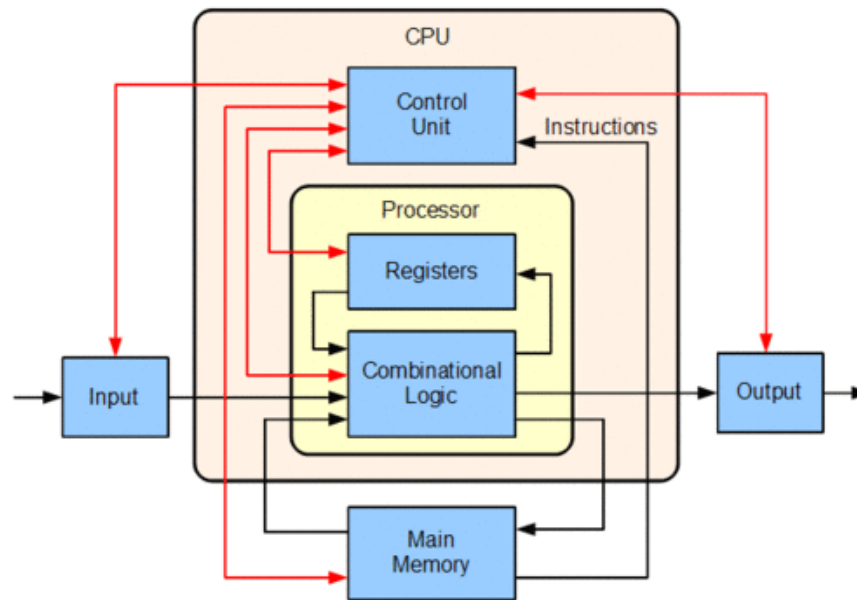
Swap out warp 4 and Execute warp 1's next instruction

Context switch

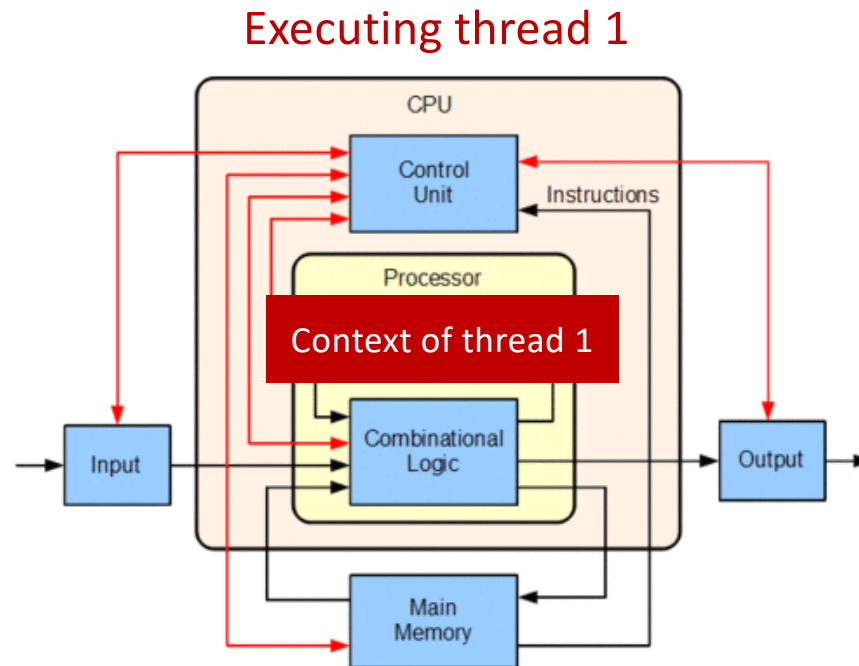
- Both CPU and GPU need context switch
- When multiple threads share limited cores, they need to take turns to use the cores for execution.
- **Context switch:** the process of storing the state of a thread, so that it can be restored and resume execution at a later point
- Context switch is designed to optimize processor utilization
 - All threads get fairly amount of executing time in a period
 - Some threads may need to wait for IO (memory, disk, network, etc) without computing needs



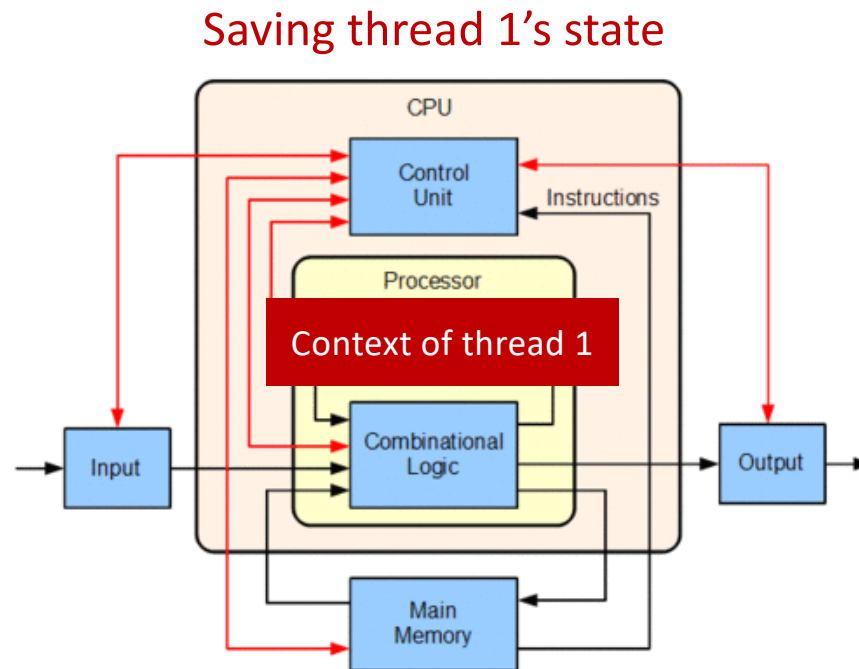
CPU Context Switch



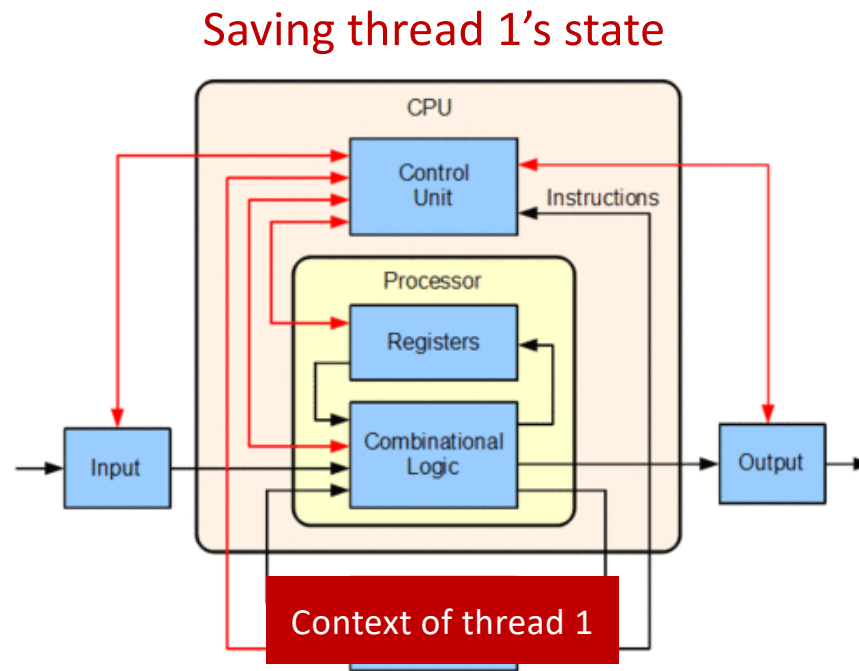
CPU Context Switch



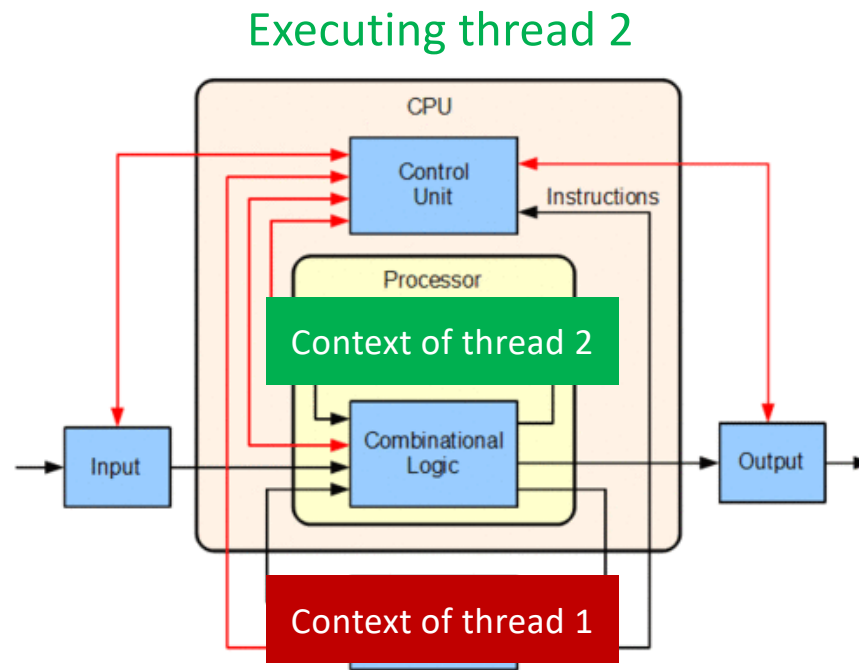
CPU Context Switch



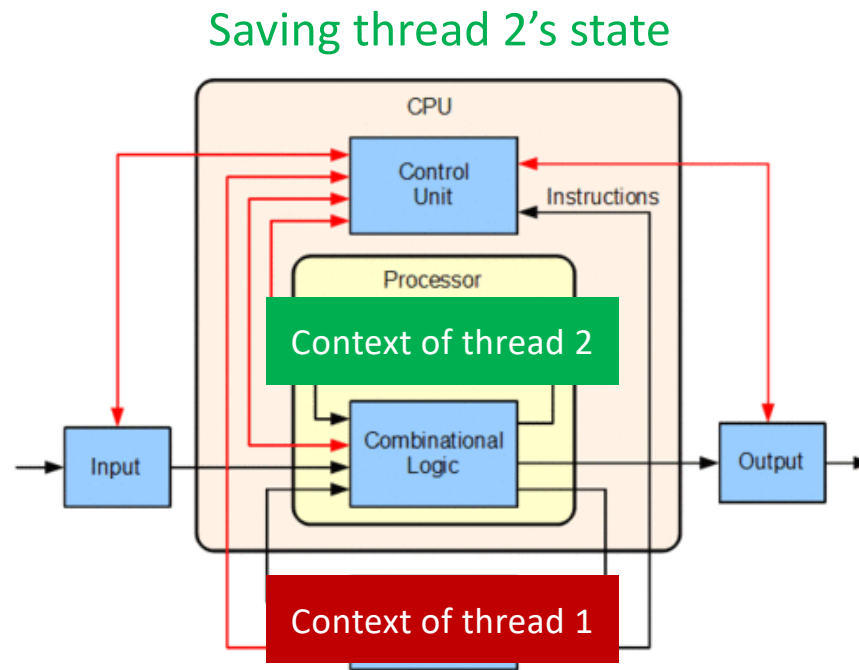
CPU Context Switch



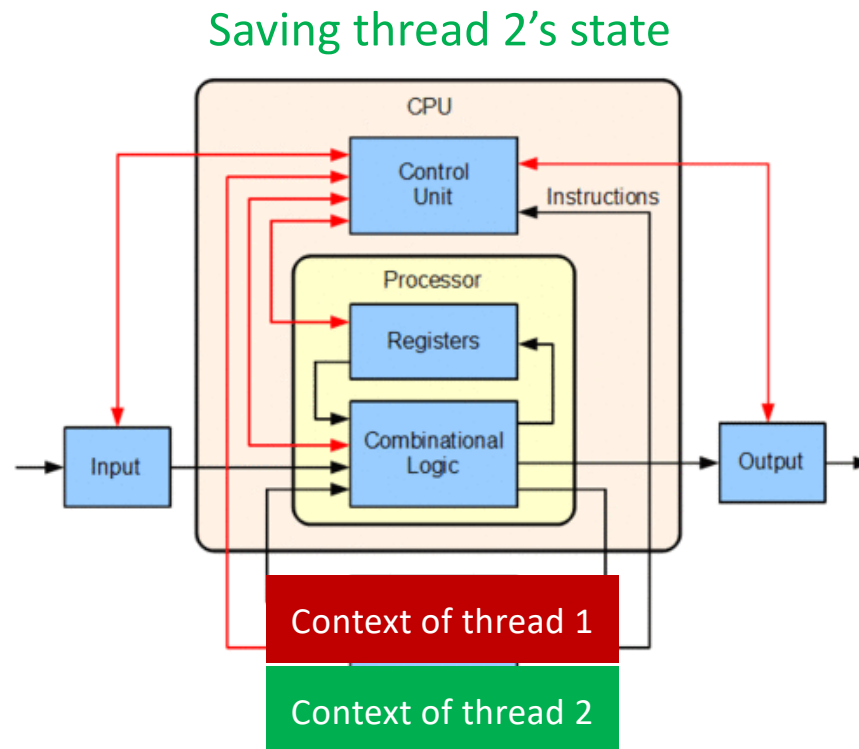
CPU Context Switch



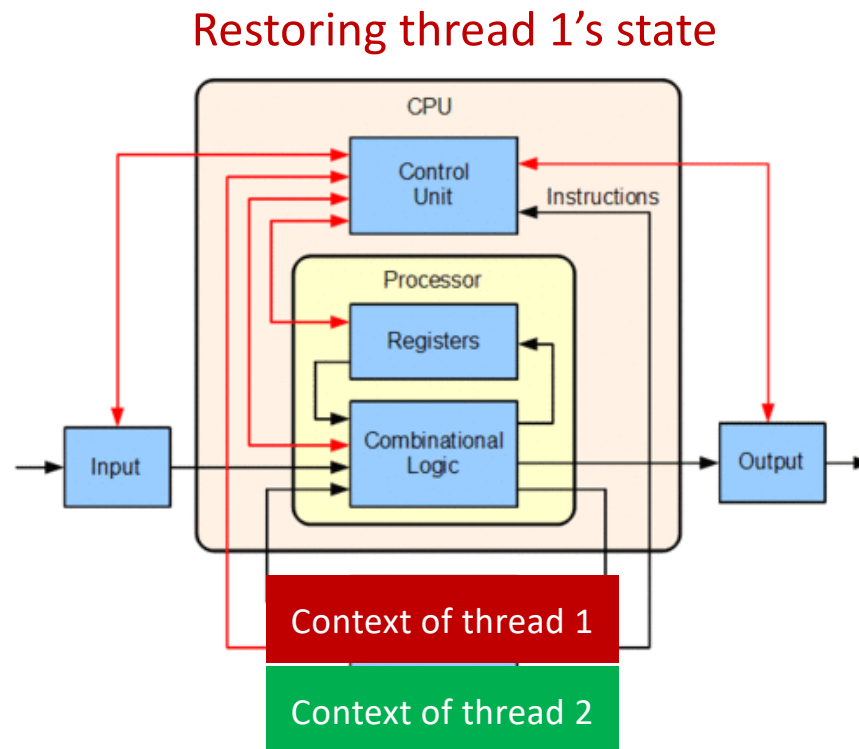
CPU Context Switch



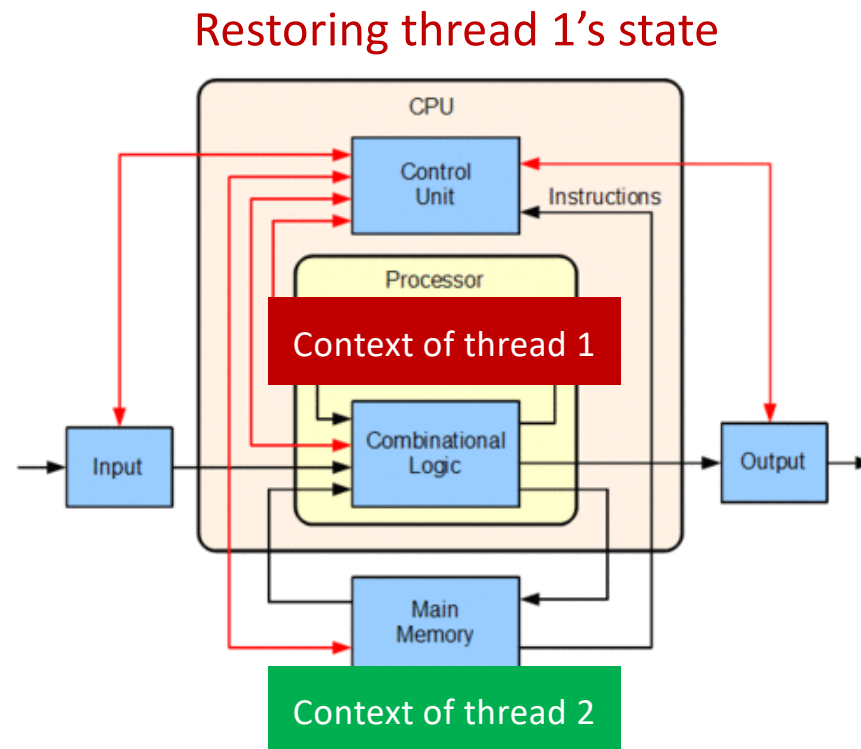
CPU Context Switch



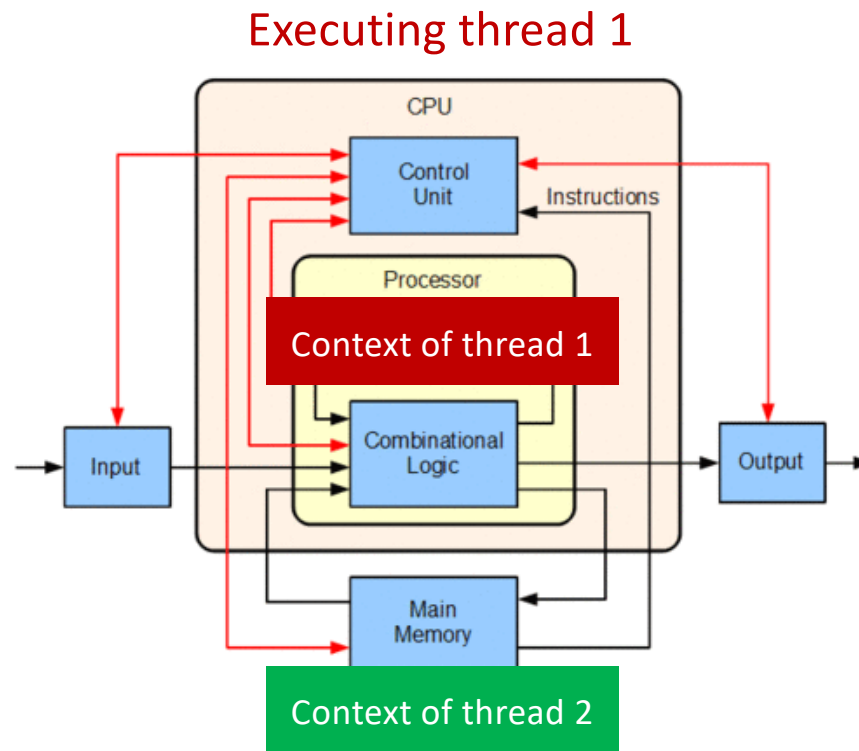
CPU Context Switch



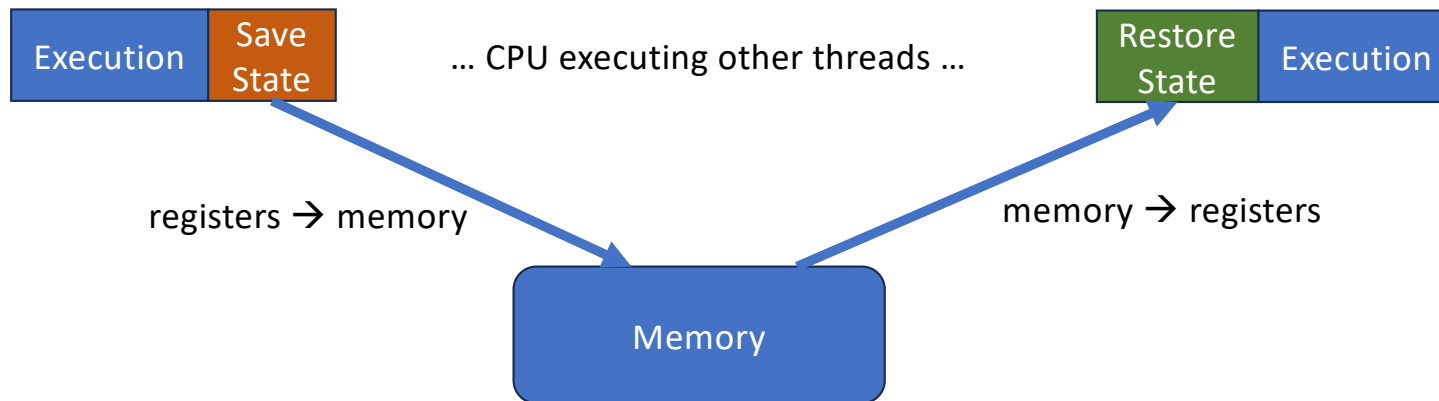
CPU Context Switch



CPU Context Switch



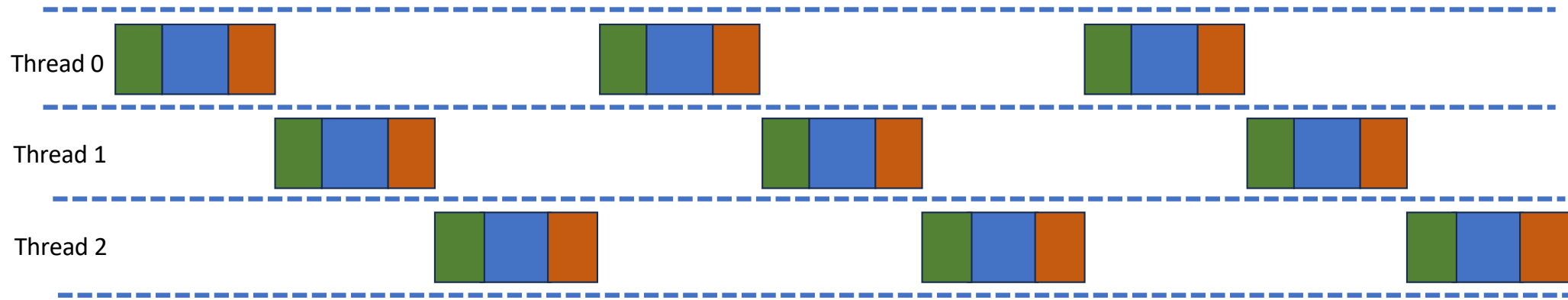
CPU Context Switch



data in all registers used by thread

- program counter
- stack pointer
- program data
- etc.

CPU Context Switch



Context Switch on CPU can be expensive!



UNIVERSITY OF
OREGON

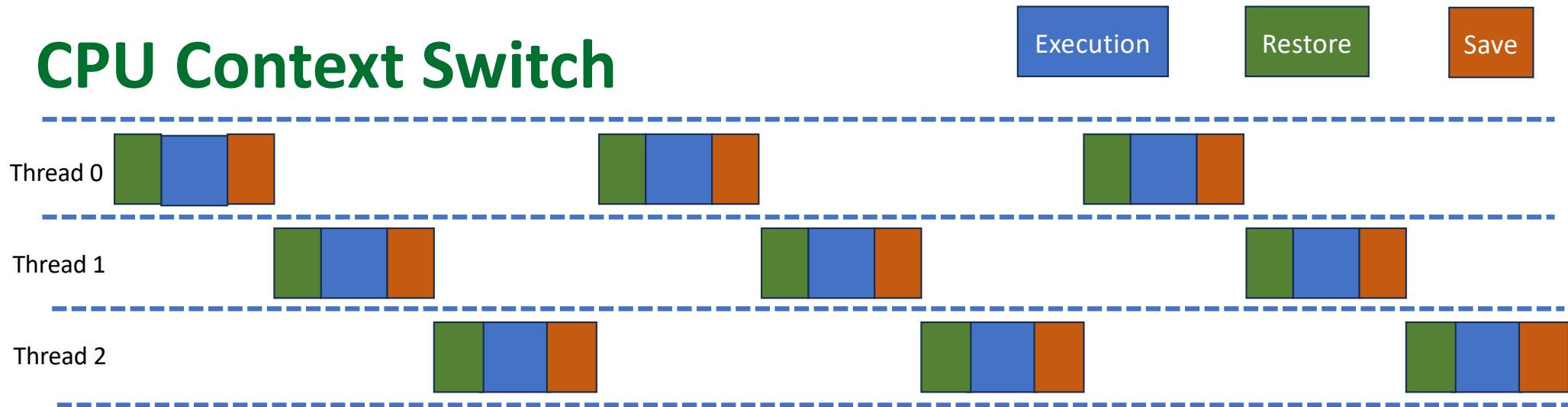
Computer Science

Execution

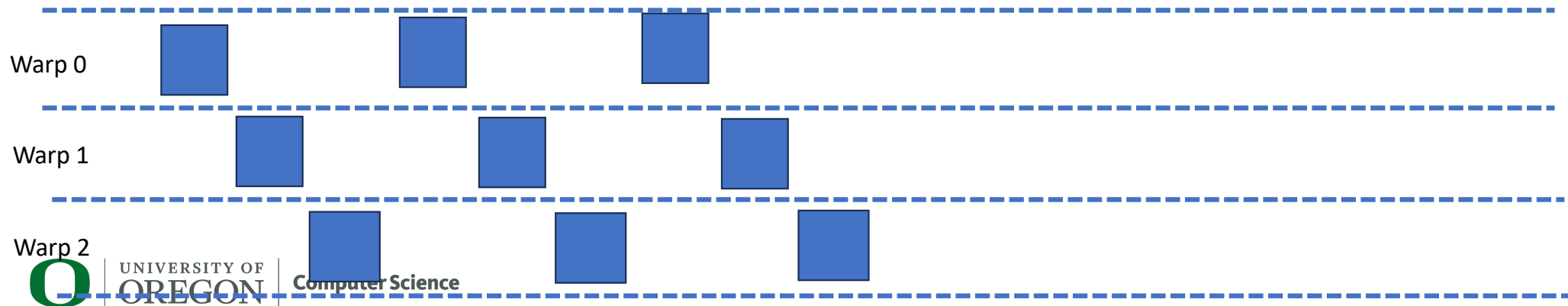
Restore

Save

CPU Context Switch

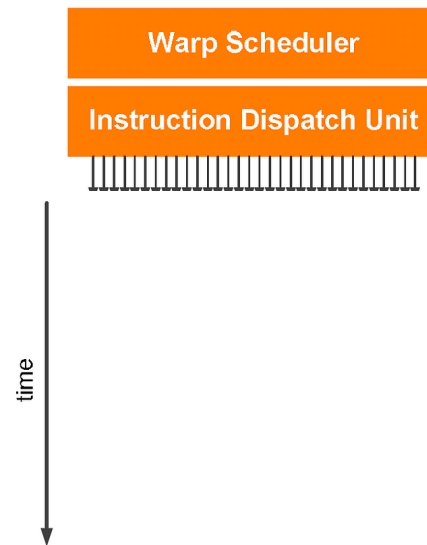


GPU Context Switch



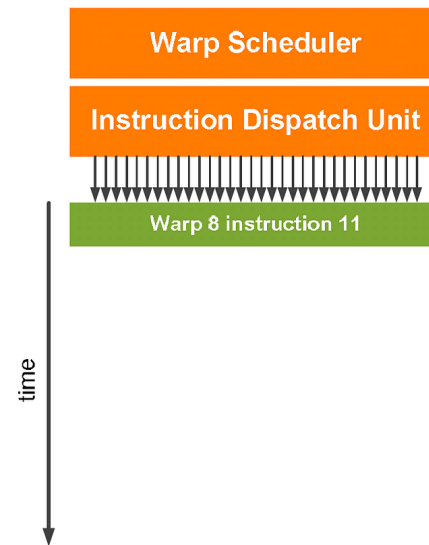
GPU's Warp scheduler

- Warp scheduler is designed to do context switch at **instruction-level granularity**
- Warp scheduler keeps monitoring the status of all warps (active and inactive)
- When the current instruction of a warp finished executing, warp scheduler will always try to execute the next instruction of an available warp regardless of which warp was running
- *There is **no performance penalty** choosing a different warp*



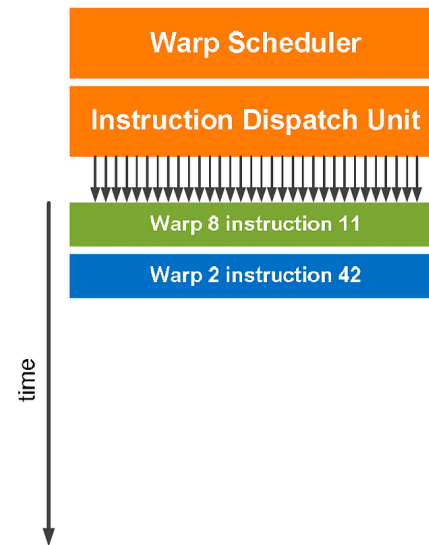
GPU's Warp scheduler

- Warp scheduler is designed to do context switch at **instruction-level granularity**
- Warp scheduler keeps monitoring the status of all warps (active and inactive)
- When the current instruction of a warp finished executing, warp scheduler will always try to execute the next instruction of an available warp regardless of which warp was running
- *There is **no performance penalty** choosing a different warp*



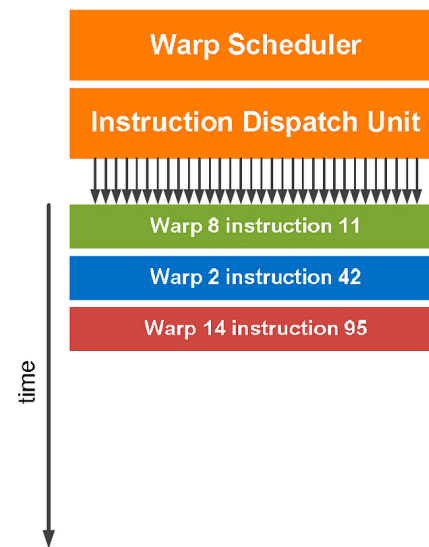
GPU's Warp scheduler

- Warp scheduler is designed to do context switch at **instruction-level granularity**
- Warp scheduler keeps monitoring the status of all warps (active and inactive)
- When the current instruction of a warp finished executing, warp scheduler will always try to execute the next instruction of an available warp regardless of which warp was running
- *There is **no performance penalty** choosing a different warp*



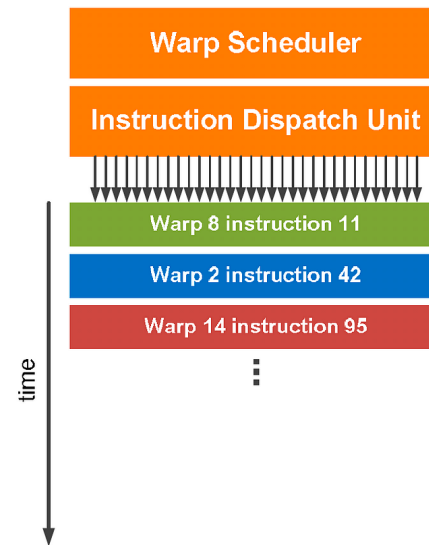
GPU's Warp scheduler

- Warp scheduler is designed to do context switch at **instruction-level granularity**
- Warp scheduler keeps monitoring the status of all warps (active and inactive)
- When the current instruction of a warp finished executing, warp scheduler will always try to execute the next instruction of an available warp regardless of which warp was running
- *There is **no performance penalty** choosing a different warp*



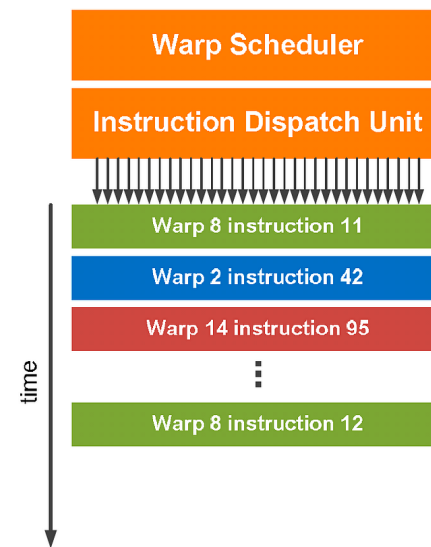
GPU's Warp scheduler

- Warp scheduler is designed to do context switch at **instruction-level granularity**
- Warp scheduler keeps monitoring the status of all warps (active and inactive)
- When the current instruction of a warp finished executing, warp scheduler will always try to execute the next instruction of an available warp regardless of which warp was running
- *There is **no performance penalty** choosing a different warp*



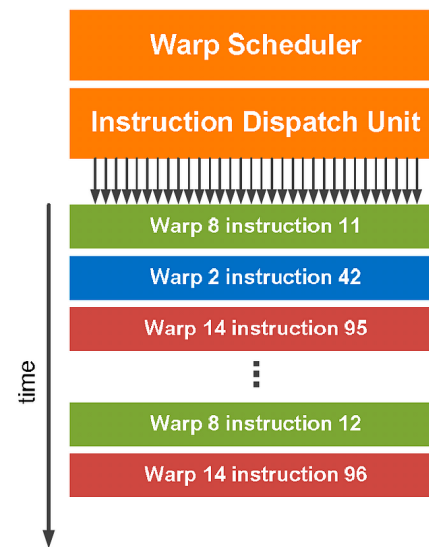
GPU's Warp scheduler

- Warp scheduler is designed to do context switch at **instruction-level granularity**
- Warp scheduler keeps monitoring the status of all warps (active and inactive)
- When the current instruction of a warp finished executing, warp scheduler will always try to execute the next instruction of an available warp regardless of which warp was running
- *There is **no performance penalty** choosing a different warp*



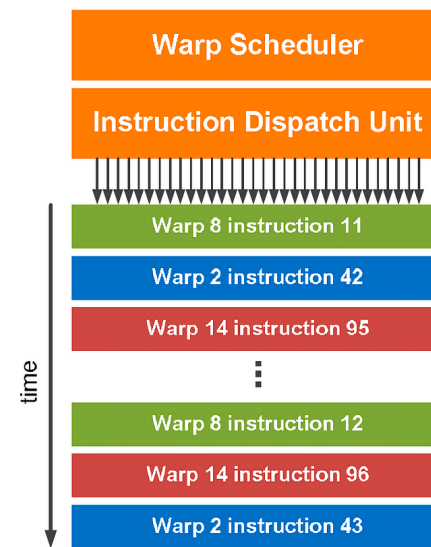
GPU's Warp scheduler

- Warp scheduler is designed to do context switch at **instruction-level granularity**
- Warp scheduler keeps monitoring the status of all warps (active and inactive)
- When the current instruction of a warp finished executing, warp scheduler will always try to execute the next instruction of an available warp regardless of which warp was running
- *There is **no performance penalty** choosing a different warp*



GPU's Warp scheduler

- Warp scheduler is designed to do context switch at **instruction-level granularity**
- Warp scheduler keeps monitoring the status of all warps (active and inactive)
- When the current instruction of a warp finished executing, warp scheduler will always try to execute the next instruction of an available warp regardless of which warp was running
- *There is **no performance penalty** choosing a different warp*

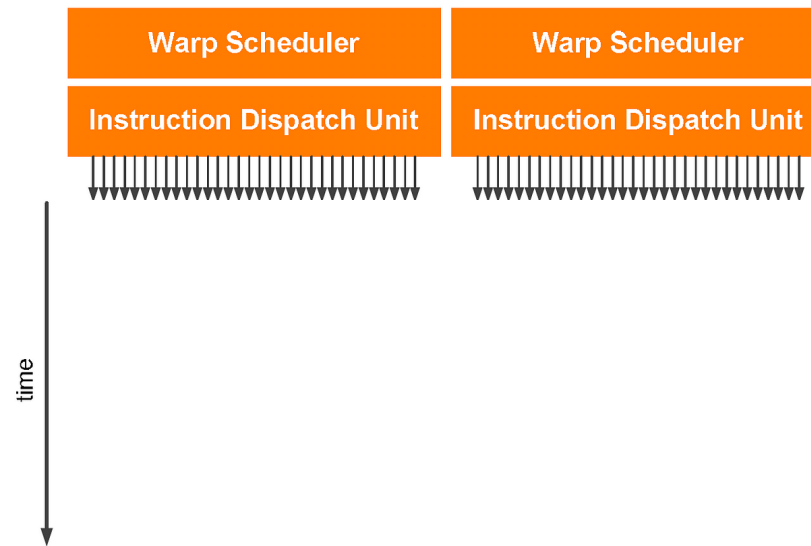


GPU's Warp scheduler

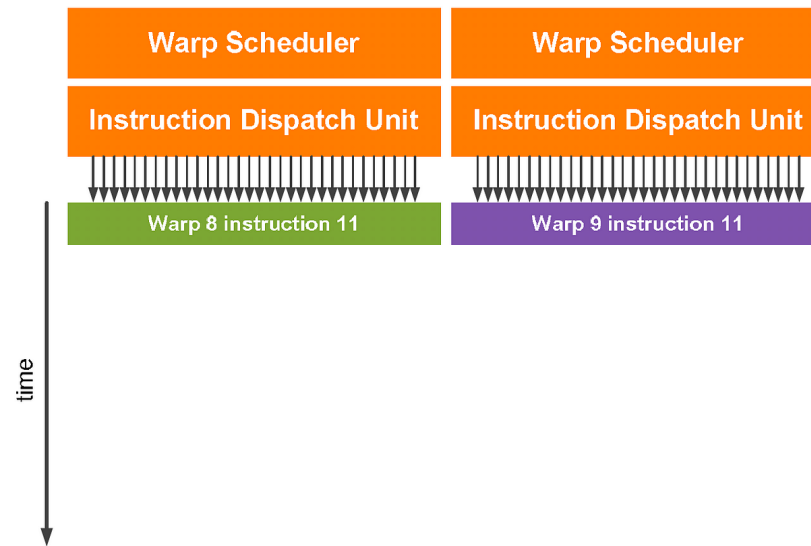
- A SM can execute **multiple warps** (active) at the same time depending on the resource capacity
- Each warp executes **independently** and
- Warps in a SM may come from **different** thread blocks
- A warp schedule can handle **multiple warps** and there may be **multiple warp schedulers** per SM
- For example, a SM in A100 has 4 warp schedulers
- More warp schedulers means the cores will be **more efficiently** used (avoid idle time)



GPU's Warp scheduler

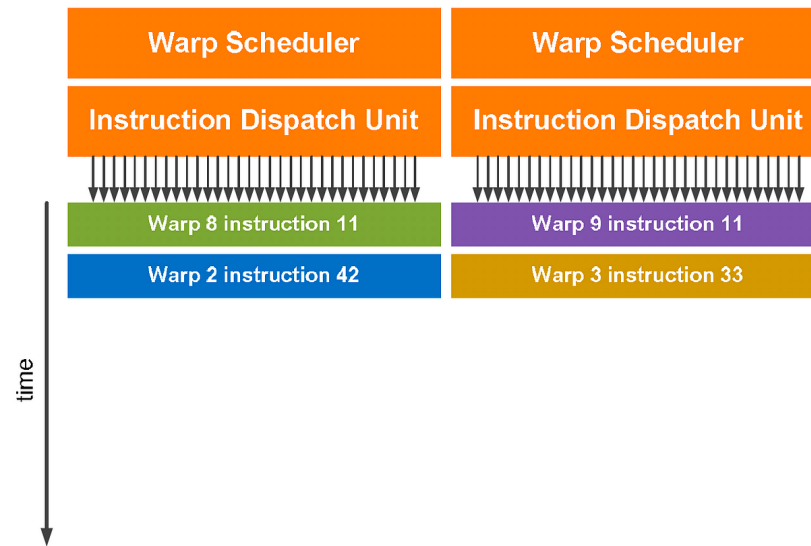


GPU's Warp scheduler

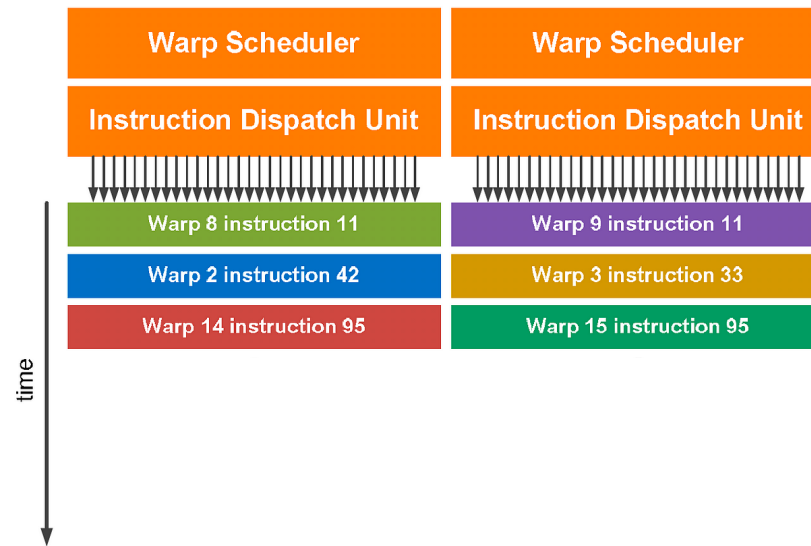


A SM can execute **multiple warps** at the same time

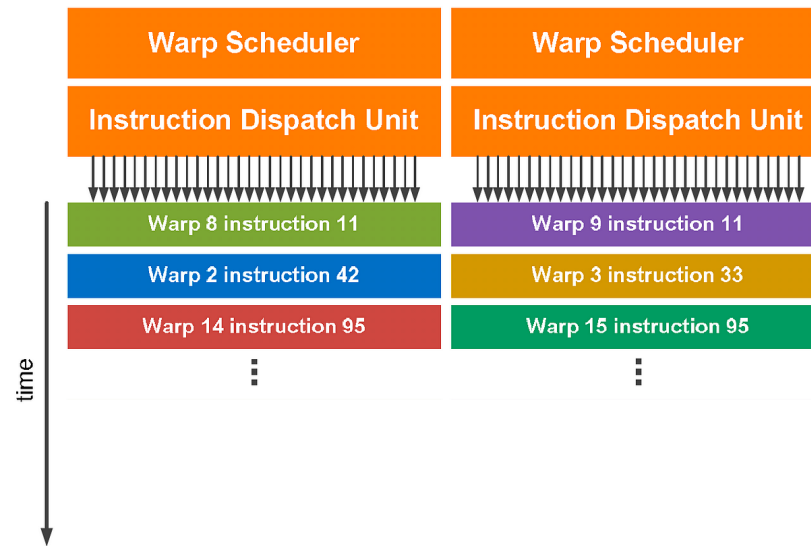
GPU's Warp scheduler



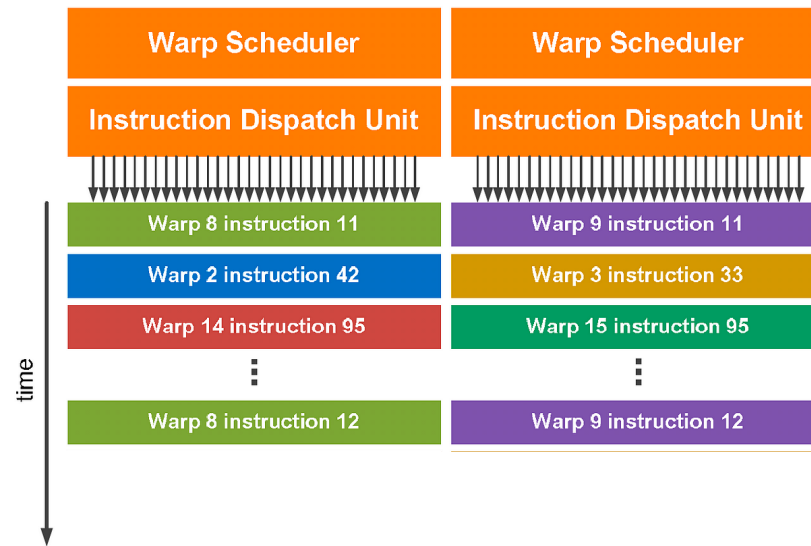
GPU's Warp scheduler



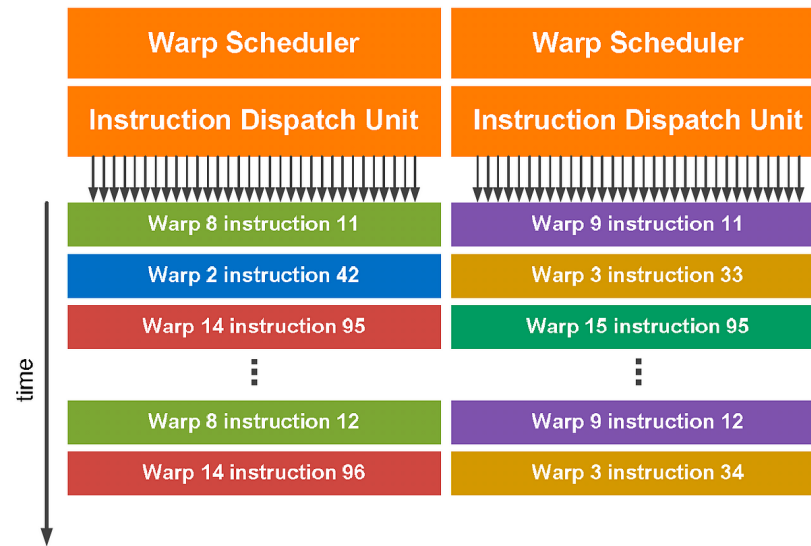
GPU's Warp scheduler



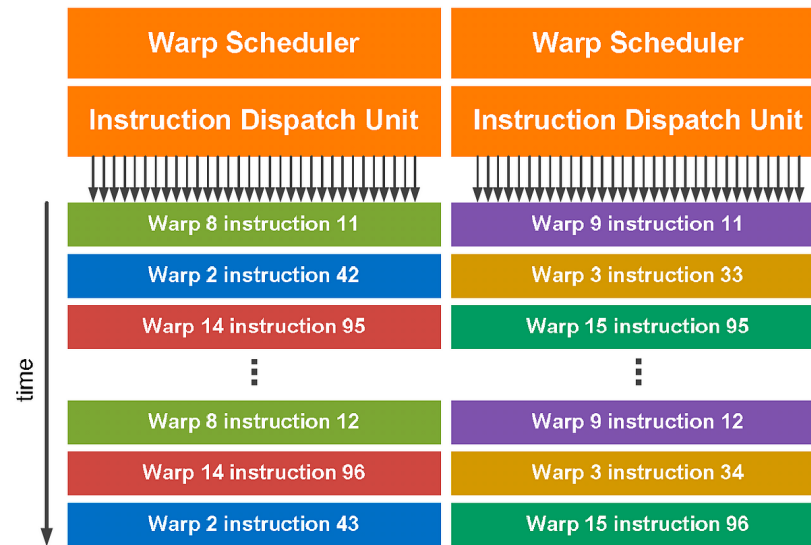
GPU's Warp scheduler



GPU's Warp scheduler



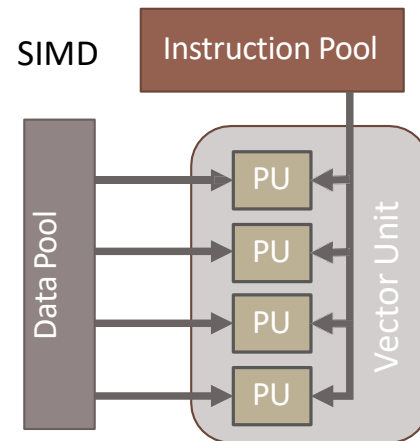
GPU's Warp scheduler



Warps

- ❑ Execution of GPU instructions is always in groups of threads called **warps**
- ❑ A warp has a single program counter (pre-volta)
- ❑ Within a warps threads always execute the same instruction (SIMD/SIMT)
- ❑ What happens if code within a warp has different control flow?

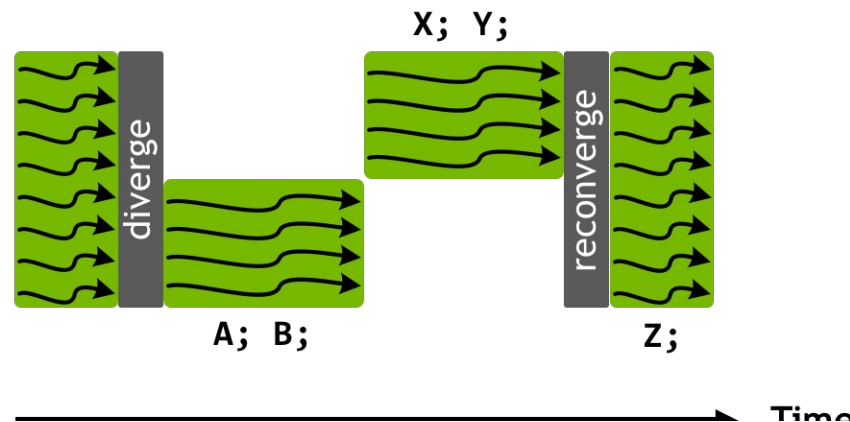
❑ Branch Divergence



Warp execution model

- A warp executes **one common instruction** at a time
- **Full efficiency**: all 32 threads agree on their execution path
- **Warp divergence**: If threads of a warp **diverge** via a conditional branch (if-else, for, while, etc.), the warp executes each branch path taken, disabling threads that are not on that path.

```
if (threadIdx.x < 4) {  
    A;  
    B;  
} else {  
    X;  
    Y;  
}  
Z;
```



- Branch divergence occurs **only within a warp**
- Different warps execute **independently** regardless of whether they are executing common or disjoint code paths.

Divergent Threads

- ❑ How to avoid divergence

1. Avoid conditional code
2. **Especially** avoid conditional code based on `threadIdx`

- ❑ Divergence-free code can still have branches

- ❑ BUT all threads in the warp follow the same path

Divergent and Coherent Code

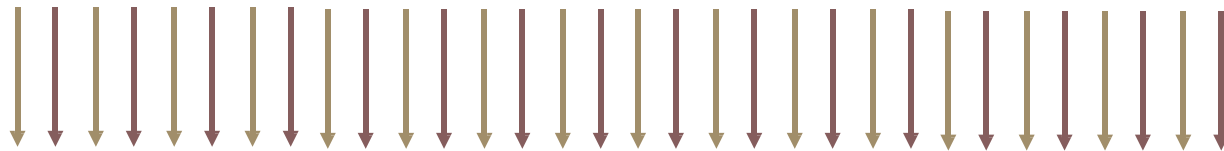
```
__global__ void a_kernel()  
{  
    if (blockIdx.x % 2)  
        //something  
    else  
        //something else  
}
```

```
__global__ void b_kernel()  
{  
    if (threadIdx.x % 2)  
        //something  
    else  
        //something else  
}
```

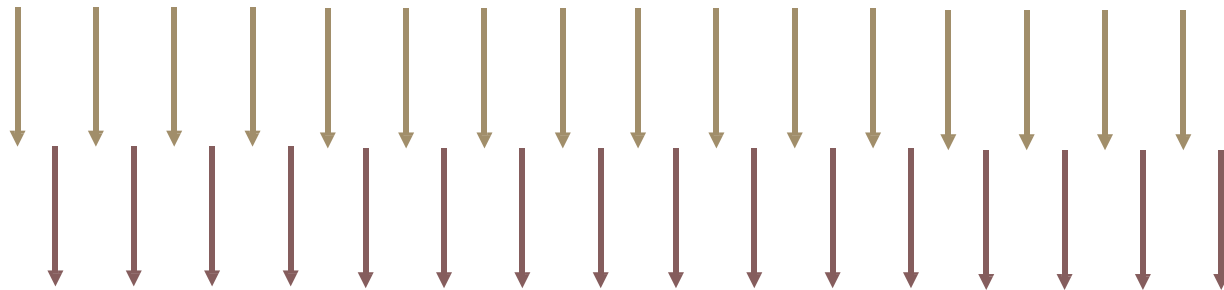
❑ Which has divergence?

Divergence Example

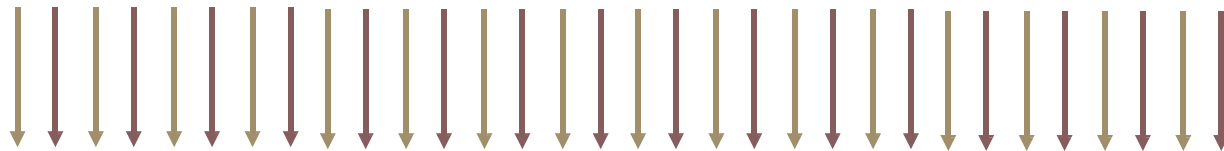
All warps



Branch (divergence)



End of Branch (convergence)



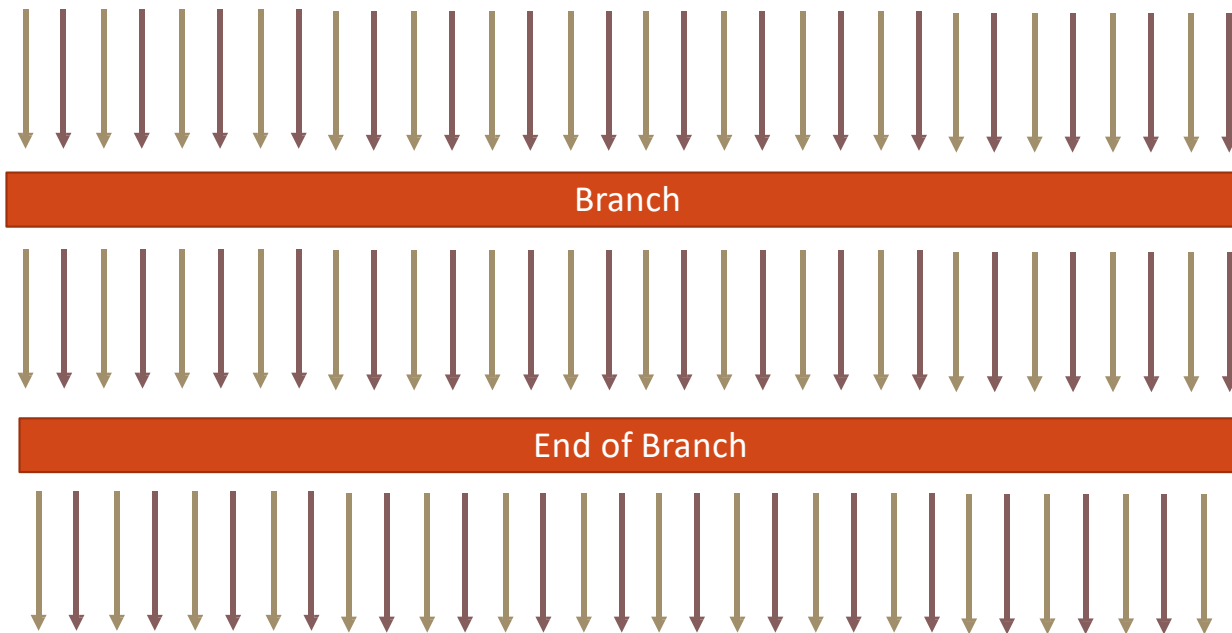
```
__global__ void a_kernel()  
{  
    if (threadIdx.x % 2)  
        //something  
    else  
        //something else  
}
```

`if (threadIdx.x % 2)`

`else`

Divergence Example Alternative

Warp 0

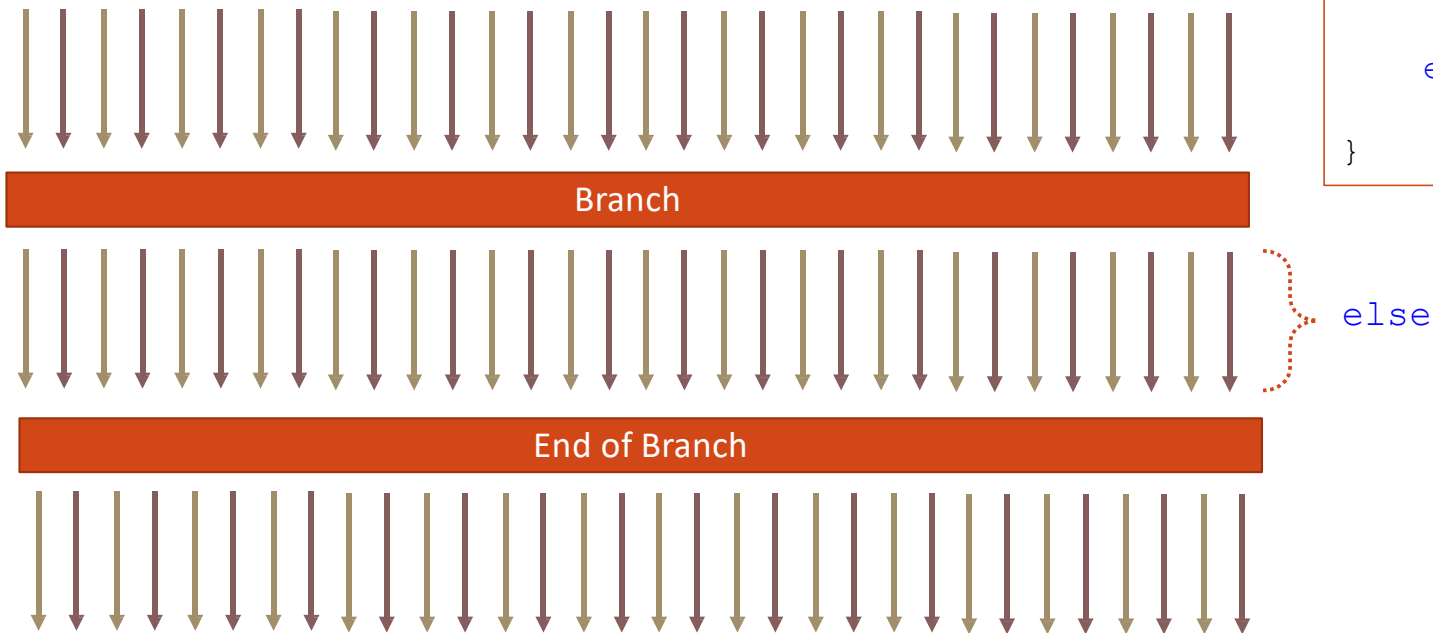


```
__global__ void a_kernel()  
{  
    if (blockIdx.x % 2)  
        //something  
    else  
        //something else  
}
```

`if (blockIdx.x % 2)`

Divergence Example Alternative

Warp 1 (assuming thread block size of 32)



```
__global__ void a_kernel()  
{  
    if (blockIdx.x % 2)  
        //something  
    else  
        //something else  
}
```


Levels of divergence

- ❑ Divergent code can be classified by how many “ways” it diverges.
 - ❑ E.g. the following examples are 4-way divergent (and functionally equivalent)
- ❑ If a warp has 32-way divergence this will have a **massive** impact on performance!

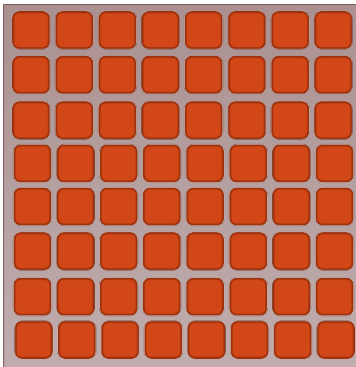
```
__global__ void a_kernel(int *A)
{
    int a = A[threadIdx.x + blockIdx.x*blockDim.x]
    if (a==0)
        //code for case 0
    else if (a==1)
        //code for case 1
    else if (a==2)
        //code for case 2
    else if (a==3)
        //code for case 3
}
```

```
__global__ void A_kernel(int *A)
{
    int a = A[threadIdx.x + blockIdx.x*blockDim.x]
    switch (a){
        case(0):
            //code for case 0 with break
        case(1):
            //code for case 1 with break
        case(2):
            //code for case 2 with break
        case(3):
            //code for case 3 with break
    }
}
```



2D blocks and divergence

Thread Block 0



```
__global__ void a_kernel()  
{  
    if (threadIdx.y % 2)  
        //something  
    else  
        //something else  
}
```

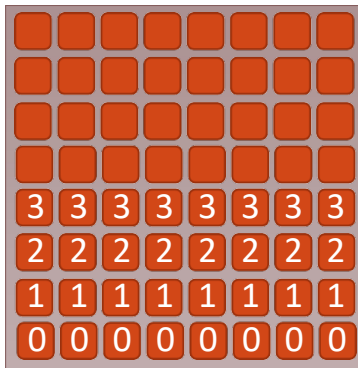
```
__global__ void b_kernel()  
{  
    if (threadIdx.y / 4)  
        //something  
    else  
        //something else  
}
```

□ How many ways of divergence?



2D blocks and divergence

Thread Block 0 – showing threadIdx.y



WARP 0

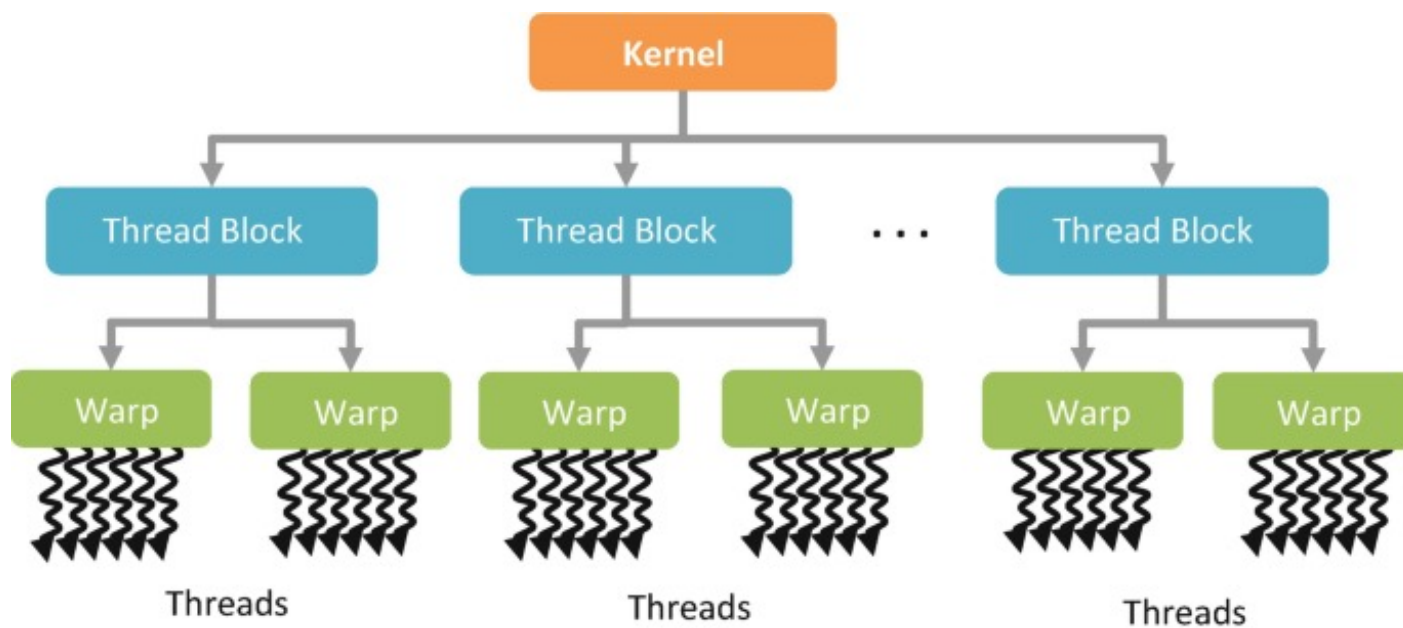
```
__global__ void a_kernel()  
{  
    if (threadIdx.y % 2)  
        //something  
    else  
        //something else  
}
```

2 way

```
__global__ void b_kernel()  
{  
    if (threadIdx.y / 4)  
        //something  
    else  
        //something else  
}
```

0 way
(none)

❑ How many ways of divergence?



HW mapping

GPU

SM

Warp scheduler

GPU cores